



ARTIFICIAL INTELLIGENCE FOR BUSINESS

A ROADMAP FOR GETTING STARTED WITH AI

JEFFREY L. COVEYDUC
JASON L. ANDERSON

WILEY

Artificial Intelligence for Business

Artificial Intelligence for Business

A Roadmap for Getting Started with AI

JEFFREY L. COVEYDUC

JASON L. ANDERSON

WILEY

© 2020 Jeffrey L. Coveyduc and Jason L. Anderson

Published by John Wiley & Sons, Inc., Hoboken, New Jersey.
Published simultaneously in Canada.

The right of Jason L. Anderson and Jeffrey L. Coveyduc to be identified as the author(s) of the editorial material in this work has been asserted in accordance with law.

No part of this publication may be reproduced, stored in a retrieval system, or transmitted in any form or by any means, electronic, mechanical, photocopying, recording, scanning, or otherwise, except as permitted under Section 107 or 108 of the 1976 United States Copyright Act, without either the prior written permission of the Publisher, or authorization through payment of the appropriate per-copy fee to the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, (978) 750-8400, fax (978) 646-8600, or on the Web at www.copyright.com. Requests to the Publisher for permission should be addressed to the Permissions Department, John Wiley & Sons, Inc., 111 River Street, Hoboken, NJ 07030, (201) 748-6011, fax (201) 748-6008, or online at www.wiley.com/go/permissions.

Limit of Liability/Disclaimer of Warranty: While the publisher and author have used their best efforts in preparing this book, they make no representations or warranties with respect to the accuracy or completeness of the contents of this book and specifically disclaim any implied warranties of merchantability or fitness for a particular purpose. No warranty may be created or extended by sales representatives or written sales materials. The advice and strategies contained herein may not be suitable for your situation. You should consult with a professional where appropriate. Neither the publisher nor author shall be liable for any loss of profit or any other commercial damages, including but not limited to special, incidental, consequential, or other damages.

For general information on our other products and services or for technical support, please contact our Customer Care Department within the United States at (800) 762-2974, outside the United States at (317) 572-3993, or fax (317) 572-4002.

Wiley publishes in a variety of print and electronic formats and by print-on-demand. Some material included with standard print versions of this book may not be included in e-books or in print-on-demand. If this book refers to media such as a CD or DVD that is not included in the version you purchased, you may download this material at <http://booksupport.wiley.com>. For more information about Wiley products, visit www.wiley.com.

Library of Congress Cataloging-in-Publication Data

Names: Anderson, Jason L, author. | Coveyduc, Jeffrey L, author.

Title: Artificial intelligence for business : a roadmap for getting started
with AI / Jason L. Anderson, Jeffrey L. Coveyduc.

Description: First edition. | Hoboken : Wiley, 2020. | Includes index.

Identifiers: LCCN 2020004359 (print) | LCCN 2020004360 (ebook) | ISBN 9781119651734 (hardback) | ISBN 9781119651413 (adobe pdf) | ISBN 9781119651802 (epub)

Subjects: LCSH: Artificial intelligence—Economic aspects. | Business enterprises—Technological innovations. | Artificial intelligence—Data processing.

Classification: LCC HC79.I55 .A527 2020 (print) | LCC HC79.I55 (ebook) | DDC 006.3068—dc23

LC record available at <https://lcn.loc.gov/2020004359>

LC ebook record available at <https://lcn.loc.gov/2020004360>

Cover Design: Wiley

Cover Image: © Yuichiro Chino/Getty Images

Printed in the United States of America

10 9 8 7 6 5 4 3 2 1

Contents

<i>Preface</i>	ix
<i>Acknowledgments</i>	xi
CHAPTER 1	
Introduction	1
Case Study #1: FANUC Corporation	2
Case Study #2: H&R Block	4
Case Study #3: BlackRock, Inc.	5
How to Get Started	6
The Road Ahead	10
Notes	11
CHAPTER 2	
Ideation	13
An Artificial Intelligence Primer	13
Becoming an Innovation-Focused Organization	23
Idea Bank	25
Business Process Mapping	27
Flowcharts, SOPs, and You	28
Information Flows	29
Coming Up with Ideas	31
Value Analysis	31
Sorting and Filtering	34
Ranking, Categorizing, and Classifying	35
Reviewing the Idea Bank	37
Brainstorming and Chance Encounters	38
AI Limitations	41
Pitfalls	44
Action Checklist	45
Notes	46

CHAPTER 3	Defining the Project	47
	The <i>What</i> , <i>Why</i> , and <i>How</i> of a Project Plan	48
	The Components of a Project Plan	49
	Approaches to Break Down a Project	53
	Project Measurability	62
	Balanced Scorecard	63
	Building an AI Project Plan	64
	Pitfalls	66
	Action Checklist	69
CHAPTER 4	Data Curation and Governance	71
	Data Collection	73
	Leveraging the Power of Existing Systems	81
	The Role of a Data Scientist	81
	Feedback Loops	82
	Making Data Accessible	84
	Data Governance	85
	Are You Data Ready?	89
	Pitfalls	90
	Action Checklist	94
	Notes	94
CHAPTER 5	Prototyping	97
	Is There an Existing Solution?	97
	Employing vs. Contracting Talent	99
	Scrum Overview	101
	User Story Prioritization	103
	The Development Feedback Loop	105
	Designing the Prototype	106
	Technology Selection	107
	Cloud APIs and Microservices	110
	Internal APIs	112
	Pitfalls	112
	Action Checklist	114
	Notes	114
CHAPTER 6	Production	117
	Reusing the Prototype vs. Starting from a Clean Slate	117

	Continuous Integration	119
	Automated Testing	124
	Ensuring a Robust AI System	128
	Human Intervention in AI Systems	129
	Ensure Prototype Technology Scales	131
	Cloud Deployment Paradigms	133
	Cloud API's SLA	135
	Continuing the Feedback Loop	135
	Pitfalls	135
	Action Checklist	137
	Notes	137
CHAPTER 7	Thriving with an AI Lifecycle	139
	Incorporate User Feedback	140
	AI Systems Learn	142
	New Technology	144
	Quantifying Model Performance	145
	Updating and Reviewing the Idea Bank	147
	Knowledge Base	148
	Building a Model Library	150
	Contributing to Open Source	155
	Data Improvements	157
	With Great Power Comes Responsibility	158
	Pitfalls	159
	Action Checklist	161
	Notes	161
CHAPTER 8	Conclusion	163
	The Intelligent Business Model	164
	The Recap	164
	So What Are You Waiting For?	168
APPENDIX A	AI Experts	169
	AI Experts	169
	Chris Ackerson	169
	Jeff Bradford	173
	Nathan S. Robinson	175
	Evelyn Duesterwald	177

Jill Nephew	179
Rahul Akolkar	183
Steven Flores	187
APPENDIX B Roadmap Action Checklists	191
Step 1: Ideation	191
Step 2: Defining the Project	191
Step 3: Data Curation and Governance	192
Step 4: Prototyping	192
Step 5: Production	193
Thriving with an AI Lifecycle	193
APPENDIX C Pitfalls to Avoid	195
Step 1: Ideation	195
Step 2: Defining the Project	196
Step 3: Data Curation and Governance	199
Step 4: Prototyping	203
Step 5: Production	204
Thriving with an AI Lifecycle	206
<i>Index</i>	209

Preface

Artificial intelligence (AI) has become so ingrained in our daily lives that most people knowingly leverage it every day. Whether interacting with an artificial “entity” such as the iPhone assistant Siri, or browsing through Netflix’s recommendations, our functional adoption of machine learning is already well under way. Indirectly, however, AI is even more prevalent. Every credit card purchase made is run through fraud detection AI to help safeguard customers’ money. Advanced logistical scheduling software is used to deliver tens of millions of packages daily, to locales around the world, with minimal disruption. In fact, the e-commerce giant Amazon alone claims to have shipped 5 billion packages with Prime in 2017 (see businesswire.com/news/home/20180102005390/en/). None of this would be possible on such a grand scale without the advances we have seen in AI systems and in machine learning technology over the last few decades.

Historically, these AI systems have been developed in-house by skilled teams of programmers, working around the clock at great expense to employers. This reality is now changing. Companies like IBM, Google, and Microsoft are making AI capabilities available on a pay-as-you-go basis, dramatically lowering the barrier to entry. For example, each of these companies provide speech-to-text and text-to-speech services to easily build voice interfaces for pennies per use. This is opening the door for smaller companies with less disposable capital to introduce AI initiatives that will produce substantial results. With the aforementioned backdrop of consumers interacting with AI on a daily basis, these consumers are becoming increasingly more comfortable and receptive to their brands adopting and incorporating more AI technology. The combination of all of these

components makes it a smart bet for any modern company to start down the road toward AI adoption.

But how do these companies get started? This question is one we have seen time and again working with clients in the AI space. The drive and enthusiasm are there, but what organizational thought leaders are missing is the “how to” and overall direction. In our day jobs working with IBM Watson Client Engagement Centers and clients around the world, we repeatedly saw this pattern play out. Clients were eager to incorporate AI systems into their business models. They understood many of the benefits. They just needed a way in. While attending tech conferences and meetups, we find similar stories as well. Though the technological barriers are lower, with vendors providing accessible AI technology in the cloud, the challenge of coming up with the overall plan was still preventing many businesses from adopting AI. Having a good roadmap is essential to feeling comfortable with starting the journey. It is for this very reason that we wrote this book. Our goal is to empower you with the knowledge to successfully adopt AI technology into your organization. And you’ve already taken the first step by opening this book.

In addition to helping you adopt and understand emerging AI technology, this book will give you the tools to use AI to make a measurable impact in your business. Perhaps you will find some new cost-saving opportunities to unlock. Maybe AI will allow your business to uniquely position itself to enter new markets and take on competitors. Although AI has become more widespread and mainstream in its use in recent years, we are still seeing a tremendous amount of room for disruption in every field. That’s the great thing about AI—it can be applied in an interdisciplinary fashion to all domains, and the more it grows, the more its capabilities grow along with it. All that we ask of you, the reader, is to start with an open mind while we provide that missing roadmap to help you successfully navigate your way to driving value within your organization using AI.

Acknowledgments

This book would not have been possible without the help of the following:

- All of our AI experts, who kindly contributed their knowledge to provide a snapshot of AI
- Nick Zephyrin, for his amazing book edits, which have kept our message consistent
- Wiley's production team, for helping us get this book out and in the hands of the world
- Our families (especially our wives, Denise and Libby), for all of their support throughout our careers
- All of our friends, especially Jen English, who read early drafts and provided feedback along the way
- IBM and Comp Three, for providing ample opportunities for learning and education

Introduction

The modern era has embedded code in everything we use. From your washing machine to your car, if it was made any time in the last decade, there is likely code inside it. In fact, the term “Internet of Things (IoT)” has emerged to define all Internet-connected devices that are not strictly computers. Although the code on these IoT devices is becoming smarter with every upgrade, the devices are not exactly learning autonomously. A programmer has to code every new feature or decision into a model. These programs do not learn from their mistakes. Advancement in AI will help solve this problem, and soon we will have devices that *will* learn from the input of their human creators, as well as from their own mistakes. Today we are surrounded by code, and in the near future, we will be surrounded by embedded artificially intelligent agents. This will be a massive opportunity for upgrades and will enable more convenience and efficiency.

Although companies may have implemented software projects on their own or with the help of outside vendors in the past, AI projects have their own set of quirks. If those quirks are not managed properly, they may cause a project to be a failure. A brilliant idea must be paired with brilliant execution in order to succeed. Following the path laid out in this book will put you on a trajectory toward managing AI projects more efficiently, as well as prepare you for the

age of intelligent systems. Artificial intelligence is very likely to be the next frontier of technology, and in order for us to maximize this opportunity, the groundwork must be laid today.

Every organization is different, and it is important to remember not to try to apply techniques like a straitjacket. Doing so will suffocate your organization. This book is written with a mindset of best practices. Although best practices will work in most cases, it is important to remain attentive and flexible when considering your own organization's transformation. Therefore, you must use your best judgment with each recommendation we make. There is no one-size-fits-all solution, especially not in a field like AI that is constantly evolving.

Ahead of the recent boom in AI technologies, many organizations have already successfully implemented intelligent solutions. Most of these organizations followed an adoption roadmap similar to the one we will describe in this book. It is insightful for us to take a look at a few of these organizations, see what they implemented, and take stock of the benefits they are now realizing. As you read through these organizations' stories, keep in mind that we will be diving into aspects of each approach in more detail during the course of this book.

Case Study #1: FANUC Corporation

Science fiction has told of factories that run entirely by themselves, constantly monitoring and adjusting their input and output for maximum efficiency. Factories that can do just-in-time (JIT) ordering based on sales demand, sensors that predict maintenance requirements, the ability to minimize downtime and repair costs—these are no longer concepts of speculative fiction. With modern sensors and AI software, it has become possible to build these efficient, self-bolstering factories. Out-of-the-box IoT equipment can do better monitoring today than industrial sensors from 10 years ago. This leap in accuracy and connectivity has increased production threshold limits, enabling industrial automation on a scale never before imagined.

FANUC Corporation of Japan,¹ a manufacturer of robots for factories, leads by example. Its own factories have robots building other robots with minimal human intervention. Human workers are able to focus on managerial tasks, whereas robots are built in the dark. This gives a whole new meaning to the industry saying “lights-out operations,” which originally meant servers, not robots with moving parts, running independently in a dark data center. FANUC Japan has invested in Preferred Networks Inc. to gather data from their own robots to make them more reliable and efficient than ever before. Picking parts from a bin with an assortment of different-sized parts mixed together has been a hard problem to solve with traditional coding. With AI, however, FANUC has managed to achieve a consistent 90 percent accuracy in part identification and selection, tested over some 5,000 attempts. The fact that minimal code has gone into allowing these robots to achieve their previously unobtainable objective is yet another testament to the robust capabilities of AI in the industrial setting. FANUC and Preferred Networks have leveraged the continuous stream of data available to them from automated plants, underlining the fact that data collection and analysis is critical to the success of their factory project. FANUC Intelligent Edge Link & Drive (FIELD) is the company’s solution for data collection to be later implemented using deep learning models. The AI Bin-Picking product relies on models created via the data collected from the FIELD project. Such data collection procedures form a critical backbone for any industrial process that needs to be automated.

FANUC has also enabled deep learning² models for situations where there are too many parameters to be fine-tuned manually. Such models include AI servo-tuning processes that enable high-precision, high-speed machining processes that were not possible until recently. In the near future, your Apple iPhone case will probably be made using a machine similar to the one in Figure 1.1.

Most factories today are capable of utilizing these advancements with minor modifications to their processes. The gains that can be achieved from such changes will be able to exponentially elevate the output of any factory.



FIGURE 1.1 Example of a FANUC Robot³

Case Study #2: H&R Block

H&R Block is a U.S.-based company that specializes in tax preparation services. One of their customer satisfaction guarantees is to find the maximum number of tax deductions for each of their customers. Some deductions are straightforward, such as homeowners being able to deduct the mortgage interest on their primary residence. Other deductions, however, may be dependent on certain client-specific variables, such as the taxpayer's state of residence. Deduction complexity can then be further compounded by requiring multiple client-dependent variables to be considered simultaneously, such as a taxpayer with multiple sources of income who also has multiple personal deductions. The ultimate result is that maximizing deductions for a given customer can be difficult, even for a seasoned tax professional. H&R Block saw an opportunity to leverage AI to help their tax preparers optimize their service. In order to help facilitate the adoption process, H&R Block partnered with IBM to leverage their Watson capabilities.⁴

When a customer comes into H&R Block, the tax preparer engages them in a friendly discussion. “Have you experienced any life-changing events in the last year?” “Have you purchased a home?” and so on. As they talk, the tax preparer types relevant details of the conversation into their computer system to be used as reference later. If the customer mentions that they purchased a house last year, that will be an indicator that they may qualify for a mortgage interest deduction this year.

H&R Block saw the opportunity here to leverage the use of AI to compile, cross-reference, and analyze all of these notes. Natural language processing (NLP) can be applied to identify the core intent of each note, which then can be fed into the AI system to automatically identify possible deductions. The system then presents the tax professionals with any potentially relevant information to ensure that they do not miss any possible deductions. In the end, both tax professionals and their customers can enjoy an increased sense of confidence that every last applicable deduction was found.

Case Study #3: BlackRock, Inc.

Financial markets are a hotbed for data. The data can be collected accurately and in real time for most financial instruments (stocks, options, funds, etc.) listed on stock markets. Metadata (data about data) can also be curated from analytical reports, articles, and the like. The necessity for channeling the sheer amount of information that is generated every day has given rise to professional data stream providers like Bloomberg. The immense quantity of data available, along with the potential for trend prediction, growth estimations, and increasingly accurate risk assessment, makes the financial industry ripe for implementing AI projects.

BlackRock, Inc., one of the world’s largest asset managers, deploys the Aladdin⁵ (Asset, Liability, Debt, and Derivative Investment Network) software, which calculates risks, analyzes financial data, supports investment operations, and offers trade executions. Aladdin’s key strength lies in using the vast amount of data to arrive at models of risk that give the user more confidence in deploying investments and hedging. The project was started nearly two decades ago, and it

has been one of the key drivers of growth at BlackRock. BlackRock's technology services revenue grew 19 percent in 2018, driven by Aladdin and their other digital wealth products.⁶ Aladdin is now used by more than 25,000 investment professionals and 1,000 developers globally, helping to manage around \$18 trillion in assets.⁷ Aladdin embeds within itself the building blocks of AI through the use of applied mathematics and data science.

BlackRock is now setting up a laboratory to further study the applications of AI in the analysis of risk and data streams generated. The huge amount of data being generated is becoming a problem for analysts, since the amount of data a human can sift through is limited. The expectation of Rob Goldstein, BlackRock's chief operating officer, is that the AI lab will help increase the efficiencies in what BlackRock does across the board.⁸ By applying big data to their existing data trove, BlackRock will be able to generate higher alphas, a measure of excess return over other portfolio managers, according to David Wright, head of product strategy in Europe. With good data generated by Aladdin and a sufficiently advanced AI algorithm, BlackRock might just emerge as the leader in analyzing risk and portfolios.

How to Get Started

The journey to adopt AI promises to bring major changes to the way your organization thinks and approaches its future. This journey will involve the adoption of new methods and process improvements that will aid you in spotting the novel ways AI can be deployed to save costs and make available new opportunities.

As with any endeavor worth starting, we must make plans for how we intend to accomplish our goal. In this case, the goal is to adopt AI technologies to better our organization. The plan for achieving this goal can vary from organization to organization, but the main steps invariably remain the same (see Figure 1.2).

1. Ideation

The first step in any technology adoption journey must start with ideation and identifying your motivation. In this chapter, we will delve into answering questions such as “What problem are you trying

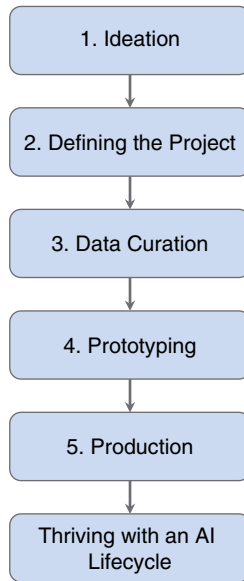


FIGURE 1.2 The AI Adoption Roadmap

to solve?,” “How does your organization operate today?,” and “How do you believe your organization will be able to benefit from AI technology?” Answering questions like these will kick-start your AI journey by establishing a clear set of goals. To properly answer these questions, you will also need a general understanding of the technology, which we will cover in the following section.

2. Defining the Project

Once you have determined that the use of AI technologies can help improve your organization or solve a business problem, you must then get specific about what you hope to achieve. During the second step, you will outline specifically which improvements you plan to attain, or which problems you are trying to solve. This will take the form of a project plan. This plan will act as a guiding document for the implementation of your project. Using the methodical techniques of design thinking, the Delphi method, and systems planning makes a plan much easier to develop. These techniques will ensure that you have a sound and realistic project plan.

User stories will also be a large part of the project plan. User stories are an excellent way to break down a project into functional pieces of value. They define a user, the functionality that the system will provide for the user, and the value that the function will provide to the organization. Well-defined user stories also quantify their results to empirically know when success has been achieved. These success criteria make it much easier to see when we have accomplished our user story's goal and communicate a clear course of action for everyone involved. Specificity is the key.

3. Data Curation and Governance

Data is paramount to every AI system. A system can only be as good as the data that is used to build it. Therefore, it is important to take stock of all the possible data sources at your disposal. This is true whether it is data being collected and stored internally or data that you externally license.

After you have identified your data, it is time to leverage technology to further improve the data's quality and prepare it to train an AI system. Crowdsourcing can be a valuable tool to enhance existing data, and data platforms such as Apache Hadoop can help consolidate data from multiple sources. Data scientists will be key in orchestrating this process and ensuring success. The quality of your data will determine the success of your project in a huge way. It is therefore essential to choose the best available data on hand. The old saying about "garbage in, garbage out" applies to AI as well.

4. Prototyping

With your project plan and data defined, it is time to start building an initial version your system. As with any project, it is best to take an iterative approach. In the prototype step, you will select a subset of your use cases to validate the idea. In this way, you are able to see if the expected value is materializing before you are completely invested. This step also enables you to adjust your approach early if you see any problems arise. Developing a prototype will help you to see, with actual results, whether the ideas and plans you defined in the previous steps have promise. In the event that they do not, you

should be able to recover quickly and adjust them using the knowledge gained from prototyping, without the wasted investment of building a full system.

During the prototyping phase, it is necessary to have realistic expectations. With most AI systems, they improve with more data and parameter tweaking, so you should expect to see increasing improvements over time. Luckily, metrics like precision and recall can be empirically measured and used to track this improvement. We will also cover the cases when more data is not the answer and what other techniques can be pursued to continue improving the system.

5. Production

With a successful prototype under your belt, you have been able to see the value of the technology in action. Now it is time to further invest and complete your system. At this point, it is also a good idea to revisit your user stories and plan as a whole to determine if any priorities have changed. You can then proceed with building the production system.

The production step is the process of converting the prototype into a full-fledged system. This includes conducting a technological evaluation, building user security models, and establishing testing frameworks.

Technological Evaluation

During the prototype phase, developers select technologies appropriate for a prototype, including using technologies and languages that are easy to work with. This mitigates risk by determining the project's feasibility quickly before investing a lot of time and money. That said, during the production step the technology must be evaluated for other factors as well. For instance, will the technology scale to a large number of users or massive amounts of data? Will the technology be supported in the long term and be flexible enough to change as requirements do? If not, pieces of the prototype might have to be rebuilt to accommodate.

User/Security Model

During the prototype phase, the project is typically only running on locked-down development machines or internal servers.

While they require some security, high levels of security are not typically needed during prototyping and will only slow down the prototyping process. Work, such as integrating an organization's user directory (single sign-on [SSO]) and permission structures, will be part of the production process.

Testing Frameworks

To ensure code quality, testing frameworks should be built alongside the production code. Testing ensures that the code base does not regress as new code is added. Development teams may even adopt a “test first” approach called test-driven development (TDD) to ensure that all pieces of code have tests written before starting their implementation. If TDD is used, developers repeat very short development cycles, writing only enough code for the tests to pass. In this way, tests reflect the desired functionality and code is written to implement that functionality.

Thriving with an AI Lifecycle

Once you have adopted AI and your organization is realizing its benefits, it is time to switch into the lifecycle mode. At this point, you will be maintaining your AI systems while consistently looking for ways to improve. This might mean leveraging system usage data to improve your machine learning models or keeping an eye on the latest technology announcements. Perhaps the AI models you have implemented can also be used in another part of your organization. Furthermore, it is important that the knowledge gained during the implementation of your first AI system be saved for future projects. As we will discuss in this book, this can take the shape of either an entry in your organization's model library or a lessons learned document.

The Road Ahead

Adopting artificial intelligence in your organization can feel like a daunting task, especially since the technology is changing so frequently. The main idea is to be aware of all the benefits, as well as the pitfalls, so that you can adequately discern between them and

navigate your way to success. Mistakes are inevitable. Keeping them small and easy to recover from will ensure that your AI transformation has the resilience it needs to prevail. To minimize the likelihood of mistakes, we list the common pitfalls associated with each step at the end of each chapter so you can take notice and avoid them. With sufficient planning and foresight provided by this book, you will be able to acquire the tools necessary to make your organizational adoption of AI a great success.

Notes

1. <https://preferred.jp/en/news/tag/fanuc/>
2. www.bloomberg.com/news/features/2017-10-18/this-company-s-robots-are-making-everything-and-reshaping-the-world
3. https://en.wikipedia.org/wiki/File:FANUC_R2000iB_AtWork.jpg
4. www.hrblock.com/tax-center/newsroom/around-block/partnership-with-ibm-watson-reinventing-tax-prep/
5. www.blackrock.com/aladdin/offerings/aladdin-overview
6. <https://ir.blackrock.com/Cache/1001247206.PDF?O=PDF&T=&Y=&D=&FID=1001247206&iid=4048287>
7. www.institutionalinvestor.com/article/b1dn-7pgfhhbxpsg/BlackRock-s-Aladdin-Adds-Alts-Power
8. www.ft.com/content/9ab2d910-1816-11e9-9e64-d150b3105d21

Ideation

An Artificial Intelligence Primer

The evolution of digital computers can be traced all the way back to the 1800s. The 1800s were an era of steam engines and large mechanical machines. It was during this era that Charles Babbage drew up the notes for making a difference engine.¹ The difference engine was an automatic calculator that worked on the principle of second-order derivatives to calculate a series of values from a given equation. This breakthrough paved the way for modern computers. After the invention of the difference engine, Babbage turned his attention to solving more equations and giving a programming ability to his machines. His new machine was called the analytical engine.

Another key figure in this era of computing was Ada Lovelace. She prepared extensive notes to aid in the understanding and generalization of the analytical engine.² For her contributions, she is generally considered to be the world's first programmer. Although she erroneously rejected that computers were capable of creative and decision-making processes, she was the first to correctly note that computers could be the generalized data processing machines we see today.

Alan Turing, in his seminal paper introducing the Turing test,³ met Lovelace's objections head on, saying that the analytical engine had the property of being "Turing complete" similar to programming language today and that with sufficient storage and time, it could be

programmed to complete the Turing test. Turing further claimed that Lovelace and Babbage were under no obligation to describe all that could be achieved by the computer.

The Turing test (aka the “imitation game”), constructed by Turing in the same paper, is a game where two players, A and B, try to fool the third player, C, about their genders. This has been modified over the years to the “standard” Turing test where either A or B is a computer and the other is a human and C must determine which is which (see Figure 2.1). The critical question that Turing was trying to answer using this game is “Can machines communicate in natural language in a manner indistinguishable from that of a human being?”⁴ Turing postulated about machines that can learn new things using the same techniques that are used to teach a child. The paper deduced, quite correctly, that these thinking machines would be effectively black boxes since they were a departure from the paradigm of normal computer programming. The Turing test is still undefeated as of this writing, but we are well on our way to breaking the test and moving on to the greener pastures of intelligence. Although many chatbots

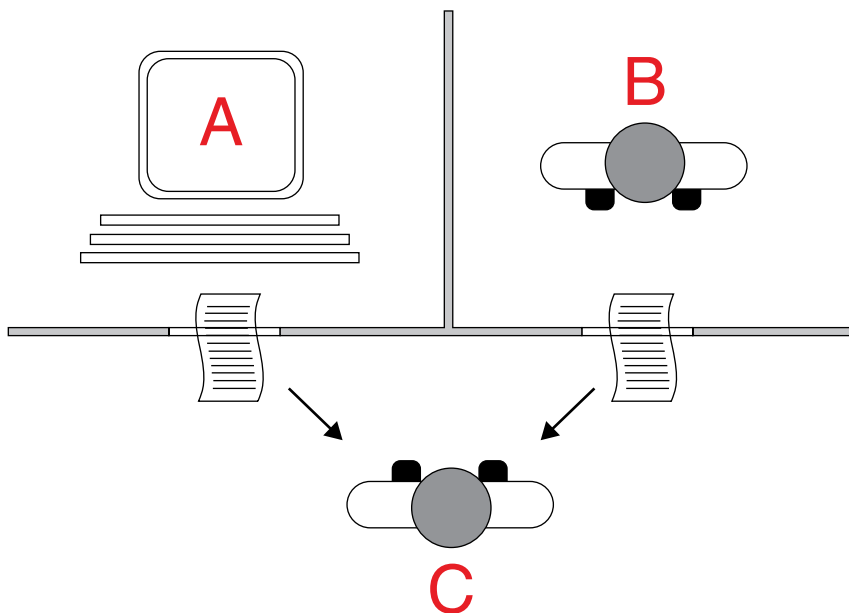


FIGURE 2.1 The Standard Interpretation of the Turing Test⁵

have claimed to break the test, it has not been defeated without cheating and using tricks and hacks that do not guarantee a long-term correct result.

Modern AI has come a long way from the humble beginnings of the analytical engine and the one simple question of “Can machines think?” Today, we have AI that can understand the sentiment and tone of a text message, identify objects in image, search through thousands of documents quickly, and almost converse with us flawlessly with natural language. Artificial intelligence has become a magic assistant in our phones that awaits our questions in natural language, interprets them, and then returns an answer in the same language, instead of just showing a web result. In the next section, we will have a brief look at the state of modern AI and its current set of capabilities.

Natural Language Processing

The ability to converse with humans, as humans do with one another, has been one of the most coveted feats of AI ever since a thinking machine has been thought of. The Turing test measures a computer’s ability to “speak” with a human and fool that person into thinking they are speaking to another human. This branch of AI, known as *natural language processing* (NLP), deals with the ability of the computer to understand and express itself in a natural language. This has proven to be especially difficult, since human conversations are loaded with context and deep meanings that are not explicitly communicated and are simply “understood.” Computers are bad at dealing with such loosely defined problems, since they work on well-defined programs that are unambiguous and clear. For example, the phrase “it is raining cats and dogs” is difficult for a computer to understand without the entirety of history and literature accessible inside the computer. To us, such a sentence is obvious even if we’re previously unaware of the meaning, because we have the entire context of our lives to judge that raining animals is an impossibility.

Programmatic NLP

The first chatbots and natural language processing programs used tricks and hacks to translate human speech into computer instructions.

ELIZA was one of the first few programs to make people believe with certain limitations that it was capable of intelligent speech. This was accomplished by Joseph Weizenbaum at the Massachusetts Institute of Technology (MIT) Artificial Intelligence Laboratory in the 1960s. ELIZA was designed to mimic psychologists by echoing the user's answer back to them. In this way, the computer seemed to hold intelligent conversation, but it clearly was not. There are other forms of NLP seen in the 1980s; they were text-based adventure games. The games understood a certain set of verbs—such as *go*, *run*, *fight*, and *eat*—and modified their feedback based purely on language parsing. This was accomplished by having a set of words that the game understood mapped to functions that would execute based on the keyword. The limitation of words that could be stored in memory meant that these early natural language parsers could not understand everything and would return errors and thus ruin the illusion very quickly.

A method of using programming techniques to parse natural language, programmatic NLP uses string parsing with regular expressions (regex) along with a dictionary of words the program can execute on. The regular expressions match the specified patterns, and the program adjusts its control flow based on the information gleaned from sentences, discarding everything except the main word. For example, the following is a simple regular expression that could be used to determine possible illness names:

```
diagnosed with \w+
```

This example looks for the phrase “diagnosed with” followed by a single word, which would assumingly be the name of an illness (such as “diagnosed with cancer”). A more complex regular expression is required to identify illnesses with multiple words (such as “diagnosed with scarlet fever”). A full discussion of regular expressions is outside the scope of this book—*Mastering Regular Expressions*⁶ by Jeffrey Friedl is a great resource if you want to learn more.

Although the techniques we've discussed can make wondrous leaps in parsing a language, they fall short very quickly when applied more generally, because the dictionary supplied with the program can be exhausted. This is where AI steps in and outperforms these traditional methods by a huge margin. Natural languages, while they

do follow the rules of grammar, follow rules that are not universal and thus something that holds meaning in one region might not hold true for another region. Languages vary so much because they are a fluid concept; new words are being added constantly to the vernacular, old idioms and words are retired, grammar rules change. This makes a language a perfect candidate for stochastic modeling and other statistical analysis, covered under the umbrella term of *machine learning for natural language processing*.

Statistical NLP

The techniques we've described are limited in scope to what can be achieved via parsing. The results turn out to be unsatisfactory when used for longer conversations or larger bodies of texts like an encyclopedia or the body of literature on even just one illness (per our earlier example). This necessitates a method that can learn new concepts while trying to understand a text, much like a human does. This method must be able to encounter new words in the same way that a human does and ask questions about what they mean given their context. Although a true AI agent that can perform automatic dictionary and other necessary contextual lookups instantly is years away, we can improve on the programmatic parsing of text by performing a statistical analysis.

For almost all the AI-based natural language parsers, there are some key steps in the algorithm: tokenization and keys, term frequency-inverse document frequency (tf-idf), probability, and ranking. The first step in parsing a sentence is *chunking*. Chunking is the process of breaking down a sentence based on a predetermined criterion; for example, a single word or multiple words in the order subject-adverb-verb-object, and so forth. Each such chunk is known as a *token*. The set of tokens is then analyzed and the duplicates are discarded. These unique chunks are the "keys" to the text. The tokens and their unique keys are used as the building blocks for probability distributions and for understanding the text in more detail. The next step is to identify the frequency distribution of the tokens and keys in the training data. A histogram of the number of occurrences of each key in the text can be used to plot the data to help better visualize the data. Using these frequencies, we can arrive

at probabilities that a word will be followed by another in the text. Some words like “a” and “the” will be used the most, whereas others like names, proper nouns, and jargon will be more sparingly used. The frequency by which a given word appears in a document is called a *term frequency* and the frequency of the same word across various documents is called *inverse document frequency*. Inverse document frequency aids in reducing the impact of commonly used words like “a” and “the.”

Machine Learning

Machine learning is the broad classification of techniques that involve generating new insights by statistical generalization of the earlier data. Machine learning algorithms typically work by minimizing a penalty or maximizing a reward. A combination of functions, parameters, and weights come together to make a machine learning model. The learning techniques can be grouped into three major categories: supervised learning, unsupervised learning, and reinforcement learning. Supervised learning is the most common type of technique used for constructing AI models, whereas unsupervised learning is more useful for identifying patterns in an input. Supervised learning will calculate a cost for each answer in the training dataset that was incorrectly answered during the training phase. Based on this error estimate, the weights and parameters of the function are adjusted recursively so that we end up with a general function that can match the training questions with answers with a high level of confidence. This process is known as *back propagation*. Unsupervised learning is only given a dataset without any corresponding answers. The objective here is to find a general function to better describe the data. Sometimes, unsupervised learning is also used for simplifying the input stream to be fed into a supervised learning model. This simplification reduces the complexity required for supervised learning. Just as cost and error is defined as two functions in supervised learning, under reinforcement learning an arbitrary cost is assigned based on the action taken. Such a cost will need to minimize or maximize a similar arbitrary reward. In this example:

Raw Data: [fruit: apple, animal: tiger, flower: rose]

supervised learning would be provided with the entirety of this set of data and would use the answers—say, “apple is a fruit”—to test itself. In unsupervised learning, only the following data would be given to the algorithm:

```
[apple, tiger, rose]
```

and the algorithm would then find a pattern among the given data. Reinforcement learning could have the computer guessing the type of noun, and the user would assign a penalty/reward for each correct/incorrect guess accordingly.

Markov Chains

In mathematics, a stochastic process is a random variable that changes with time. This random variable can be modeled if it has a *Markov property*. A Markov property indicates that the state of the variable is affected only by the present state; if you know the present state, you can predict the future state with reasonably high accuracy. *Markov chains* work on the principle of the Markov property; by incrementally “guessing” the next word, a sentence can be formed, and sentences together make a paragraph, and so on. The Markov chain uses the tf-idf frequencies to assess the probability of each word that can appear next and then chooses the word with the highest probability. Because the reliance on tf-idf is quite high, Markov chains require large amounts of training data to be accurate. It is of vital importance that Markov models are parametrized and developed properly. Such a statistical approach has been proven better than modeling grammar rules. Variations on this model include smoothing the frequency and giving access to more data to the model. Although Markov chains are very fast and can give a better result than traditional approaches, they have their limitations and are unable to produce long forms of coherent text on a particular topic.

Hidden Markov Models

A *hidden Markov model* is one where the state of the program is “hidden.” Unlike the regular models, which act in a very deterministic

method, hidden Markov models have the possibility of having infinite states and deriving more information than a regular Markov model. Hidden Markov models have had a recent resurgence when combined with technologies such as Word2vec (developed by Google), which can create word embedding. Nevertheless, the application of stochastic processes to languages has severe limitations, even with the larger numbers of parameter values possible with hidden Markov models. Hidden Markov models, just as traditional models, also require large amounts of data to reach better accuracy. Neural networks can be used with smaller datasets, as you'll see next.

Neural Networks

Neurons are the smallest decision-making biological component in a brain. Although it is currently impossible to identically model biological neurons inside a computer, we have been successful in approximately modeling how they work. Neurons are connected together in a neural network and accept some input, perform an operation on it, and then generate an output. In a neural network, the output is determined by the “firing” of a digital neuron. Neural networks can be as simple as a single node (see Figure 2.2), or they

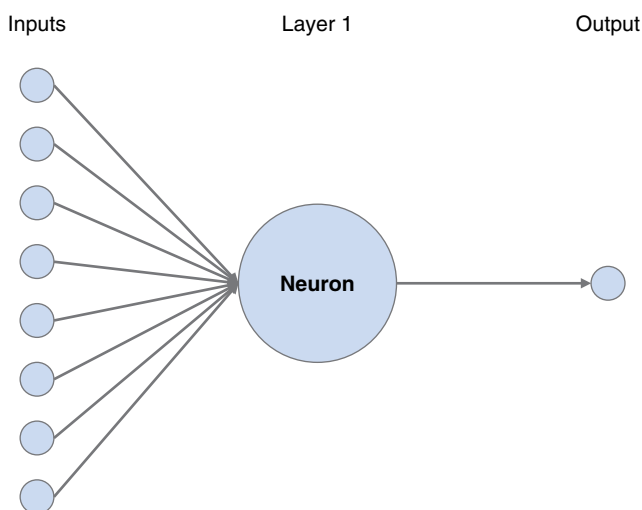


FIGURE 2.2 A Neural Network with a Single Neuron

can contain multiple layers with multiple neurons per layer (see Figure 2.3). This neural network–based approach is also referred to as *deep learning* because of the potentially vast number of neural layers that a single model can contain. Deep learning is a type of machine learning that is considered AI (see Figure 2.4).

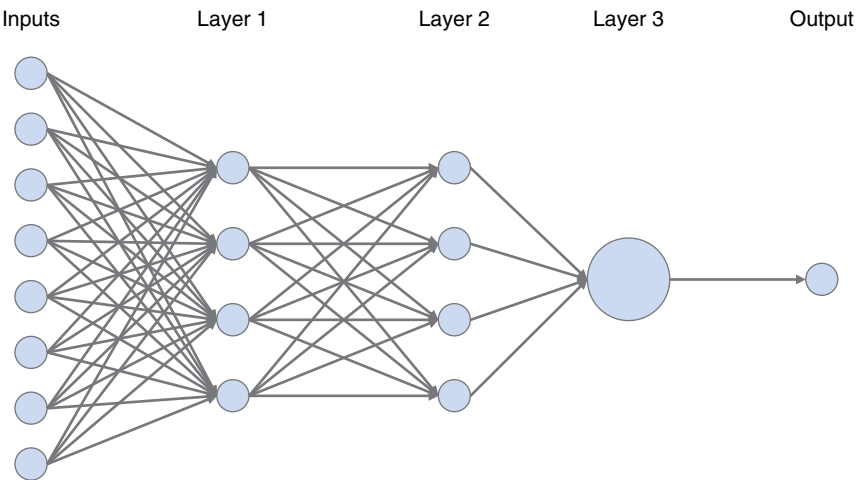


FIGURE 2.3 A Fully Connected Neural Network with Multiple Layers

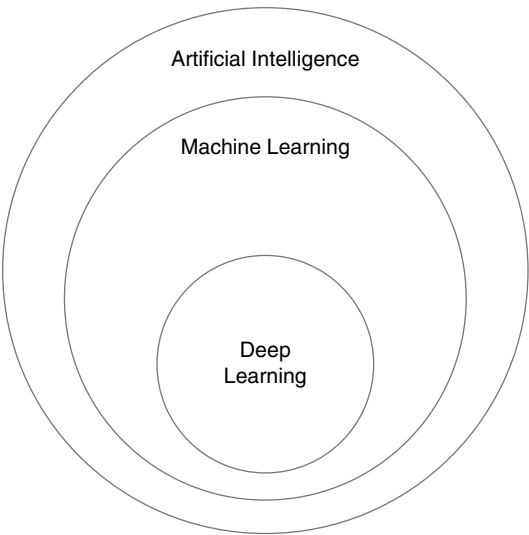


FIGURE 2.4 A Venn Diagram Describing How Deep Learning Relates to AI

One use case for deep learning is creating neural networks that are capable of creating new sentences from scratch based on a prompt. In this case, neural networks serve as a probability calculator that ranks the probability of a word “making sense” as part of a sentence. Using the simplest form of a neural “network,” a single neuron could be asked what the next word is in the new sentence it is generating, and based on the training data, it would reply with a best guess for the next word.

Words are not stored directly in the neuron’s memory but instead are encoded and decoded as numbers using *word embedding*. The encoding process converts the word to a number, which is more easily manipulated by the computer, whereas the decoding process converts the number back to a word. Such a simplistic sentence-generating model with a single neuron can hardly be used for any serious applications. In practice, the number of neurons would be directly proportional to the complexity of text being analyzed and the quality of expected output. Adding more neurons (either in the same layer or by adding additional layers) does not automatically make a neural network better. The techniques to improve a neural network will vary based on the problem at hand, and the model will need to be adjusted for unexpected outcomes. In a practical implementation, there would be multiple neurons, each having a specific weight; the weight of the neuron would determine the final output of the neural network. This method of adjusting weights by looking at the output during the training stage is known as a *back propagation neural network*.

If we pass the output received from a neuron back through it again, it would give us a better opportunity of analyzing and generating the data. A recurrent neural network (RNN) does exactly that, and multiple passes are made recursively, feeding the output back into the neuron as input. RNNs have proved to be much better at understanding and generating large amounts of text than traditional neural networks. A further improvement over RNNs are long short-term memory (LSTM) neural networks. LSTM networks are able to remember previous states as well, and then output an answer based on these states. LSTM networks can also be finely tuned with gate-like structures that can restrict what information is input and output by a neuron. The gates use pointwise multiplication to determine if all or no information will go in through the gate. The gates are

operated by a sigmoid layer that judges each point of data to determine whether the gate should be opened or closed. Further variations include allowing the gate to view the output of a neuron and then pass a judgment, thus modifying the output on the fly.

Chatbots and text generators are some of the biggest use cases for NLP-based neural networks. Speech recognition is another area where such neural networks are used. Amazon's Alexa uses an LSTM.⁷

Image Recognition/Classification

Images contain a lot of data, and the permutation and combination of each pixel can change the output drastically. We can preprocess these images to reduce the size of the input by adding a convolutional layer that reduces the size of the input data, thus reducing the computing power required to scan and understand larger images. For image processing, it is critical to not lose sight of the bigger picture—a single pixel cannot tell us if we are looking at an image of a plane or a train. The process of convolution is defined mathematically to mean how the shape of one function affects another. In a convolutional neural network (CNN), maximum pooling and average pooling are also applied after the convolutional process to further reduce the parameters and generalize the data. This processed data from the image is then fed into a fully connected neural network for classification. CNNs have proven their effectiveness in image recognition; they are very efficient at recognizing images, and the reduced parameterization aids in making simpler models.

Becoming an Innovation-Focused Organization

The world of technology moves at a high speed. Innovation will be the key business differentiator going forward. The advantage of being the company to present a key piece of innovation in the marketplace is game changing. This first-mover advantage can be achieved only via the relentless pursuit of innovation and rigorous experimentation with new ideas. Innovations using AI can also lead to cost-saving practices, offering the competitive edge needed to fiscally outmaneuver the competition. Organizations should develop a culture of innovation by establishing suitable processes and incentives to encourage their workforce.

A culture of innovation is hard to implement correctly, but it yields splendid results when done well. Your organization should have policies that encourage innovation and not limit it. A motivated workforce is the cornerstone of innovative thinking. If the employees are unmotivated, they will think only about getting through the day and not about how to make their systems more efficient.

IBM pioneered the concept of Think Fridays, where employees are encouraged to spend Friday afternoons working on self-development and personal research projects.⁸ This in part has led IBM to being called one of the most innovative companies given that 2018 marks their 26th consecutive year of being the entity with the highest number of U.S. patents issued for the year.⁹

Google similarly has a 20 percent rule that allows employees to work on personal projects that are aligned with the company's strategy.¹⁰ This means that Google employees, for the equivalent of one day a week, are allowed to work on a project of their own choosing. This morale-boosting perk enables employees to work on what inspires while Google maintains the intellectual property their employees generate. Famously, Gmail and Google Maps came out of 20 percent projects and are now industry-leading Google products in their respective fields.

Not every organization needs to be as open as Google, but even a 5 percent corporate-supported allowance can have a big impact, because employees will use this time to focus on revolutionizing the business and will feel an increased level of creative validation with their work. Ensuring that employees are adequately enabled and motivated is a task of paramount importance and gives them the tools to modernize and adapt their workflows.

An organization that fosters creativity among its employees is bolstered to succeed. It will be the first among its peers to develop and implement solutions that will directly impact key metrics and result in savings of time and money. When existing business processes can be revamped, major transformations of 80 to 90 percent increased efficiency and productivity are possible. The main idea is to allow employees time to think freely. Every employee is a subject matter expert about their own job. Transferring this personal knowledge into shared knowledge for the organization can lead to a valuable well of ideas.

Innovation should be a top priority for the modern organization. At its core, prioritizing innovation means adapting the business to the constantly changing and evolving technological landscape. A business that refuses to innovate will slowly but surely wither away.

The organization that chooses to follow the mantra “Necessity is the mother of all invention” will continuously lag behind its peers. It will be following a reactive approach instead of a proactive approach. Such an organization will, by definition, be perpetually behind the technological curve. With that said, there are certain upsides to this strategy. If you are merely following the curve, you avoid the mistakes made by early adopters and can learn from those mistakes. Such an organization would save on short-lived, interim technologies that are discarded and projects that are shelved after partial implementation.

At the other end of the spectrum is an organization that yearns for change. This organization will try to implement new technologies like AI and robotics as much as it can. This proactive policy has the potential to yield far greater returns than the organization that follows the curve. This modern business, however, will need to control and manage its change costs carefully. It runs the risk of budget overruns and faces the disastrous possibility of being thrown completely out of the race. Care should be taken to minimize the cost of mistakes by employing strategies like feedback loops, diversification, and change management, all of which we will discuss in more detail in the coming sections. The rewards in this scenario are great and will lead an organization to new heights.

This path of constant innovation and learning is the smartest path to follow in the modern world. Passively reacting to technological changes only once they have become an industry standard will greatly hamper any organization.

Idea Bank

Organizational memory can be fickle. To aid the growth of an innovative organization, an “idea bank” should be maintained. The idea bank stores all ideas that have been received but not yet implemented. An innovation focus group should be given the authority to add and delete ideas from the bank, though the idea bank should be

adequately monitored and protected since it will contain the way forward for your organization and possibly quite a few classified company internals.

An organization should designate a group of managers to focus on innovation on an ongoing basis as part of their jobs. Members of this group should be selected so that all departments are represented. This innovation focus group should hold regular (weekly, monthly, or quarterly) meetings, which will include reviewing new suggestions from other employees as well as feedback and suggestions received from other stakeholders like vendors and customers. Such a group would have the official responsibility of curating and reviewing the idea bank.

The idea bank should allow submissions from all levels of the organization, with their respective heads as filters. The final say for inclusion in the idea bank should be left with the innovation focus group. This allows the idea bank to grow rich with potential ideas for implementing changes to the way the organization works while maintaining quality control. Employees should also be rewarded if an idea they submit gets executed. This will provide employees with more motivation for submitting and implementing ideas.

Approved submissions should be clear and complete so that it is possible to pick up and implement proposals even in the absence of their authors. A periodic systematic review of the idea bank should be conducted to ascertain which ideas are capable of immediate implementation. Table 2.1 shows an example of an idea bank.

TABLE 2.1 A sample idea bank

Idea	Estimated impact	Estimated investment	Submitter
Automated Venture Evaluation	Increase revenues by 20%	\$100K over 3 months	Margaret Peterson, CEO
AI Support Chatbot	Save 30% of the support budget	\$250K over 6 months	Mallory McPherson- Wehan, Customer Service Manager

TABLE 2.1 A sample idea bank (continued)

Idea	Estimated impact	Estimated investment	Submitter
Manufacturing Workflow Automation	Improve manufacturing by 10%	\$50K over 2 weeks	Zack Kimura, Engineering Lead
Advertising Channel Optimization	Reduce required advertising budget by 10%	\$100K over 2 months	Mike Laan, Social Media Marketing

Business Process Mapping

Business process mapping can be a major asset in helping you to identify tasks that can be automated or improved on. Each business process should have a clear start and finish, and your map should contain the detailed steps that will be followed for a complete process. Flowcharts and decision trees can be used to map processes and their flow within the organization. Additionally, any time taken by moving from one step to the next should be recorded. This additional monitoring will help identify any bottlenecks in the process flow. Another document that can be used to help chart the flow of processes is a detailed list of everyone’s jobs and their purposes in your organization.

Who? What? How? Why? Asking these four questions for every process will ensure that all data necessary for the process map is available. These simple questions can generate complex answers necessary to find and fill the gaps. The questions should be asked recursively—that is, continuing to drill down into each answer—to ensure that all the necessary information is generated. As an example, let’s map the details for a department store chain starting at the store level:

- Who controls the stores?
 - Store manager
 - Who controls the store manager?
 - Reports to the VP of Finance
 - What does the store manager control inside the store?
 - Processes related entry inward/outward and the requisition of goods

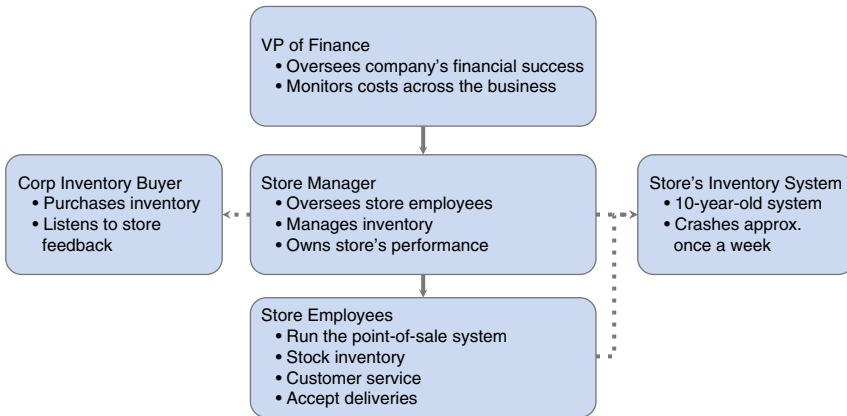


FIGURE 2.5 An Enhanced Organizational Chart

- How does the store manager control the stores?
 - Uses custom software written 10 years ago
- Why does the store manager control the store?
 - To ensure that company property is not stolen or otherwise misused
- What is in the stores?
 - . . .
- How does it get there?
 - . . .
- Why is it needed?
 - . . .

This example has answers to critical questions as we draw up the organizational flowcharts, information flow diagrams, and responsibilities. Figure 2.5 starts this process by providing an interaction diagram between the actors in the system and the roles they play. For instance, does the 10-year-old software still address the needs of the store with minimal service disruptions?

Flowcharts, SOPs, and You

With the information collected so far, we can prepare our organizational flowchart. Organizational flowcharts are a helpful tool for viewing the processes currently being followed in your organization. Standard operating procedure (SOP) documents are great to use

along with manuals to help trace the flow of information and data throughout an organization, but SOPs will only get you so far. Conducting interviews and observing processes firsthand as they happen will give more specific insight to the deviations and edge cases that arise from SOPs.

As an example, an organization could have the following SOP:

The Assistant Marketing Manager collates the necessary data for sales, completes an Invoice Requisition Form, and emails the data and the form to their manager and the accountant. The accountant prepares an invoice once approved by the Marketing Manager and their own manager as well.

This policy will have quite a few practical exceptions. The marketing manager could send the data themselves if the assistant is on leave, or sometimes the accountant might prepare the invoice only based off the email received without a copy of the invoice requisition form. Such practical nuances can only be ascertained in interviews and a review of what was actually done. The SOP in this case is also a perfect example of one that can be automated using NLP or via the use of more structured forms by allowing direct entry into the system, with the accounting function being done automatically. Such a system, when implemented correctly, would strengthen the internal controls by requiring compulsorily documented assent from the managers, without which an invoice would not be printed.

Information Flows

Another vital tool in your toolbox of idea discovery is the tracking of information and dataflows throughout your organization. Tracking what data is passed across various departments, and then how it is processed, will lead to fresh insights on work duplication, among many other efficiency issues. Processes that have existed for years may have a lot of information going back and forth, with minimal value-added. Drawing up flowcharts for these dataflows will allow you to visualize the organization as a whole (see Figure 2.6 and Figure 2.7). It helps to think of the entire business as a data processing unit, with external information being the inputs and internal

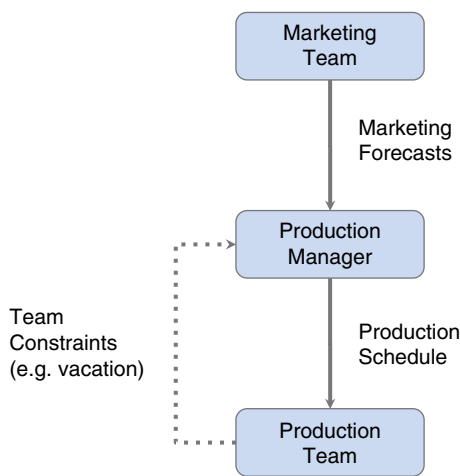


FIGURE 2.6 An Information Flow Before an AI System

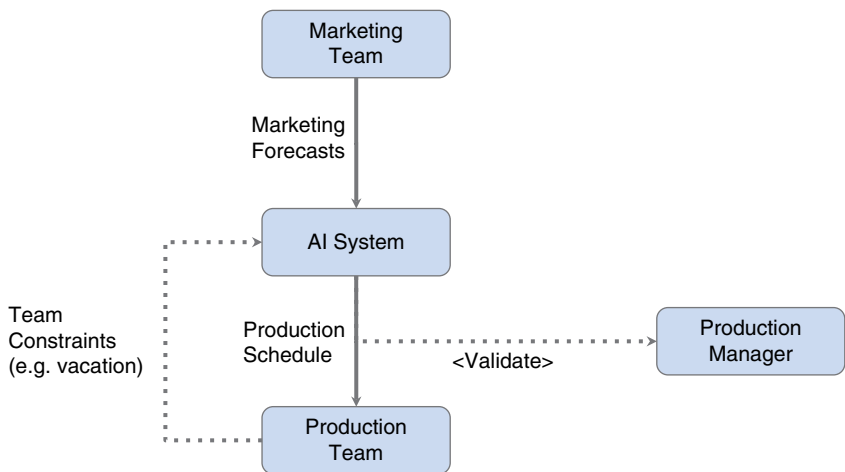


FIGURE 2.7 An Information Flow After an AI System

information and reports sent outside being the final output. For example:

A production team receives their production schedule from the production managers. The production managers use the forecasts prepared by the marketing team to make production schedules.

In this scenario, the production schedule can be made automatically using AI with the marketing forecasts and the constraints of the production team. This efficiency improvement can free up the production manager's time for higher-value activities.

Coming Up with Ideas

Once the process flowcharts, timesheets, information flows, and responsibilities are established, it is time to analyze this wealth of data and generate ideas from the existing setup about how to revamp it. Industry best practices should be adopted once the gaps have been identified. Every process should be analyzed for information like its value-added to internal and external stakeholders, time spent, data required, source of data, and so forth. The idea is to find processes that can be revamped to provide substantial improvements that will justify any revamp investment.

A note of caution as you embark on idea discovery: When you are holding a hammer, everything looks like a nail. Care should be taken that processes that do not need any upgrading are not being selected for revamping. Frivolously upgrading processes could lead to disastrous results; for every new process, modification has a cost. A detailed cost-benefit analysis is a must when implementing a process overhaul to identify any value that will be added or reduced. This will ensure that axing or modifying a process does not completely erode a necessary value, which could cause the business to fail.

Value Analysis

Artificial intelligence can drastically change the level of efficiency at which your organization can operate. Mapping out all your business processes and performing a value analysis for the various stakeholders in your company will help you isolate processes that need to be modified using an AI system. "Value" is a largely subjective concept at this point and need only be shared with the stakeholders who are directly affected. We must, however, consider both internal and external stakeholders when making this decision. For example, your company's tax filing adds no direct value to your customers. However, it

is important to stay in business and avoid late fees. Therefore, tax filing has important ramifications and is thus deemed “valuable” to a government stakeholder. The major stakeholders in a business can be grouped into five categories:

- Customers
- Vendors
- Employees
- Investors
- Governments

Each of these stakeholders expects to receive a different kind of value from the business. It is in the business’s best interest to provide maximum value at the cheapest cost to each stakeholder in order for the business to remain solvent.

The investigation into existing processes can start with interviews, followed by drawing flowcharts for processes to be done by a user. Every step in the flowchart needs to be assessed for value provided to each of the various stakeholders. Remember that “value” here does not exclusively mean monetary value. Some processes, for example, can provide value in terms of control structure, prevention of fraud, or misuse of company resources. Such a process would need to be retained, even though the process itself might not generate any pecuniary revenue. The key takeaway is to ensure that your costs are justified and that each process is necessary while keeping in mind that “value” can take many different forms and is dependent on the specific stakeholder’s interest.

The value analysis will help to identify processes that can be overhauled in major ways. If the process is adding little value to the customer (stakeholder), then it should be axed. If the process feels like it can be improved on, it should be added to the idea bank. The addition of value to a stakeholder can be considered a critical factor for identifying and marking processes. A process like delivery and shipment, done correctly, adds a sizable value to the end customer. After all, delivery is one of the first experiences a customer will have

with your organization after a purchase has been made. First impressions do matter and can buy some goodwill over the lifetime of a customer.

This process of identifying value can take a long time to complete for all the processes that a business undertakes. In this regard, the entire business process list should be segmented based on a viable metric that covers entire processes from bottom to top, and each segment should be analyzed individually. For instance, with a company that manufactures goods, the processes can be segmented based on procurement, budgeting, sales, and so forth.

A Value Analysis Example

Widgets Inc. sells thousands of products on its website, `Widgets.com`. The CEO notices that new products take about two days to go live after they are received. The CEO asks the chief technology officer (CTO) to draw up the process map and plot the bottlenecks and more time-consuming parts of the process. The CTO starts this task by examining each of the processes in question as they are outlined in the company's records. Training manuals, system documentation, and an enterprise resource planning process are some of the documents the CTO uses to create a detailed flowchart for each of the processes she examines. Process manuals are often created using theoretical descriptions of their processes, as opposed to practical ones, so these records may not reflect the actuality of what each worker is doing and can offer misleading data on a specific process's functionality. Keeping this in mind, our CTO conducts interviews with the employees involved in each of the processes she examines. She marks any discrepancies and the actual time taken for each step against those listed in the flowchart. An example of this flowchart is shown in Figure 2.8.

The CTO notices that step 5, where marketing adds content and tags, takes an average of four hours for each product. Multiple approvals are involved, and approvers typically only look to ensure that the content roughly matches the product and that it is not offensive in any way.

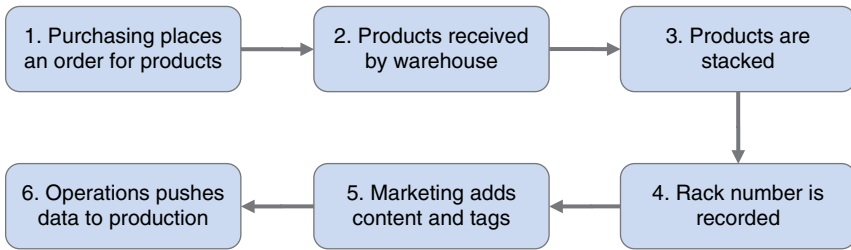


FIGURE 2.8 A Sample Process Flowchart

The CTO submits her findings to the CEO:

Tagging and categorization of products is an issue that takes about 4 hours for every new product introduced on the web-site. Natural language processing and artificial intelligence can help us shorten this time by 80 percent. The program would first learn based on data generated by humans and run parallel to the existing system for the first six months. Then the artificial intelligence would continue to train and improve over time with feedback. The AI can be implemented to rely on the same source material currently being used, like vendor websites, instruction manuals, and the same product descriptions that our marketing department relies on. Once the AI program is adequately trained, it can be sent out independently with only a manual review, saving the company about 60 to 80 percent of marketing's time spent manually tagging and classifying products.

This idea can be implemented right away, or it can be filed for later use in the idea bank, to be implemented once the organization chooses to allocate resources.

Sorting and Filtering

The idea bank, due to its very nature, will quickly become a large database of useless information if it is not periodically sorted and ranked. Due to economic, time, and occasionally legal constraints, it

is impossible to implement all ideas at once. Because of this fact, it is necessary to filter and sort the idea bank by priority in order to realize the benefits of maintaining one in the first place. For instance, ideas that require minimal investment but that have a large cost savings impact should be prioritized first. Conversely, ideas that will take a long time to implement and that have little impact should be low in the idea bank prioritization list, perhaps never being implemented. That said, even low-priority ideas should never be deleted from the idea bank because in the future, circumstances might change to improve the idea's priority. With a long list of prioritized ideas on hand, the next step is to start giving them some structure.

Ranking, Categorizing, and Classifying

Ranking the items in the idea bank based on various metrics will help the future decision makers get to the good ideas faster. Ideas should be ranked separately on various dimensions that enable filtering and prioritization. Some good examples of dimensions to consider when ranking ideas are estimated implementation time, urgency, and capital investment. A point-scoring system can help immensely with this. The relative scale for point values should be clearly established and set out at the start so that subsequent users of the data bank are not left wondering why “automate topic tagging” is prioritized over “predictive failure analysis for factory machinery.”

One of the easiest ways to start organizing our idea bank is by grouping the brainstormed ideas into similar categories. Groups can be as wide or as narrow as you want them to be. Categories should fulfill their purpose of being descriptive while still being broad enough to adequately filter ideas. Here are some examples of the kinds of groups that may be useful for filtering ideas:

Time

Each idea is classified according to the time taken for development of an AI solution and the time needed to change management structures and practices: specifically, short-term (within the next year), medium-term (between one and five years), and long-term (longer than five years). Although these will be only estimates, they are dictated roughly by how quickly you believe your

organization may change, as well as a general idea of the technological feasibility to implement the idea.

Capital Allocation

Priority evaluation based on the anticipated capital necessary to make each idea functional. Capital includes the initial investment, along with recurring maintenance costs and routine costs (if any). Large ideas are ones that involve more than 20 percent of annual profits, medium ideas at 10 to 20 percent of annual profits, and small ideas at less than 10 percent of annual profit required.

Employee Impact

Priority evaluation considering the estimated impact that ideas will have on employees, in terms of labor hours, changing their workflows and processes, and streamlining efficiency. Categories should exist to emphasize the net-positive impact of implementation and should therefore also consider the number of employees expected to be affected by implementation. Impact scores for each idea would then be multiplied by weighted values based on the percentage of the organization's employees expected to be noticeably affected by implementation.

Risk

Every idea should have a risk classification. A risk assessment should be done for threats to the business due to implementation of the idea and a suitable risk category should be awarded: High, Medium, or Low.

Expected Returns on the Idea

The expected returns on an idea should be identified. These can be in the nature of cost savings or incremental revenue. Projects with zero returns can also be considered for implementation if the effect on other key performance indicators is positive.

Number of Independent Requests for an Idea

Perhaps three people suggested your organization interact completely without the use of manual processes such as the filling out of paper forms. Keeping a count of how many independent individuals suggest the same idea can be a quick gauge as to which ideas are most needed or desired, or at least give an impression of similarly regarded issues within the organization.

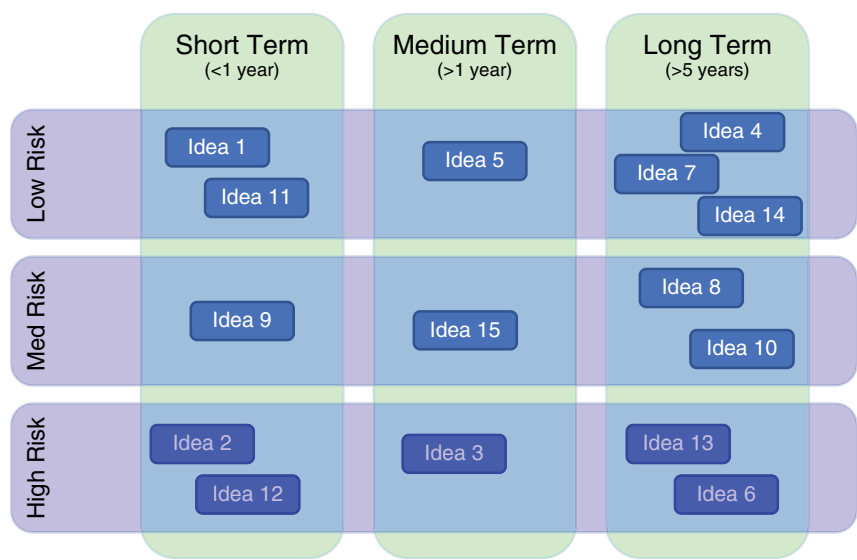


FIGURE 2.9 An Example Grouping of Ideas

Selecting better tags will ultimately aid the decision makers in filtering and sorting through the idea bank. At this stage, most of these tags and information should be educated guesses and not actual thorough investigations into the feasibility of an idea. The classifications are merely tools to maneuver through the list in a structured manner. A sample idea grouping using time and risk is shown in Figure 2.9.

Reviewing the Idea Bank

Reviewing the idea bank on a regular basis will be significantly easier, and better ideas will consistently rise to the top, if the methods of ranking and sorting are well implemented. As company priorities change, the urgency metrics of ideas will also change. If new technology becomes available or an organization finishes a previously roadblocking project, the estimated implementation time for ideas might change as well. This periodic reevaluation will ensure that also the best ideas are always being selected for the following step.

The idea review is as crucial as the ideas themselves and should be done by a special focus group to ensure that the best ideas are

selected for implementation. Selecting an idea for implementation costs time and money, so it is imperative that only the best ideas from those submitted are selected. Implementing ideas in a haphazard manner might incur consequences that could prove disastrous for the organization. The review meetings should also note what ideas were selected for implementation and explain their reasons for doing so. This explanatory nature will aid the future decision makers in avoiding the same mistakes of the past.

After selection, a cost–benefit analysis should be undertaken for ideas that require major changes in workflow or a large capital investment. These new prioritization processes should be efficient and not just increase bureaucracy for the employees of the organization. Whenever possible, a trial run should be attempted before implementing any idea organization-wide. Innovation for the sake of innovation will merely saddle an organization with increased costs, increase unnecessary bureaucratic structures, and leave everyone unhappy.

Brainstorming and Chance Encounters

In an innovative company, it is critical to have your employees motivated and discussing ideas. All major discoveries in the modern era (since the 1600s) can credit the use of the scientific method, which is the process of

1. Making an observation
2. Performing research
3. Forming a hypothesis and predictions
4. Testing the hypothesis and predictions
5. Forming a conclusion
6. Iterating and sharing results

The scientific method relies on criticism and constant course correction to maintain the integrity and accuracy of its findings. The same can be applied to an organization's innovation method by allowing people to critique, discuss, and debate. Providing ample time and space to share ideas and discuss possible innovations is paramount for an organization's growth. These collaborative consortiums,

or brainstorm sessions, work best when conducted at regular and predictable intervals, in small, functional groups, and guided by an organizer who is a peer of the participants. Small groups of employees being guided through strategic thought-sharing sessions will allow the organization to gain insights from many different sources, help employees lend their personal voices to their organization while feeling validated, and also help clarify the organization's intentions, plans, and obstacles to its employees, unifying them in their goals.

A distinction needs to be made at this point between constructive criticism and frivolous or destructive criticism. Useful critiques must always be directed at the idea, not at the person who had the idea. Criticisms should offer data that directly conflicts with the statement presented or should present issues that may conflict in a way that will help them be managed or avoided. Wherever possible, criticism should not be about shooting down an idea, but about how to safeguard it or pivot it into something more viable and sustainable.

Debate enables us to look at a concept from a different perspective, offering us new insight. It is very important for an organization to come up with new ideas, and equally important for it to strike down bad ones. A single person can easily become biased, and bias can be difficult to spot in one self. When presented constructively, hearing opposing viewpoints allows a group to overcome the internal biases of an individual.

In every organization the managers are a critical link, trained to understand the core of the business. They understand the elements that help run the business efficiently and effectively. Asking the managers to submit a quarterly report for ideas that can be implemented on a quarterly or semiannual basis can also be an effective source for gathering ideas, filtered through the people who are likely to best understand the company's needs, or at least the needs of their own particular department.

One final idea, which research is proving to be even better than regular brainstorming sessions, is encouraging chance encounters and meetings. Chance encounters among members of different teams will lead to spontaneous and productive discussions and increased ease of communication, and result in improved understanding with which to generate higher-quality ideas than those from larger, scheduled brainstorms. Chance encounters can be encouraged by

creatively designing workspaces. For instance, Apple designed their Infinite Loop campus with a large atrium where employees can openly have discussions. During these discussions, employees might see someone they have been wanting to talk to. In this way, a productive exchange, which may not have otherwise happened, is able to take place through this chance encounter.

Cross-Departmental Exchanges

In an organization, multiple departments need to work with one another to attain the objectives of the organization. Exchanges between departments typically only occur as much as minimally needed to get the job done. The downside to this “needs”-based approach is that two departments that could come together and offer each other valuable new perspectives on process sharing and troubleshooting rarely interact. Accounting is a function that needs a familiarity with every employee in an organization, but this cross-talk needed by accounting, relating to the filing of reimbursement claims and other documentation needs of the department, can be too one-sided for chance encounters and too tangential for members of accounting to join in the brainstorming sessions of other departments. On the flipside, it is easily possible for brainstorming sessions to be set up in a cross-functional manner. Many principles can be shared from one expert to another if they are aware of the role that the other person plays. The avenues of such cross-talk need not be limited just to meetings and brainstorming sessions. Users should be provided and encouraged to use more collaborative programs like discussion forums, social events, interdepartmental internships, cross-departmental hiking trips, and so forth. The key is to let two departments become comfortable enough to share their workflow and ideas among themselves. An approach like this will help the organization to grow and adopt newer technologies across domains.

While implementing departmental cross-talks, care should be taken that the “walls” that separate the departments due to ethical, legal, and privacy-based concerns are not taken down. “Walls” is legalese for the invisible walls separating two departments within an organization, whose objectives and integrity demand independence from each other, where comingling could lead to conflicts of interest.

In an investment bank, for example, the people marketing the product should not be made aware of material nonpublic financial information received from the clients. Control should not be sacrificed for ideation.

In another example, Auto Painting Inc. paints cars for companies with fleets of cabs. The accounting manager generates invoices based on data received in emails from the marketing department. Due to this manual process, invoices are sometimes delayed, causing a cash flow problem for the company. In an effort to automate, Auto Painting Inc. hires a development team to develop a new website. Luckily, the company has forums on an internal intranet, the use of which is encouraged, especially for cross-talk among departments. The account manager and the developer for the website converse over the procedure for generating invoices. The developer suggests that with an implementation of an NLP algorithm, the generation of invoices can be automated, saving hours of company time and money. The idea is fast-tracked based on the company's priorities, and in a few months, the overall company cash flow is improved.

AI Limitations

It is very important to understand the limitations of artificial intelligence. Knowing what you cannot do right now will help you to temporarily reject ideas that could be out of reach. The thing to bear in mind while rejecting ideas is to ensure that you do not lose sight of ideas generated if the technology has not reached you yet. The AI field can still be considered to be in its nascency, and it is a very actively researched field. To ensure great ideas that are currently blocked by AI's limitations are not lost, they should be committed to the idea bank. The idea should be recorded with the current blocker to ensure that when technology catches up, your organization can start implementing the idea and get the first-mover advantage. Although there are many specific limitations of AI frameworks as of this writing, here are a number of general limitations that are currently true:

Generalizations

Artificial intelligence as it currently stands cannot solve problems with a single, general approach. A good candidate problem that can give you a good return on investment should have a

well-defined objective. A bad candidate problem is one that is more loosely defined and has a broad scope. The narrower your scope, the faster a solution will be developed. For instance, an AI system that tries to generate original literature is a very hard problem, whereas learning to generate answers from a script based on finding patterns among questions is a problem with a much smaller scope and therefore much easier to solve. In technical terms, a “strong artificial intelligence” is an AI that can pass the Turing test (speaking with a human and convincing the human that it is not an AI) or other tests that aim to prove the same levels of cognition. Such programs are likely years, if not decades, away from any kind of market utilization. On the other hand, “weak artificial intelligence” is AI designed to solve smaller, targeted problems. They have limited goals, and the datasets from which they learn are finite. A business example is tagging products based on their dimensions, classification, and so forth. This is a very important task in most businesses. In this case, tagging based on model number is easier, compared to identifying products based on images.

Cause and Effect

A lot of artificial intelligence in today’s world is a *black box*. A black box is something you feed data to and it gives you an answer. There is no reason (at least one easily understood by humans) why certain answers were chosen over others. For projects like translations, this does not matter, but for projects involving legal liability or where “explainability” of logic is of high importance, this could be troublesome. One type of AI model assigns weights to each possible answer component and then decides, based on training, whether to reduce or increase weights. Such a model is trained using an approach called *backpropagation*. These weights have no logic besides leading the AI closer to a correct answer based on the training data. Hence, it is imperative to assess AI for ethical and legal concerns regarding “explainability.” This topic is discussed further by our AI expert, Jill Nephew, in Appendix A.

Hype

Artificial intelligence is cutting-edge technology. With every cutting-edge technology comes its own marketing hype that is created to prey on the asymmetry of information between the users and the researchers. It is important to do your due diligence on any AI products or consultants before using them to build a solution. Ask for customer references and demonstrations to see for yourself. It is very important to ensure that AI being developed is feasible in terms of current technological capability.

Bad Data

Artificial intelligence, which is built using supervised learning techniques, relies heavily on its training data set. If the training data set is garbage, the answers given will also be garbage. The data needs to be precise, accurate, and complete. In terms of development of an AI project, data availability should be ensured at the conception of the project. A lack of data at a later stage will cause the entire project to fail, and more time and resources to have been wasted. Good data is a critical piece of the AI development puzzle.

Empathy

Most AI lacks empathy of all manner and kind. This can be a problem for chatbots and other AI being developed for customer service or human communication. AI cannot build trust with people the way a human representative can. Disgruntled users are more likely to feel even more frustrated and annoyed after a failed interaction with a chatbot or an Interactive Voice Response (IVR) system. For this reason, it is necessary to always have human interventionists ready to step in if needed. AI cannot necessarily make a customer feel warm and welcome. It would be best not to use AI chatbots or other such programs in places where nonstandard and unique responses are needed for every question. AI is more suitable in service positions that are asked the same questions repeatedly but phrased differently. Here is an example of such a scenario inside an IT company that provides support for a website: “How to reset my password?”; “How do I change my password?”; “Can I change my password?” Such

similar questions can be handled very well by an AI program, but an interaction on what laws are applicable to a particular case is very difficult to implement and will require resources that are greater in orders of magnitude.

Pitfalls

Here are some pitfalls that you may encounter during ideation.

Pitfall 1: A Narrow Focus

Artificial intelligence is an emerging field with wide applications. Although trying to solve every problem with artificial intelligence is not the right approach, care should be taken to explore new potential avenues and ensure that your focus is not too narrow. During the ideation stage, it is essential to be as broad-minded as possible. For instance, consider how AI might be able to improve not only your core business but also auxiliary functions such as accounting. Doing so while acknowledging the limits of real-world applications will facilitate idea generation. Some applications for AI can also be relatively abstract, benefiting from lots of creative input. All ideas that are considered plausible, even those in the indeterminate future, should be included in the idea bank.

Pitfall 2: Going Overboard with the Process

It is easy to get carried away with rituals, thus sidelining the ultimate goal of generating new ideas. Rituals such as having regular meetings and discussions where people are free to air their opinions are extremely important. Apart from these bare necessities, however, the focus should be placed on generating ideas and exploring creativity, rather than getting bogged down by the whole process. The process should never detract from the primary goal of creating new ideas.

Pitfall 3: Focusing On the Projects Rather than the Culture

For an organization, the focus should be on creating a culture of innovation and creativity rather than generating ideas for current projects. A culture of innovation will outlast any singular project and

take your organization to new heights as fresh ideas are implemented. Creating such a culture might involve a change in the mindset around adhering to old processes, striving to become a modern organization that questions and challenges all its existing practices, regardless of how long things have been done that way. Such a culture will help your organization much more in the long run than just being concerned with implementing the ideas of the hour.

Pitfall 4: Overestimating AI's Capabilities

Given machine learning's popularity in the current tech scene, there are more startups and enterprises putting out AI-based systems and software than ever before. This generates a tremendous pressure to stay ahead of the competition, sometimes using just marketing alone. Although incidents of outright fraud are rare, many companies will spin their performance results to show their products in the best possible light. It can therefore be a challenge to determine whether today's AI can fulfill your lofty goals or if the technology is still a few years out. This fact should not prevent you from starting your AI journey since even simple AI adoption can transform your organization. Rather, let it serve as a warning to be aware that AI marketing may not always be as it seems.

Action Checklist

- ___ Start by building a culture of innovation in your organization. Ideas will come from the most unexpected places once this culture is set in place.
- ___ Form an innovation focus group consisting of top-level managers who have the authority to make sweeping changes.
- ___ Start maintaining an idea bank.
- ___ Gather ideas via scrutinizing standard operating procedures, process value analyses, and interviews.
- ___ Sort and filter the idea bank using well-defined criteria.
- ___ Do timely reviews to trim, refine, and implement ideas.
- ___ Learn about existing AI technologies to gain a realistic feel for their capabilities.

- Apply the idea bank to the AI models learned in the previous step to find those ideas that are suitable for the implementation of AI.

Notes

1. www.nytimes.com/2011/11/08/science/computer-experts-building-1830s-babbage-analytical-engine.html?mtrref=undefined&gwh=3AC8088201F9C684FE137681F50CAF1E&gwt=pay&assetType=REGIWALL
2. <http://ed-thelen.org/comp-hist/CBC-Ch-02.pdf>
3. www.csee.umbc.edu/courses/471/papers/turing.pdf
4. <https://crl.ucsd.edu/~saygin/papers/MMTT.pdf>
5. Image originally developed by Juan Alberto Sánchez Margallo and used without modification. https://en.wikipedia.org/wiki/File:Turing_test_diagram.png
6. Friedl, Jeffrey. 2009. *Mastering Regular Expressions*, 3rd Edition. Champaign, IL: O'Reilly Media.
7. www.allthingsdistributed.com/2016/11/amazon-ai-and-alexa-for-all-aws-apps.html
8. Audrey J. Murrell, Sheila Forte-Trammell, and Diana Bing. 2008, *Intelligent Mentoring: How IBM Creates Value through People, Knowledge, and Relationships*. New York: Pearson Education.
9. www.research.ibm.com/patents/
10. www.forbes.com/sites/johnkotter/2013/08/21/googles-best-new-innovation-rules-around-20-time/#555a76982e7a

Defining the Project

Now that you have an introduction to artificial intelligence and an idea bank in place, you are ready to take the next step toward implementing AI technology and harnessing its many benefits.

In this chapter, we will look at how to take an idea from your idea bank and construct a plan to actualize it. A methodical approach here will help you not to lose sight of your progress or allow your project to end up in the “never completed” bin. This step will build a high-level roadmap for the successful implementation and completion of any of your chosen tasks. Most traditional software development philosophies start with building an extremely detailed specification document, covering individual development tasks and coming up with estimates for each. This approach, however, requires a large upfront investment at the point in your project when you, by definition, know the least. This means that lots of the painstakingly identified details and minutia put into your document may well become irrelevant soon after the project starts.

For reference, you can probably imagine how such a detailed document would inhibit the development process in the context of AI models. Assume that after a couple of weeks, the selected AI model “X” specified in the requirements document is proven to be ineffective for the type of problem it is being considered for. In such a case, it becomes imperative to try different models in an attempt to find one that will fulfill the project’s goals. When using a requirements

specification approach, however, any change would invalidate the decisions and tasks dictated by this particular AI model. To emphasize the model's failure, we ignore any of its triumphs or potential. There must be a better way.

Today, these massive specification documents are being traded in for a much more flexible approach. Generally labeled under the umbrella term *Agile*, a lightweight project plan can be defined, leaving the specifics of implementation until later in the process. This approach allows the project to grow and evolve without being hampered by prior decisions. While being flexible enough to allow for future adaptations and evolutions, the project plan still provides guidance to keep the project on track.

Additionally, the project plan will emphasize the importance of being able to measure the success of a project. These metrics can be in terms of KPIs, financial returns, or some other qualification applicable to the project, but it must be present in order to accurately calibrate your approach. This exercise of measurement can be accomplished only if the goals are clearly defined, as stated in the initial project plan.

The What, Why, and How of a Project Plan

Once an idea has been selected for implementation, it is time to break it down into a project plan. The first step in this process is planning your objectives in a systematic manner. This project plan document will lay out the desired functions and production goals for your system. In this chapter, we will look at three approaches to create a project plan from an idea.

Your project plan should describe the objectives, the measurability, and the scope of your project. Its contents will aid the AI, data science, and software engineering teams in assessing the success and progress of the project. The project plan should not be restrictive, staying open-ended in terms of implementation specifics. This is a key feature of the software development philosophy called Agile, which we will discuss in more detail in Chapter 5, "Prototyping." Again, the aim is to provide enough initial guidance to be focused but not too much such that the project cannot adapt as needed.

Under Agile, there is no formal specification document. The only thing required to start a project is the project plan. A formal specification document and a project plan are not synonymous and should be treated differently. Designing a full-blown specification document for a project using Agile is a recipe for disaster. Management would try to stick to the implementation defined in the specification document, whereas the development team would see the necessary changes to make from their insights developing the system so far. A project plan will be a detailed document, but the key difference is the kind of detail being specified.

The project plan is the master document that will be used by the team (stakeholders, management, developers) to assess the progress of the project and plan ahead. It is a critical document at this early stage, and we must get it right.

The Components of a Project Plan

A well-defined project plan should ideally include the following elements:

A Project Scope

Detailing the extent of the project, the systems being implemented, and the project's overall intents and purposes.

Defined Work Schedules and Locations

Aiding in the development and implementation process, and acting as a roadmap for the *Scrum master* (a facilitator for the Agile development team) to steer the project and implement suitable infrastructure like videoconferencing and physical meeting rooms.

Project Oversight Activities

These activities include the following:

Project Governance

Who is responsible for the project? What are their responsibilities? This section should clearly state who the project owner is, as the reins of the project will be in their hands. A list of all the project stakeholders should also be included.

These are individuals, or a representative of those individuals, who stand to be either directly or indirectly affected by the project. For instance, if the project is creating an AI-powered system to improve supply chain efficiency, a representative factory manager should be included to get their perspective on the project and to find out if they have any requirements that must be included. In this example, the factory manager might need visibility into the predicted deliveries to fully realize the new system's benefits. Their input at this point could mean the difference between a successful and an unsuccessful project.

Project Kickoff

When is the expected start date of the project? Who should be included? It is much easier to get a stakeholder's buy-in when you have included them from the start, rather than having to win their support after the project's already been in motion.

Work Stream Activities

These are high-level user stories, sometimes referred to as *epics*, that will be worked on in the project (developers break them into smaller user stories before implementation). These activities will act as future goals for the development teams. Each activity should have defined deliverables (tangible or intangible products), such as code, documentation, and infrastructure readiness. Lastly, these activities should include success criteria to ensure that they can be decided on empirically if completed after the deliverables have been submitted.

For example:

Login Interface for the Chatbot

Goal: Users can log in securely to the application so that their information is not compromised.

Defined deliverables: A login page; documentation for necessary authentication maintenance; a list of planned updates for the web app.

Success criteria: The web app must pass security tests against known threats like SQL injection and cross-site scripting, with a response time of less than three seconds.

Assumptions

Every project has a set of assumptions (various tasks and items expected to be necessary to the project) that need to be listed before the project can commence. These may include code hosting, selecting cloud providers, and determining which type of workflow to be used. In AI projects, assumptions can include data availability and access to the correct subject matter experts (SMEs). SMEs are not typically found within the IT department of your organization, but rather are the experts of their own particular domains, likely spread throughout the organization or as external specialists in their fields.

List of Deliverables

All things that will be “delivered” at the end of the project should be included. This should be an exhaustive and detailed account so that the company does not end up with code that is undocumented or consequently unmaintainable. Sample deliverables include the code and all documentation related to it, maintenance guides, user manuals, and training to be imparted to users post-implementation. AI projects will also have models as deliverables. These models, along with documentation, can be contributed to a model library, which we will discuss further in Chapter 7, “Thriving with an AI Lifecycle.”

Project Completion Criteria

This section of your project plan will detail when the project will be considered “complete.” Provisions can also be included for the necessary, premature termination of a project due to budget overruns or changes in technology that have rendered the project infeasible. This section will contain the measurability criteria for KPIs and financial success, as will be discussed in the “Project Measurability” section. AI projects in particular can present certain difficulties in setting completion criteria. Because most AI systems can continue to be improved on with more data, you must be able to define a benchmark level of performance and accuracy that you and your organization will be comfortable with. For example, is a support service that is able to answer 95 percent of users’ questions correctly sufficient? AI provides

standardized metrics such as precision, recall, and an F1 score (details of which will be discussed in Chapter 7) to help define these success criteria.

Change Request Process

It is unnecessary and impractical to predict and plan for all contingencies while designing your project plan. To account for this, a formal change request process should be designed and specified here. This is critical under Agile, since frequent changes might transform the project into something completely different than what was originally envisaged, making it unable to reach its goals. The final call on such a process should rest with the product owner, as opposed to a committee of users. This will ensure the project stays on track and is not unnecessarily bogged down with changes.

Estimated Schedule

A list of the high-level activities mentioned earlier under “Project Oversight Activities,” with the projected start and end dates of each.

Charges/Budget

Assuming this is a fixed-budget project, this section of your project plan lists the high-level activities mentioned under “Work Stream Activities,” and the costs associated with each activity in terms of developer time, infrastructure cost, equipment costs, and so forth. If it is an internal project, it should include opportunity costs for the employees’ time, based on a calculated hourly rate. Just as it would for any external project, this section should cover all types of costs for an internal project, such as server costs, cloud hosting charges, domain charges, and other miscellaneous expenses like team dinners. AI projects should include the expected cost of SME time, allowing for multiple iterations. These SMEs will likely reside in other parts of the organization and are subject to other overhead requirements. Additionally, hosting costs for AI models can be sizable, depending on their complexity and performance requirements. Finally, the cost associated with any data licensing should be included.

Approaches to Break Down a Project

Sitting down to create a project plan from scratch can be daunting. Using our sample as your guide, or better yet, coupling it with a project plan from a previous project completed by your organization, you can give yourself a helpful framework on which to base your new plan. Despite this shortcut, it will also be convenient to have specific approaches you can follow that will facilitate your immersion into this particular project's details to ensure that all aspects are being covered. While there are many approaches to defining a project, we will introduce three that the authors have personally used.

Approach 1: Design Thinking

There are many approaches to using AI in your enterprise, and not all of them require an in-depth project plan to be completed before getting started. How your organization proceeds with adopting AI tools is ultimately at the discretion of your business and technical leaders. The concepts presented here, which are part of *design thinking*, are a good primer to get you started regardless of the process and methodology you choose to follow. Your team will likely come up with a wide variety of ideas for these kinds of projects. Use design thinking to flush out the best ideas and further refine them into a set of achievable goals.

Able to be used without any prerequisite knowledge, design thinking is a process used for developing new solutions to current problems and preempting issues before they arise. It focuses on understanding a problematic domain and then coming up with a fictional (but accurately representative) individual from that domain and brainstorming, in a collaborative environment, to find solutions that will make that individual's role/job/life easier. It has applications in many industries and has been used successfully for crafting AI solutions at both large and small organizations.

Sample Design Thinking Session

Design thinking sessions start by assembling potential stakeholders and identifying end users who will be using and benefiting from the

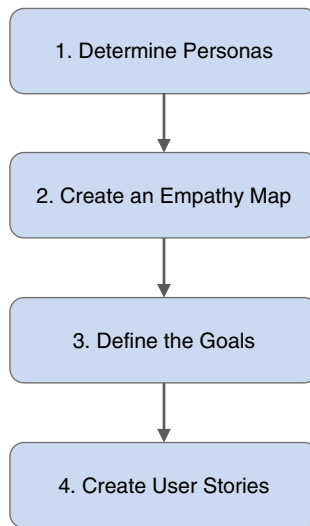


FIGURE 3.1 The Design Thinking Process

system. These are the personas. Personas should not be caricatures. They should be as close to a real person as possible. Empathy maps are then created for each of the identified personas. Empathy maps are a tool to visualize an end user's current state to determine their "needs" and "wants" for a particular product/service. The design thinking session ends with the creation of goals and user stories that address the needs of each of the personas along with an outlined project plan from those user stories (see Figure 3.1 for an overview).

Step 1: Determine Personas

Before starting any project, it is imperative to determine the users. Are they internal employees, lawyers at a firm, doctors at a practice, analysts at a financial company, or simply people coming to your website in search of a product or service? Delineating the users as internal and external will help determine which functions such a user is likely to use and select the appropriate metrics with which to judge the success of the system. This determination will drive every subsequent action.

Use market research to identify the user type, if appropriate, and get started listing the responsibilities and needs of these users.

Everybody should contribute ideas to the persona development, although not all ideas need to be included. The goal is to create a persona that is realistic and representative of the most common user types likely to engage with the AI solution. Use simple words and terms and make a list of a few “wants” that a person might have to make their job easier.

Each persona under consideration should have characteristics that pertain to a specific character-type or personality (e.g., accuracy of their reports, frequency of their interaction with the system, urgency of their use of the system, the “needs” and “wants” the persona will try to fulfill with the system, the various challenges faced by the persona that the system will try to resolve). The key here is to define a “user type” so that it is easy to generate clear and distinct user stories.

Let's start by considering an example company, Widgets ‘R’ Us, that is looking to expand their manufacturing to multiple sites but first wants to improve the efficiency of their current site. Widgets ‘R’ Us creates widgets and has a quality assurance (QA) team that monitors the outbound widgets for defects via a QA specialist's visual inspection. This process is fairly manual and fatiguing to the QA specialists.

For the purposes of this example, we will create a persona, Jake, who is a QA specialist. Not only must Jake spend the majority of his time doing visual inspections, he then also has to create a report at the end of each day outlining the number and type of defects he identified. This is currently a paper form that he completes. Once a month, Jake must tally his forms to create a monthly total, which Widgets ‘R’ Us uses to ensure defects stay under a tolerable threshold.

Step 2: Create an Empathy Map

Given that we now have our persona(s) and completely understand their roles and responsibilities, the next step in the design thinking process is to create an empathy map. An *empathy map* details what a person currently *does*, *thinks*, *feels*, and *says* with respect to their current situation. This crucial step involves putting yourself in the shoes of your users to truly understand their day-to-day activities. This step can be done directly with the design thinking participants

if they have personal experience as the persona(s) they are considering or through interviews, focus group style, with the actual users.

Here is a sample empathy map for our QA specialist Jake:

- *Does:*
 - “Visually inspects widgets for defects.”
 - “Creates reports detailing defect breakdowns.”
 - “Tallies defects each month.”
- *Thinks:*
 - “It is a pain to tally my reports every month.”
 - “I could be doing more if I only had to look at the potentially defective widgets.”
 - “Wow, this is repetitive.”
- *Feels:*
 - “Eyestrain from having to monitor *every* widget for defects.”
 - “Bored sometimes with the repetitiveness of the job.”
 - “Pride being able to ensure outbound widgets always work.”
- *Says:*
 - “Is there a better way than me manually writing down defect counts?”
 - “I have developed patterns that help me identify defects quickly.”
 - “I am able to identify most of the defective widgets, but some defects could be detected earlier.”

Step 3: Define the Goals

With the persona(s) defined and the empathy maps created, we now have a good idea of what challenges face our users. We now must identify the goals of the project that can help one or more of the users. For instance, the QA specialist can benefit from the following three goals:

- Build a visual defect identification system to send parts that are likely defective into the manual inspection queue. In this way, QA specialists will need to check only, say, 5 percent of the total number of widgets instead of 100 percent of the widgets. This is a 20-times reduction in manual work for the QA specialist.

- Digitize defect tracking so that daily reports can be automatically created based on the QA specialist's inputs.
- Replace the monthly report tallying with reports created automatically from the already provided digital data. This will completely eliminate this step from the QA specialist's list of responsibilities.

Step 4: Define User Stories

The last step is to formalize these goals into potential user stories. User stories outline a capability provided to a user, along with the concurrent benefit(s) that user can expect. User stories typically are written in the following format:

As a <role>, I want to be able to <feature> so that I can <benefit>.

Transforming goals into user stories might generate the following list:

- As a QA specialist, I manually inspect the widgets that the automated system identified as potentially defective, to ensure high-quality widget output.
- As a QA specialist, I enter identified defects into a digital system so that they are tracked and reports can be automatically generated.
- As a QA specialist, I review the monthly defect report to ensure that it aligns with what I inspected for the month.

Creating user stories using the design thinking approach, similar to this example, provides the necessary project plan content that can be directly acted on. Following the design thinking process can help answer questions like these:

- Will this AI project deliver a tangible return on investment (ROI)?
- If so, will the ROI be realized in the short or the long term?
- Am I working with a vendor who has delivered value similar to what my persona needs/wants?

- What is the scale of the impact delivered by a solution to address these needs/wants based on the number of individuals who fit this persona?

Design thinking is a helpful process for focusing your AI system to address the wants and needs of real users. Creating personas, defining empathy maps, creating goals, and finally identifying user stories turns ideas into actionable tasks.

Approach 2: Systems Thinking

If you followed the methods for generating process maps and organization charts for the ideas in the previous chapter, you can carry those practices forward to aid you as you develop a project plan as well. A business can be thought of as a system, with multiple complicated processes interacting with one another, activating internal and external triggers designed to help it reach a specified goal. All smaller processes and departments within that business would also be their own subsystems, with the affairs of other departments falling outside of the boundaries of each subsystem.

Imagining business processes in this way may help you translate business processes into computer code more easily. *Systems thinking* will aid in the formulation of project plans for larger, longer-term projects. Defining boundaries and setting expected outcomes also becomes easier with this.

Boundaries

In systems thinking, all systems have a boundary, past which they interact with external factors. The interactions can be in the form of input, output, or stress. Any given system should be able to handle all three interactions effectively to ensure its survival and continued usefulness. Failure to accept input and convert it to output will make the system useless. This boundary also helps to define the scope of the system.

Let's take, for example, an inbound boundary for an AI system. In this example, the AI consumes data from a transactional manufacturing system. This data is used to train predictive models, which are

used to detect when equipment is likely to fail and if manufacturing yield adjustments indicate additional issues. The results of the predictive models can then be sent to an executive dashboard system, which would serve as an outbound boundary.

External stressors should be mitigated through a combination of coping and evolutionary mechanisms. Some stressors for an AI-based system could come from interacting with outdated data structures, meaningless input (garbage in, garbage out), destructive viruses, and so forth. In our manufacturing example, an external stressor might be electromagnetic interference (EMI), which can cause inaccuracies in the source data or missing data altogether. It is pertinent to note any anticipated stressors at the time of planning the project so that they can be provisioned against.

Subsystems

No system works in isolation. At the very least, each one needs data from other systems to generate its own output. Therefore, we must consider whether the proposed system can be used with existing systems smoothly or if existing systems need modification. Such modifications should be considered part of the new project. This will help to ensure that a workable and complete system is delivered, instead of one where more investment is necessary to make it start working.

An important note when employing systems thinking is to think of the largest system possible and then work toward each of its smaller parts. These smaller parts are called subsystems, and they will be the starting point for all user stories generated for Agile. It should be possible to break down any complex system into its constituent parts. These parts are linked and exchange input and output between one another. The goals of any subsystem should always be in line with the goals of the larger system it is a part of.

At MicroAssembly Inc., the production manager, Claire, is looking to get better yield from the production process. She starts by looking at the entire manufacturing process as one large system. This system would have stores, vendors, and so forth as outside its boundary. The inputs are the raw materials, and the outputs are waste and finished products. The factory has been outfitted with devices and

microcontrollers to ensure that the machines are operating at desired levels. Within the large system of the manufacturing process, each subprocess will be a smaller subsystem, with its own input from the previous process, and its own waste and intermediate product as the output. Claire decides to apply artificial intelligence by collecting all the data from the sensors and systematically sorting it through subsystems and sensors from each subprocess. This will give her an answer as to which subprocess she should monitor more closely and tweak to get higher yields.

In this way, artificial intelligence becomes a new tool to help optimize subsystems that were likely already optimized in the past using different techniques. This is an interesting exercise, because efficiencies are quantifiable. Therefore, you can empirically measure and report the benefits of including AI.

Approach 3: Scenario Planning

Primarily developed and used by the military to simulate war games, *scenario planning*, or *scenario analysis*, can also be used in the business domain to make strategic and tactical plans. Scenario planning is the art of predicting the future through current trends and information, and extrapolating them into estimations about the future. Scenario planning involves three major steps: assessing the present circumstances, choosing and modeling key drivers, and creating scenarios based on expected changes to drivers.

The analysis of the present circumstances can be done using the STEEP model, which comprises sociological, technological, economic, environmental, and political inputs. The basic assumptions derived from this analysis will generate the drivers. The drivers are the factors that will have an impact on the scenario. By tweaking the drivers, we can extrapolate, predicting future trends and generating scenarios for them.

The Delphi Method

Combining scenario planning with the Delphi method will lead to more accurate predictions. The Delphi method involves creating a panel of experts to participate in multistage, anonymous written

surveys, with an impartial judge communicating the results. After every survey round, the experts should modify their answers based on new information gained from other experts. This will lead to a convergence toward the median opinion. The survey rounds will stop once the preestablished survey criteria have been reached, such as a specified number of rounds and adequate convergence of opinion.

Using scenario planning with the Delphi method will generate long-term projects and present the opportunity to drill those down into user stories. Such a strategy is useful as a resilient process in complicated and nuanced fields like health care, with much better results than hiring one technical consultant. A panel of doctors would be better able to answer which diseases are indicated by which symptoms. The data to base the user stories on would be of excellent quality. Having a panel of experts avoids the trap of data corruption due to personal biases or incomplete knowledge from individual experts.

In scenario planning, at least seven scenarios should be generated based on modifying the drivers and predicting the outcomes of such modifications. This should be further narrowed down to four workable and realistic outcomes, which would form part of a 2x2 matrix. The four outcomes need not be complete opposites, or “good vs. bad.” This is indicative of bad analysis since the models are a representation of the future and reality is rarely binary. The matrix will aid in decision making by averaging and clarifying the ideal solutions for the problem in question.

With the three types of approaches discussed (design thinking, systems thinking, and scenario planning), you will be more than equipped to make a detailed project plan. The three approaches are not mutually exclusive either and can be used together for better results. Design thinking is primarily used for coming up with innovative processes and systems that have never been implemented before. Systems thinking is more suited to coming up with improvements to existing systems and processes. Scenario planning can be used to plan for projects that revolve around contingent events like disaster recovery procedures and business recovery planning. In an ideal situation, you can start with whatever approach best applies to the case you are looking to solve. If you hit a dead end, there is no harm in using cues from the other methods.

Project Measurability

A good project plan will have metrics set at the beginning to measure the project's successes and failures. This critical part of the project plan will assist in review of the project after its completion and, in the case of project failure, will aid in a root cause analysis.

Metrics need not only be based on cost. In the modern business world, they can also be linked to KPIs if the benefits derived from a project do not translate directly into cost savings for the company. For each project, critical success factors need to be identified, and the impact of the project should be assessed based on those factors. Some examples of KPIs that could be used are

- Customer satisfaction
- Reports accuracy
- Employee performance
- Time saved by employees
- Time taken to resolve a support query

These last two indicators could be dramatically improved by an AI chat system that automatically answers straightforward customer questions. Not only do automated chat AIs respond immediately, they do not require time from employees. Instead, an employee is involved only if the AI cannot handle the question. In this way, these two KPIs would be great for a project looking to automate some of their chat support. Additionally, it is important to include the first example KPI, customer satisfaction, because an automated system is not worth the saved time and money if customers hate it. This is a good example of needing a broad set of KPIs that encompass everything that matters in your organization so that one is not maximized to the detriment of another.

Every project and company is different. The example list is intended just to give you an idea of various KPIs that can be used to judge the success of a project. It is very important to set measurable goals for any qualitative factors in terms of numerical values so as to be able to judge and measure them empirically. This can be done in the form of surveys, in which the concerned individual marks the performance on a scale of 1 to 10, or by indirectly judging the

performance based on parallel indicators like time spent waiting for resolution of a support question.

Balanced Scorecard

A balanced scorecard is a document that will help you organize metrics in a holistic and comprehensive manner. The four perspectives of a balanced scorecard will ensure that the standards for a project are set properly:

Financial Perspective

This would include cost savings, increased sales, higher returns on investment, and so on. Most business organizations will be able to easily come up with measurements of success for this perspective.

Customer Perspective

This is a more difficult perspective to measure and study. It should be studied with the aid of tools like market surveys and through objective evaluations of the quality of products built by the company compared to other competitive products available in the market. Another useful metric for this perspective is the share of the market captured by the organization. Judging whether the company's share is increasing on a consistent basis can offer lots of insight in reference to its customers.

Internal Process Perspective

This angle aims to judge the benefits generated by a project to existing processes. Time saved, efficiency in production, and reduction in servicing time to customers due to improvement in processes will all be measured here.

Learning and Growth Perspective

The growth of the organization as a direct consequence to the project will be measured here. This includes increases in institutional learning as the project is rolled out to production. Organizational memory is fickle if it remains only with employees. A well-documented system will accrue many benefits and should be measured in this section.

Building an AI Project Plan

Given that we are in the age of AI, we can focus on taking our project plan a step further. Using the techniques explained in the previous sections, we can start building a solution that will help us solve problems faster and more efficiently.

Today, AI capabilities allow us to completely automate a chat conversation involving simple, standard questions. For instance, when a customer asks “What time does the Manhattan branch open?,” this query can be handled exclusively by the chatbot system. First, it would analyze the user question to understand the intent of the inquiry. This intent identification comes with a confidence value for how certain the chatbot is that it knows what the user is asking. Typically this confidence comes in the form of a decimal between 0.0 and 1.0, where 0.0 is completely uncertain and 1.0 is completely certain. If the confidence is above the established minimal threshold, the chatbot can search its response database (seeded with responses from the aforementioned FAQs) for that intent and determine an appropriate response. Furthermore, information about the user’s current context can provide additional input to the response search process, customizing the experience. For instance, if the system is able to determine a user’s location within Manhattan, the system could provide the opening times for the three closest branches if there are multiple branches in a reasonable proximity. Having this capability frees up the organization’s chat specialists to focus on the more complex user issues, which unequivocally results in a cost savings and an increased likelihood of customer satisfaction.

For example, ABC Bank uses a standard approach for customer communication using a traditional interactive voice response (IVR) system, which eventually ends in the customer connecting to a human. The CEO decides to further automate this process with a support chatbot project. A meeting is called to gather data about current customer behavior from the IVR system and the customer interactions in branch offices. Employees notice that a majority of customers call to ask their balance, to check their previous transactions, or to request a statement over email. The bank can also group their retail customers into categories based on the banking services used. Personas are developed based on the questions asked by the customers in each group. Following the design thinking process, the

bank analyst then generates empathy maps with the personas' thoughts, feelings, statements, and actions.

Using this analysis could yield the following customer user stories:

- As a customer, I want to have access to a human if the chatbot is not fulfilling my needs.
- As a customer, I want the chatbot to have access to my account data so I do not have to manually enter account numbers or other data it should already know.
- As a customer, I want the chatbot to learn the kinds of questions I ask over time so that it can handle them directly instead of me having to be transferred to a human every time I have the same request.
- As a customer, I want the chat interface to be secure so that I can feel confident that data I send to the chatbot will not be abused.
- As a customer, I want to be able to communicate with my chat client of my choice (text message, Facebook, Messenger, etc.) to get an answer where I am most comfortable.
- As a customer, I want my problems to be resolved in a timely manner since my time is valuable.

With the user stories established, the project's scope can be written up:

This project is to create a learning chatbot that will answer questions posed to it by customers. The chatbot should be able to learn and adapt its behavior for the efficiency and efficacy of its responses as per the needs of the customer, based on qualitative feedback.

The success criteria for the project can also be established here:

- The chatbot should be able to answer a question or make the decision to assign the question to a human representative within three seconds.
- The human customer support representative should have their customer interaction volume cut in half.
- Customer satisfaction should increase by 10 percent.

Once the project plan is developed, it is time to select a few user stories and start prototyping the solution.

Pitfalls

Here are some pitfalls you may encounter during project definition.

Pitfall 1: Not Having Stakeholder Buy-In

AI solutions tend to affect all parts of an organization. Their transformative nature requires data from multiple groups and stakeholders. As a whole, organizations are often resistant to change, so it is important that each stakeholder's input is incorporated at the earliest stages of the project. It is in every project's best interest to ensure that its projected benefits have been clearly explained to everyone involved. The only possible path to shaking the fear of change is to explain why that change is in the best interest of the stakeholders. The current users of the system might be facing a problem that the proposed system is not addressing. New research may have revealed an alternative revenue prospect that your organization was previously unaware of. Whatever the cause for change, in the event that a stakeholder is not consulted beforehand, they may become the biggest roadblock for your project's adoption. The effects of this pitfall will likely not be felt until you've deployed your solution in production during step 5, but it is important to address this pitfall during the project definition step, before it has time to snowball.

One way to avoid this is to have an initial kickoff meeting that includes all possible stakeholders. It is important to be liberal with the attendee list because this is the meeting where everyone will formerly discuss the project for the first time. Although people may have heard about the project in hallway conversations, they must at some point be officially informed about the project and invited to participate in its development and success. Keeping people in the loop, with a clear channel to propose comments and suggestions, will enable them to have a sense of ownership with the project down the line during its implementation.

Pitfall 2: Inventing or Misrepresenting Actual Problems

One hazard when exploring a new technology is searching for a problem you can solve with it, rather than the other way around. Just because you have this shiny new hammer does not mean the world is suddenly full of nails. The focus, as we have discussed, should be on the pain points of your organization. Decide what problems exist or what opportunities are out of reach, and then develop a solution to resolve the problem. If you avoid this advice, you will end up trying to fix things that are not broken or addressing problems that never even existed in the first place. Do not try to fix what is not broken; that will only lead to extra costs and delays. Having a firm grasp of what the process flows of the company look like will help you target only the deficient areas.

Pitfall 3: Prematurely Building the Solution

As you work through developing your project plan, you may be peripherally aware of services and commercial technologies available today that provide AI capabilities. It would be a mistake, at this point, to select a particular vendor with whom to partner. This is a common mistake made by organizations, who, by selecting a vendor too early, unintentionally limit themselves to the capabilities offered by that vendor. Instead, continue focusing exclusively on developing your user stories. Think about the users of your system and how you can make their lives easier. If you know your users use social media, for example, consider making social media integration part of your scope. Establishing these requirements during the project definition phase will greatly simplify selecting a vendor once you reach that stage.

Another risk you run in prematurely selecting a vendor is being tempted to include features available through your vendor that have no relevance to your users. For example, if that chat technology you found provides a text-to-speech capability, you might be tempted to include that in your project, not because it is helpful to your users, but because you want to fully take advantage of your investment. This runs the risk of complicating your efforts, obscuring your project's aim, and unintentionally spending your limited resources on

capabilities with minimal value. Again, your focus at this stage must be your users and defining your project plan, not on picking a technology or partner.

Pitfall 4: Neglecting to Define Formal Change Request Procedures

Change is a natural part of the Agile process. Although this change can be a great thing, it is also possible for projects to become bogged down or impeded by a lack of consistency. If the requirements of a project change too often, or without sufficient cause, that can lead to developer confusion and a failed project. For this reason, it is vital to establish a lightweight but formal change request procedure. Every change should be approved by the product manager, as they have the ultimate responsibility for the project. The product owner should scrutinize and evaluate each change request carefully to assess the impact on the project. Requests that have no clearly illustrated necessity or benefit should be ignored and not passed on to the developers.

Pitfall 5: Not Having Measurable Success Criteria

For large and small projects, it is imperative to assess their impact on the organization. Such an assessment will identify shortcomings and lessons for future projects. In order for this kind of assessment to take place, the project's scope and success, as discussed earlier in the section "The Components of a Project Plan," need to be defined clearly. Projects can fail. It is what you learn from each one as an organization that will separate you from the rest. People who are afraid to fail might neglect to set empirical goals by which to measure their success or failure. However, it is vitally important for you to know if you are off track sooner than later so that you can make course corrections. Agile is built around the philosophy of fail quickly, fail regularly, and fail tiny. Small failures are easier to recover from and allow learning in a nondestructive manner.

Action Checklist

- ___ Identify the idea to be implemented.
- ___ Identify all possible stakeholders for the system you want to build.
- ___ Select the appropriate method for making the project plan (design thinking, systems thinking, scenario planning, etc.).
- ___ Use design thinking (if applicable) to come up with persona(s) who will be using your system.
- ___ Define and prioritize measurable user stories that, when implemented, will provide user value.
- ___ Establish success criteria for the entire project.
- ___ Finalize the project plan and begin prototyping.

Data Curation and Governance

When building an intelligent system, there are two main components. The first is the collection of algorithms that build the machine learning models underlying the technology. The second is the data that is fed into these algorithms. The data, in this case, is what provides the specific intelligence for the system.

Historically, the field of machine learning has focused its research on improving the algorithms to produce increasingly better models over time. Recently, however, the algorithms have improved to a point where they are no longer the bottleneck in the race for improved AI technology. These algorithms are now capable of consuming vast amounts of data and storing that intelligence in complex internal structures. Today, the race for improved AI systems has turned its focus to improvements in data, both in quality and volume.

Due to this shift in focus, when building your own AI system, you must first identify data sources and gather all the data necessary to build the system. Data that is used to build AI systems is typically referred to as *ground truth*—that is, the truth that underpins the knowledge in an AI system. Good ground truth typically comes from or is produced by organizational systems already in use. For instance, if a system is trying to predict what genre of music a user might like at a particular time of day, that system's ground truth can be pulled from the history of what users have selected to play throughout the day in the past. This ground truth is genuine and representative of

real users. If no existing data is available, subject matter experts (SMEs) can manually create this ground truth, though it would not necessarily be as accurate.

After it is selected, the ground truth is then used for training your AI system. Additionally, it is a best practice for a percentage of the ground truth to be reserved for validation so that the AI system's accuracy can be empirically validated. Now that you understand the importance of data to your machine learning system, let's discuss how to find and curate the data you will be using to fulfill your use cases.

Before continuing with our ground truth discussion, let's investigate a particular type of machine model called a *classifier*. A classifier groups an input into one of two or more possible output classes. A simple classifier might distinguish between pictures of chocolate or fruity candies. This classifier has two possible output classes: (a) chocolate candies and (b) fruity candies. A classifier like this could be further developed or extended to recognize pictures of specific types of candies such as (a) M&Ms, (b) Reeses, (c) Snickers, (4) Skittles, (5) Starbursts, and (6) Gummy Bears. Now that we have covered the concept of classifiers, we'll continue with ground truth.

There are typically two key methods to build the distribution of your ground truth when building an AI model. The first is to have a balanced number of examples for each class you want to recognize. In the classic example of recognizing pictures of handwritten characters (also known as the EMNIST¹ dataset), having an equal number of training examples for lowercase letters would mean having approximately 1,000 examples of each letter, for a total of 26,000 training data samples. This would be considered a balanced ground truth.

Alternatively, instead of a balanced ground truth, your training data could be proportionately representative of how your system will be used. Using the same lowercase letters example, a proportional ground truth would mean there would be more *s* training data samples than there would be *q* training samples because *s* is a more commonly used letter than *q* in the English language. However, if the AI system was to be used for a different language, such as Spanish,

the proportion of training samples for each letter would have to change to ensure that the ground truth continued to be representative of the users' inputs.

Typically, both balanced and proportional ground truths are valid approaches, but there is one scenario where balanced can be advantageous: in the case of outlier detection, where outliers are so uncommon they make up less than 3 percent of the total, real-world distribution. Looking at an example of image analysis for skin cancer detection, it is likely that most submitted images will not denote a form of skin cancer but that a few will. In this case, a proportional approach will likely not have enough training samples to intelligently recognize the outlier. Instead, including additional training samples of the small class (images representing skin cancer) will be important. Because the underrepresented class is an outlier, you might have a hard time getting an equal number of training examples, but including more samples will improve your AI system's overall accuracy.

Data Collection

Before you can start to curate the data for your machine learning system, you must first identify and acquire it. Data can not only come from your own organization, but it can also be licensed from a third-party data collection agency or consumer service, or created from scratch (see Figure 4.1). In fact, it is not uncommon for an AI system to depend on data from all of these sources. For a better look, let's examine each of these approaches in greater detail.

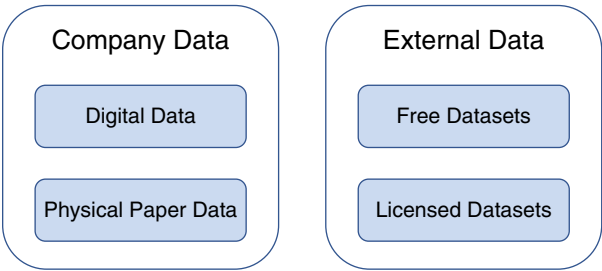


FIGURE 4.1 Data Available for Training AI Models

Internal Data Collection: Digital

With technology touching every part of our personal lives, it makes sense that it is also ubiquitous within our organizations. Technology helps make us more efficient and simplifies some of the mundane processes that keep our organizations moving. This is some of the same value you are planning to harness by reading this book and adopting AI systems.

Fortuitously, there is an amazing side effect to having all these technological systems in your organization. They typically generate a significant amount of valuable data. Sales, manufacturing, employee, and many other systems all generate data. It is kept in many forms, from structured databases to unstructured log files. In this era of big data, the value of data is known, and therefore most organizations default to saving all their data for future use, regardless of having any immediate use for it. It is impossible to go back in time and save discarded data once you have found a use for it.

Some organizations choose to take a more active role in data generation. They seek to equip themselves with valuable data to improve the efficacy of business decisions they make. One of the primary growth areas for this has been with the Internet of Things (IoT) and networked devices. Companies who distribute their products, whether they be hardware or software, are able to collect enormous amounts of usage data—everything from the time of day a person uses the product to the specific functions being performed and in what order and, sometimes, even geospatial data as well.

What happens, however, if you want to build an AI system but have not started to make data collection a priority? Even in this scenario you may already have more data than you think. In this scenario it makes sense to do an internal data exploration and come up with a data collection strategy for your organization. The two parts of this data exploration will be digital systems and manual systems.

The first part of data exploration consists of identifying and listing all existing digital systems used in your organization. With this list in hand, inquire as to what data is being stored internally by this system. Again, this can be data explicitly being stored by the system (e.g., customer records) or just system usage data that is being saved in log files. Ask if this data is easily accessed or exported so that it

can be used within other systems (e.g., the AI system you are building). Some possible access methods include the following:

Application Programming Interface (API)

The best-case scenario for data integration is that the existing system provides a well-documented API to access the data. APIs are preferable since they are secure, are easily and pragmatically accessed, and provide real-time access to data. Additionally, APIs can provide convenient capabilities on top of the raw data being stored, such as roll-up statistics or other derived data. Since APIs (especially those based in HTTP) are one of the newer methods of exposing data, legacy systems will likely not have this capability.

File Export

If a system does not have a convenient API for exporting data, it might have a file export capability. This capability is likely in the system's user interface and allows an end user to export data in a standardized format such as a comma-separated value (CSV) file. While this method is officially supported, only certain data may be able to be exported, instead of all possible internal data. For instance, highly structured data with a lot of internal structure may be harder to export in a single file. The other downside of this method is that it will probably not be easy to access programmatically, and therefore will have to be manually exported periodically. This might not be a problem if you are looking for monthly reports, but the reports will never be current in the way that a real-time dashboard is.

Direct Database Connection

If the system does not provide any supported data exporting capabilities and it is infeasible or monetarily ineffective to add one, you could instead connect directly to the system's internal database if one exists. This involves setting up a secure connection to the database that the system uses in order to directly access database tables. While this data is structured, and therefore easier to work with, you will likely have to reverse-engineer the internal table schemas to see what data is available. Before reverse-engineering the database's structure, however, speak

with the system's vendor to see if they have any documentation they can share or an SME. An important point to keep in mind is that you should access this data in a read-only fashion so you don't inadvertently affect the application. Additionally, you must be aware that the system might programmatically transform the internal data before it is displayed to the end user. For instance, if there is a field in the database called `<code>circle_size</code>`, it is ambiguous whether this is the radius or the diameter of the circle. Furthermore, the units of the circle are unknown. Is the value in the database in inches, centimeters, or what? Without documentation like source code from the vendor, your only option is to map that value into the UI of the system and reverse-engineer the mapping.

With the digital systems' data accounted for, it is time to start looking at what nondigital data is available.

Internal Data Collection: Physical

The next step in data exploration is to identify potentially existing manual processes used within your organization. For example, doctors' offices often collect insurance information each year from their patients using a physical form. This process generates a large amount of data, which, if not digitized after using manual data entry, is locked away in its physical form. You may find that a number of the systems in your organization have physical components as well. Perhaps your employees' timecards or inspection report forms coming from those in the field are physical. This data is just waiting to be included in an analytical system.

This data is valuable, but a decision must be made to determine how valuable it is. Data in its physical form must be digitized before it can be used to create an AI system. However, digitization is not a trivial task for a large organization, or for smaller ones that have been around a while (e.g., doctors' offices). Therefore, the decision needs to be weighed against the amount of time (and cost) required to digitize the data. Is this historical data relevant? Is it sufficient to just start collecting data in a digital format going forward?

If you do decide the physical data is valuable, then it makes sense to start using a digital system that can automatically record the data, thus removing the manual data entry piece. For instance, employees in the field might start using tablets with a mobile application to track their hours and inspections. In this way, not only is the data saved in a historical record, but it is also immediately available for dashboards and other systems to consume. Implementing these new systems will not happen overnight, but the earlier you can invest in such systems the more value from data and increased efficiency you will realize.

Data Collection via Licensing

If you have not been collecting data, or you require data that you are unable to collect internally, one possible option is data licensing. Many companies are built with the business model of selling data to support their operations. For instance, the company OneSignal² provides easy-to-use mobile push notification capabilities to developers for free. This is because OneSignal makes their money by selling aggregated data such as generalized phone numbers and usage times. Other free consumer services, such as Facebook, monetize by building highly detailed profiles for their users and allow advertisers to use that data for extremely effective marketing (a method that has come under scrutiny in recent times). In the age where consumers have come to expect free services, this is an implicit assumption outlined in privacy policies. There is a common saying in Silicon Valley:

If you are not paying for it, you are the product.

This personal information (albeit typically anonymized) can also be made available to be directly licensed to others, including your organization.

To determine if data licensing is feasible for you, you can first see whether any sites or companies have a freely available dataset that fits your purposes. You might be surprised how many datasets are free on the web today. Typically, free datasets are published by public

institutions, within academia, or by data enthusiasts. Here are some good places to start:

- [data.gov](#)³: Over 200,000+ public datasets
- [Kaggle](#)⁴: A data science competition website that makes competition data available
- [Awesome Public Datasets](#)⁵: An index to a number of accessible datasets

Although these datasets might be incomplete or consist of older data, they can be a good starting point to vet potential ideas.

If you are lucky and the data license is agreeable, your licensed data search could already be over. However, if you are not so lucky to find a free dataset that works for you, then it is time to look at data licensing companies. There are a number of them, ranging from small, focused dataset companies such as YCharts,⁶ which compiles company data for sale as CSV files to large media organizations such as Thomson Reuters, which owns a vast library of content. Thinking outside the box, approaching tech companies in the particular area you are focusing on could also be relevant. For instance, if you need transportation data, licensing data from one of the ride-sharing services might be an option. If you need geospatial images, a number of satellite companies provide imaging at various resolutions. Assuming you are not building a product that directly competes with them, it is likely they will be amenable to you using their data.

If you can find a company that likely has the data you require but is not openly advertising licensing opportunities, it is worth starting a dialogue with them. Perhaps they are bound by current privacy policies, or perhaps they just have not thought of that monetization strategy. If they do not know the demand exists, they might have just written off the idea prematurely. Regardless, starting a partnership dialogue is a good first step.

One of the main disadvantages of licensing non-free data is that data can be costly. The big data and AI movements have made data's value more apparent than ever. That said, data licensing pricing, especially in larger deals, can be negotiated based on how you are planning to use the data. For instance, using data to build a machine learning model instead of directly displaying the data to your end

users might be cheaper. Using any derivative or aggregate data from that dataset might be more affordable. Additionally, your use case might play into their pricing. Using data to feed a system that is used by a small team of 10 internal employees might be cheaper than, say, if that same data is being displayed to 1 million end users. Scale-based pricing might be advantageous to you and also allow the licensing company greater upside based on your success. Lastly, data recency can affect price. Licensing static data from last year will likely be cheaper than receiving a real-time data stream. Again, all of this depends on your use case, but data licensing companies want to set prices that establish a long-lasting economic benefit for both parties.

Another potential disadvantage of licensing data is that you are somewhat beholden to the licensing company for their data. Unless a perpetual license is established with defined payment terms, renegotiation will occur at some point. If, for whatever reason, a new agreement cannot be reached, the system you built using this data may all of a sudden become useless. Although this is an unlikely scenario, it is important to keep in mind that this is a risk with licensing data. One way to hedge against this risk, if your use case allows, is to just use the licensed data to *bootstrap* your system. That is, use the licensed data to build the initial system, but also collect usage data from your AI system that can be used later instead. That approach will enable you to use the data you own and choose whether you want to renew your data license.

If you do not have the data you require, whether you have not been collecting it or it is impossible to collect, data licensing can be very effective.

Data Collection via Crowdsourcing

We have covered obtaining data from within your organization and data licensing, but what happens when you do not have the data you need and cannot license it due to unavailability, poor quality, or cost issues? In this case, crowdsourcing technologies might be applicable.

Crowdsourcing platforms consist of two different types of users. The first are users who have questions that need to be answered. For instance, if I am trying to build an image classifier to categorize an image as daytime or nighttime, I need to make a labeled training set consisting of daytime images and nighttime images. All I need to

make use of a crowdsourcing platform are my unlabeled daytime and nighttime images. I can then create a job in the crowdsourcing platform specifying the question “Is this a daytime or a nighttime image?” The crowdsourcing platform then notifies their users that an image classification job is available.

The crowdsourcing platform’s other class of users are the humans who will be answering these questions. In this way, they are imbuing your AI system with their intelligence. These users are monetarily incentivized to answer questions quickly and with high accuracy. Typically, the same question is asked to multiple people for consistency. If there are discrepancies for a single question, perhaps the image is ambiguous. If one particular user has many discrepancies, this might mean the user is answering randomly and should be removed from the job or that the user did not understand the prompt. Crowdsourcing platforms use the power of large numbers to ensure accurate responses to the questions being asked.

After a sufficient number of users have answered the question for each data point in your dataset, you then receive a job summary. This summary includes the individual responses as well as a summarized view of the judgments for each data point. This data can be used to train your AI system.

There are a number of crowdsourcing platforms to choose from. They range across a spectrum of “cheap with low-quality answers” to “expensive with high data quality.” A few you might want to look into are Figure Eight,⁷ Mechanical Turk,⁸ and Microworkers.⁹

Although crowdsourcing can be a great way to label datasets so that they can be used as training data for machine learning systems, it does have some other limitations. For instance, it is hard to use a crowdsourcing platform if you have zero data to start with. That is, crowdsourcing jobs tend to take the form of a survey where you have users either give some judgment, based on data, or complete some menial lookup task and provide you with the result, based on data. If you truly have no data, you can take a look at existing intelligent systems to seed your crowdsourcing job.

Leveraging the Power of Existing Systems

Related to the idea of free datasets, there are already a number of intelligent systems available that can be used to generate a dataset. For instance, Google search results can give you a dataset of pages related to a particular keyword. A starter dataset of images can be created similarly from an image site such as Google Images or Flickr. Just make sure you select the appropriate license filter depending on your use case (e.g., *labeled for reuse*).

Let's continue our previous example of building a daytime/nighttime image classifier. If we need a set of labeled daytime and nighttime images for training, we could mobilize users on a crowdsourcing platform to label images for us, or we could use the power of an existing system. We could use Google Images with a search of "daytime pictures" and then use a browser extension to download all the images on that page. With all the daytime images downloaded, a human can easily glance through them and throw out any images that are not "daytime." This process can be repeated with a search of "nighttime pictures" to get a collection of nighttime images. In the span of 30 minutes, it is easy to build your collection of 600 images (300 of each type) to build a daytime/nighttime image classifier.

Although this approach can be a great way to obtain labeled training data quickly, you may choose to ultimately license a dataset that is already labeled and ready for training. Even if that is the case, using intelligent systems to generate a quick dataset can help you test an idea without a large upfront investment. In this way, many competing ideas can be heuristically whittled down to those that are most promising.

The Role of a Data Scientist

With all these data curation and transformation tasks, it is important that you have someone skilled to complete these tasks. *Data scientists* are pioneers that work with business leaders to solve problems by understanding, preparing, and analyzing data to predict emerging

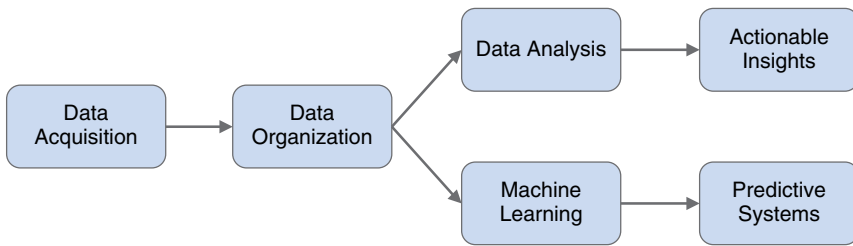


FIGURE 4.2 The Typical Data Science Flow

trends. They are able to take raw data and turn it into actionable business insights or predictive models, which can be used throughout the organization. An example data scientist's flow is shown in Figure 4.2.

Data scientists also apply AI technology to supplement human intelligence for more informed worker decisions. Data scientists typically have a background in the following:

- Data storage technology
- Machine learning/deep learning
- Natural language processing
- Numerical analysis and analytics software

Data science has a place in every modern organization, and if your organization does not already have one, it makes sense to start looking. Familiarity with your existing data systems is a plus, but most experienced data scientists will be able to get going quickly with new data environments. Analysis languages tend to have similar capabilities, and libraries primarily differ only in syntax.

Just remember that as your data volume grows, the more data scientists you will need.

Feedback Loops

As with software development processes, feedback loops are very important while developing an AI system. The quality of the output generated by an AI system is based on the dataset used to train it. A bad training dataset will lead to all kinds of disasters.

One goal of a data scientist is to shepherd this process and ensure that data quality and integrity is maintained with each feedback loop. Each loop is a sprint toward the stated objectives, and at the end of each sprint, feedback should be given by either end users or SMEs to ensure maximum benefit from the adoption of Agile loops. The feedback thus generated should be constructive, focusing not only on “what went wrong” but rather on “how to improve the next iteration.”

Feedback loops will aid the organization in creating a faster usable program. The loop will ensure that errors are caught early through regular reviews of functionality. This helps reduce cost escalations due to errors in the project. Error costs occur when development time needs to be spent to correct past mistakes. Via feedback, the project managers will have information that will allow them to course-correct sooner. As feedback is given at the end of every sprint (typically once every two weeks), this ensures that engineering time is not being spent on unproductive or “rarely used” features.

SMEs are central to this process. They will help the engineers find gaps and inaccurate predictions by the AI. The feedback given by a SME should hold the most weight, and future development sprints should be done after taking this feedback into consideration.

Abraham Wald was a statistical expert who worked on wartime problems during World War II. American bombers were being evaluated for structural reinforcement. The initial plan was to reinforce the areas of the planes that were hit with bullets. Wald came to the conclusion that only the planes that survived were available for analysis and those that were shot down completely were not. Hence, it would not be prudent to limit reinforcing the areas with holes, as the planes had clearly survived despite them. Wald, an SME, helped save costs and put into place a better scheme for reinforcement around the engines since those were more likely the planes that did not return from the battle. Feedback given by SMEs should not be ignored.

Feedback should be constructive and not merely criticize the existing program. The outlook should be on the future, but care should be taken not to fall into the sunk cost fallacy. Sunk costs are costs that have already been incurred in the past and cannot be recovered. Since such costs cannot be recouped, future decisions should not necessarily be based on them. To illustrate, say a support department decides to develop a chatbot. After two months of development,

the project manager should not be afraid to abandon the entire project if met with an impassible problem or when potential gains will likely be outweighed by the costs of completion. Past costs that have already been incurred cannot decide the future viability of a project.

A feedback loop can help prevent the butterfly effect from ruining the project. The nature of AI code dictates that a small error at the start of a project, like choosing the wrong dataset, can expand the errors in the latter stages of the project. Errors are cheaper to fix at the beginning of a project. Therefore, a feedback loop will help to detect and correct errors at the earliest stage possible.

When an AI program is in the learning, or “training,” stage, it can become overfitted. *Overfitting* is the technical term for an AI that has developed models that work only with the training and validation datasets. In such a case, feedback about increasing the variability of the dataset or changing the model parameters should be given.

Feedback should be given as soon as testing begins. Feedback must be forward-looking, clear, concise, and direct. It should focus on the output of the AI and whether the development is in line with the objectives designated at the start of the project. At the end of every sprint, the system should be tested thoroughly and suitable course-corrections should be made.

Making Data Accessible

After the data is collected, it is typically stored in an organization’s data warehouse, which is a system that colocates data in a central location to be conveniently accessed for analysis and training. Regardless of how the data is stored, the important point here is that the organization now has all the data it needs in a central, accessible format. Keeping necessary data accessible is critical when building AI systems.

Having data and being able to use it to train an AI system are two different things. Data in an organization can sometimes be siloed, meaning that each department maintains their own data. For instance, sales and customer data might be stored in a customer relationship management (CRM) system such as Salesforce, whereas the operations data is stored separately in a shop floor database. This division makes it hard to have a holistic view of all the data within an organization.

For an example of the limitations of siloing data, imagine trying to link from a customer's record to that customer's manufacturing yields. This would be nearly impossible in a silo-structured system. If this were the goal, the organization would need to build a data platform that collected and compiled all siloed data into a central location. Platforms such as Apache Hadoop¹⁰ provide a method to create an integrated schema and synchronize data. Setting up this data and unlocking its value is a task well suited to the newly established role of the data scientist.

Data Governance

Data being the cornerstone of artificial intelligence, it is important to understand the ethical and legal ramifications of obtaining and using any data. *Governance* is a term that has been applied to a number of areas of technologies. Governance is about ensuring that processes follow the highest standards of ethics while following legal provisions in spirit, as well as to the letter of the law. We have the ability to capture large amounts of data today. Most of this data comes from customers' devices and equipment, and the vast majority of internal data is also usually customer data. The bigger problem that comes with storing vast amounts of data, from whatever the source, is ensuring its security. As the holder of data, an organization is responsible for not disclosing any user's data to third parties without the user's consent. For good theft-prevention and regulatory practices, data privacy cannot merely be an afterthought after the information has been stolen. Data storage systems, if designed with security in mind, can go a large way in thwarting attempted hackers.

Data governance involves ensuring that the data is being used to further the goals of the organization while remaining compliant with local laws and ethical requirements. On the ethical side, data should not be obtained without the consent of the individuals featured in it. Additionally, the individuals featured, as well as the proprietors of said data, should all be aware what the data will be used for. Data that is collected without the express consent of the user should not be used and should not have been collected in the first place. "Do Not Track" requests by browsers of the users should also be honored. Most modern browsers allow this option to be set, but the implementation is left up to the integrity of individual websites.

Complete data governance should be the goal, but it will likely take some time for the organization's thinking to mature, so the initial focus should be on improving processes and avoiding repeat mistakes. A proactive approach is always better in data management, since abuse of data could have potentially huge impacts. Companies have gone bankrupt in the wake of critical data breaches.

Listed next are some data policy measures that can act as a good starting point:

Data Collection Policies

Users should be made aware of what data is being collected and for how long it will be stored. Dark design patterns that imply consent rather than ask the user explicitly should not be implemented. Data collection should be “opt-in” rather than “opt-out,” or in other words, data collection should not be turned on by default and no data should be collected without specific user approval. If data will be sent to third parties for processing/storage, the user should also be informed of this upfront. A smart data collection policy goes a long way in establishing goodwill and customer satisfaction.

Encryption

Encrypting sensitive information such as credit card information has become a standard industry practice, although user data remains unprotected. The impact of a data breach can be lessened if the data is encrypted. Encryption alone could be the sole factor between a bankrupting event and simply a public relations issue. It is obviously necessary to protect the keys and the passwords used to encrypt the data as well. If the keys are exposed, they should be revoked and passwords should be changed for good measure.

User Password Hashing

In some areas such as user passwords, a technique called *hashing* can be applied instead of encryption. Hashing is a process that takes a user's password and turns it into a unique text string, which can be generated only from the original password. This process works in only one direction, meaning that there is no way to retrieve the original password from the password hash.

For example, the password `password123` (which is a terrible password) could be converted using a hash function into the string `a8b3423a93e0d248c849d`. This hashed password is then stored in the database instead of the real password. The next time a user wants to log into the system, the provided password is hashed and then checked to see if it matches the stored hashed password. In this way, hackers would be able to steal only password hashes that are worthless and not the original passwords. This extra layer of protection especially aids users who use the same password for multiple sites (also not a recommended practice), since once a hacker has a password, they can attempt to use that same email and password combination on other popular Internet sites, hoping to get lucky.

Access Control Systems

All data should be classified based on an assessment of factors such as its importance to the user and the company and whether it contains personal data of users or company secrets. For example, the data could be classified as “public,” “internal,” “restricted,” or “top secret.” Based on the classification assigned to the data, appropriate security measures should be established and followed. Access to data must be controlled, and only approved users should be granted access.

Anonymizing the Data

If the data needs to be sent to a third parties or even other less secure internal groups, all potentially identifying information, such as names, addresses, telephone numbers, and IP addresses, should be scrubbed from the data. If a unique number is allotted to an individual, it should be randomized and reset as well. No data should be shared with third parties without sufficient consent being obtained from the users who are featured in the data being shared.

Creating a Data Governance Board

As you have seen, data governance is a critical part of any organization’s data strategy. In order to develop the initial data governance policies, a data governance board can be constituted. The board will develop the organization’s data governance policies by looking at

best practices across the globe like General Data Protection Regulation (GDPR), Health Insurance Portability and Accountability Act (HIPAA) provisions (more on these in a moment), and so forth. The board should be formed with people who can drive these big decisions. The necessity may arise for the board to push through difficult decisions that are at odds with the aims of the organization in order to protect the rights of the people whose data is at risk.

Initiating Data Governance

In most cases, it is easier to start with an existing set of data governance rules and then adapt the rules to fit your organization. Your data governance board will aid in making key decisions for which policies may not yet have been established, setting precedents, and then instating newer policies as the organization evolves and grows to handle more data. Such a process will help to ensure that the costs of governing the data do not exceed the benefits derived from it.

HIPAA

The Health Insurance Portability and Accountability Act (HIPAA) dictates the procedures to be followed and the safeguards to be adopted regarding medical data. If you are dealing with medical data, it is critical to be compliant with these laws and regulations from the get-go.

In 2013, the Health Information Technology for Economic and Clinical Health Act (HITECH Act) was also implemented. The HITECH Act makes it mandatory to report breaches that affect 500 or more people to the U.S. Department of Health and Human Services, the media, and the persons affected. Only authorized entities are allowed to access patients' medical data.

With this in mind, an organization should be careful that the data being sourced is not violating any provisions of HIPAA or HITECH and that it is ethically and legally sourced.

GDPR

In 2018, the European Union introduced a new set of privacy policies called the General Data Privacy Regulation (GDPR). These privacy

policies put the user as the data owner, irrespective of whether data is stored. Under GDPR, data collection must be explicit, and any implicit consent—such as “fine print” stating that signing up for an account implies that your data can automatically be collected—is in contravention of GDPR. GDPR also mandates that requests for deletion of user data should be as simple as the form for consent. GDPR mandates that users be made aware of their rights under the policy, as well as how their data is processed, what data is being collected, and how long will it be retained, among other things. GDPR is a step in the right direction for user privacy, aimed to protect the users from data harvesters and unethical data collection.

The responsibility and accountability have been put squarely on the shoulders of the data collectors under GDPR. The data controller is responsible for the nondisclosure of data to unauthorized third parties. The data controller is required to report any breaches of privacy to the supervisory authority; however, notifying users is not a mandatory requirement if the data was disclosed in an encrypted format.

Although these regulations are only legally applicable to users in the European Union, adopting GDPR policies for users across the globe will put your organization at the forefront for compliance and data governance practices.

Are You Being Data Responsible?

Securing your data should be considered a critical mission rather than an afterthought. For an AI-oriented organization, data is the cornerstone of all research activities. Good data will lead to better decisions, and the age-old cliché about computers, “garbage in, garbage out,” still holds true today. Ethically sourced data will add goodwill and keep your organization on the right side of the law. Data governance might seem like a daunting task, but with the help of a solid plan, it can be managed just like everything else.

Are You Data Ready?

Data is critical to any organization, but it is essential when building an AI system. It is important to take stock of all your digital and manual systems to see what data is being generated. Is this data

sufficient for your system's needs? Do you need to start looking at data licensing or starting your own crowdsourcing jobs? Do you have the necessary talent (such as data scientists) to make this happen? Have you established your data governance model? Once you have answered these questions, you are prepared to move to the next step: prototyping.

Pitfalls

Here are some pitfalls you may encounter during data curation and governance.

Pitfall 1: Insufficient Data Licensing

When it comes to data, having sufficient licensing is critical. Using unlicensed data for your use case is the quickest way to derail a system just as it is about to launch. Sometimes, developers will take liberties with data in the name of exploration, stating "I am just seeing if this approach will even work first." As time goes on, the solution is built using this "temporary" data while the sales and marketing teams run with it, without knowing that the underlying data licensing has not been resolved. At this point, hopefully, the data licensing problem surfaces before users are onboarded to the system. In the worst case, you find the issue when the data owners bring legal action against your organization. To prevent this, it is imperative to have a final audit (or even better, periodic audits) to review all the data being used to build the system. This audit should also include validation of third-party code packages, because this is another area where licensing tends to be ignored for the sake of exploration.

Pitfall 2: Not Having Representative Ground Truth

This pitfall relates primarily to the role data plays in training a machine learning system. Specifically selected data will serve as the system's ground truth, which means the knowledge it will use to provide its answers. It is important that your ground truth contains the necessary knowledge to answer these questions. For instance, if

you are building the aforementioned daytime and nighttime classifier but your ground truth does not include any nighttime images, it will be impossible for your model to know what a nighttime image is. In this case, the ground truth is not representative given the target use case, and it should have included training data for every class you wished to identify.

Pitfall 3: Insufficient Data Security

For information to be useful, it has to satisfy three major conditions: confidentiality, integrity, and accessibility. In practice, however, accessibility and integrity overpower confidentiality. In order to ensure legal, ethical, and cost-effective compliance, security should not be an afterthought, especially for your data storage systems. Data stores should be carefully designed from the start of the project. Data leakage can lead to major trust issues among your customers and can prove to be very costly. Companies have gone bankrupt over inefficient security. Customer data should be stored only in an encrypted format. This will ensure that even if the entire database is leaked, the data will be meaningless to the hackers. It should be confirmed that the encryption method that is selected has sufficient key strength and is used as an industry standard, like RSA (Rivest–Shamir–Adleman) or Advanced Encryption Standard (AES). The key size should be sufficiently long to avoid brute-force attempts; as of this writing, anything above 2,048 bits should be sufficient. The keys should not be stored in the same location as the data store. Otherwise, you could have the most advanced encryption in the world and it would still be useless.

Employees also need to be trained in security best practices. Humans are almost always the weakest link in the chain. Spear phishing is the technique of using targeted phishing scams on key persons in the organization. Such techniques can be thwarted only through adequate training of personnel. It is important to include not only employees but also any contract resources that you are using to ensure that they are trained in the best security practices. Training and hardening your organization's managers, engineers, and other resources, just like your software, is the best way to avoid security compromises.

Computer security is a race between hackers and security researchers. In such a scenario, one other critical component to winning is to patch everything as soon as possible. Auditing your infrastructure and servers by professional penetration testers will go a long way in achieving your organization's security goals. These specialists think like hackers and use the same tools that are used by hackers to try to break into your system and give you precise recommendations to improve your security. Although getting security right on the first attempt might not be possible, it is nonetheless necessary to take the first steps and consider security from the beginning of the design phase.

Pitfall 4: Ignoring User Privacy

Dark designs are design choices that trick the user into giving away their privacy. These designs work in such a way that a user might have given consent for their data to be analyzed/stored without the user understanding what they have consented to. Dark design should be avoided on an ethical and, depending on your jurisdiction, legal basis. As the world progresses into an AI era, more data than ever is being collected and stored, and it is in the interest of everyone involved that the users understand the purposes for which their consent is being recorded. A quick way to judge whether your design choices are ethical is to check whether answering “no” to data collection and analysis imposes a penalty on the user beyond the results of analysis.

If third-party vendors are used for data analysis, it becomes imperative to ensure that anonymization of the data has taken place. This is to lessen the likelihood that the third party will misuse the data. With third-party vendors, it becomes necessary to take further measures like row-level security, tokenization, and similar strategies. Conducting software checks to ensure that the terms of a contract are upheld is very important if third parties are going to be allowed to collect data on your behalf. Cambridge Analytica abused its terms of service as Facebook merely relied on the good nature and assumed integrity of Cambridge Analytica's practices. Having software checks that ensured third parties could access data only as defined in their

contracts would have shortened Cambridge Analytica's reach by a huge amount, as it would not have been able to then collect data on the friends of people taking their quizzes.

Respecting users' rights and privacy in spirit is a process. Although it might be costly, it is necessary given the amount of data that is now possible to collect and analyze. When fed into automated decision-making AIs, these large amounts of data have the potential to cause indefinite and undue misery. It is in this interest that it becomes necessary to implement policies that make the user aware of how their data is being collected, how it will be analyzed, and most importantly, with whom it will be shared.

Pitfall 5: Backups

Although most people today understand the importance of backups, what they often fail to do is implement correct backup procedures. At a minimum, a good backup plan should involve the following steps: backing up the data (raw data, analyzed data, etc.), storing the backup safely, and routinely testing backup restorations. This last step is frequently missed and leads to problems when the system actually breaks. Untested backups fail to recover lost data or produce errors and require a lot of time to restore, thus costing the organization time and money to fix the problems. To resolve this, you should routinely restore full backups and ensure that everything still works while operating on the backup systems. A full data-restore operation should be undertaken on preselected days of every year, and all live systems should be loaded with data from the backups. Such a mock drill will identify potential engineering issues and help locate other problems as well, enabling you to develop a coherent and reliable restoration plan should the actual need for one ever arise.

With cloud storage becoming so commonplace, it is essential to remember that the cloud is "just another person's computer" and it can go down, too. Although cloud solutions are typically more stable than a homegrown solution because they are able to rely on the economies of scale and the intelligence of industry experts, they can still have issues. Relying only on cloud backups may make your life easier in the short term, but it is a bad long-term strategy. Cloud

providers could turn off their systems. They could have downtime when you need to do that critical data recovery procedure. It is therefore necessary to implement off-site and on-site physical storage media backups. These physical backups should also be regularly tested and the hardware regularly upgraded to ensure that everything will work smoothly in the case of a disaster.

All data backups should be encrypted as well. This is especially important to prevent a rogue employee from directly copying the physical media or grabbing it to take home. With encrypted backups, you will have peace of mind and your customers will sleep soundly, knowing their data is safe.

Action Checklist

- ___ Determine the possible internal and external datasets available to train your system.
- ___ Have a data scientist perform a data consolidation exercise if data is not currently easily accessed.
- ___ Understand the data protection laws applicable to your organization and implement them.
- ___ Appoint a data governance board to oversee activities relating to data governance in order to ensure your organization stays on the right track.
- ___ Put together a data governance plan for your organization's data activities.
- ___ Create and then release a data privacy policy for how your organization uses the data it accesses.
- ___ Establish some data security protections such as using data encryption, providing employee security training, and building relationships with white-hat security firms.

Notes

1. www.nist.gov/itl/products-and-services/emnist-dataset
2. <https://onesignal.com>
3. www.data.gov

4. www.kaggle.com/datasets
5. <https://github.com/awesomedata/awesome-public-datasets>
6. <https://ycharts.com>
7. www.figure-eight.com
8. www.mturk.com
9. www.microworkers.com
10. <http://hadoop.apache.org>

Prototyping

Once the project has been defined and the data has been acquired and curated, it is time to start creating a prototype of the solution. The prototype will serve as a preliminary model of the solution and enable the project stakeholders to provide early feedback and course corrections as needed. Additionally, building a prototype forces a reality check on the project as a whole, since the prototype will necessarily be a vertical piece of functionality, testing that most elements of the technology stack work together.

Assuming your user stories had an initial prioritization set during the project planning step, your prototype should implement the top few user stories. In this way, you will start to see business value as soon as the initial prototype is complete. Note that the combination of user stories you tackle during this phase should leverage most of the technical components of your planned system. For instance, if using training data from a particular data source will be critical to your system's success, you should ensure that at least one of your user stories selected for the prototype step requires that data.

Is There an Existing Solution?

Before you spend resources going down the path of building an AI solution, the first question you should ask yourself is “Can my problems

be solved by an existing solution?” For instance, if a business wants to have a simple automated chat capability for their customers, an all-in-one solution likely exists that can be purchased outright or licensed on a monthly basis. Some configuration is always involved (e.g., what specific questions are my customers likely to ask?), but this solution could be dramatically cheaper and ready more quickly than building a solution from scratch. There are a few ways to look for existing systems.

The first method to find out if a complete solution is available on the market is to search the Internet for your problem. Chances are, if you have a problem that needs solving, other people will have the same problem. For instance, a search for “automated chatbots products” returns an overwhelming number of chatbot product comparisons and product reviews. Problems that affect a large number of people tend to attract those with an entrepreneurial spirit who will come up with a solution to such a problem. In doing so, they may start building their own businesses and try to grow themselves. Part of this process for them will involve advertising and promoting themselves online, so given the right search terms, it is certainly possible to find online solutions for most common problems.

Another avenue for finding existing all-in-one solutions is by speaking with other businesses that have a similar capability to what you are looking to build. If you find that you are a customer of another business with automated chat technology, reach out to that company and see which path they took to provide that capability. Did they purchase a solution or build their own using open-source technologies? What previous approaches did they try that failed? What other “gotchas” and useful tips did they glean? Assuming your business is not a direct competitor, most people tend to be helpful, especially when it comes to explorations of new technologies. As you travel further along your journey toward AI adoption, you can just as easily become an ally to them and reciprocate with your own lessons learned down the road.

The added benefit of directly reaching out to a fellow business is that you tend to avoid the marketing spin that surrounds the “Google” research approach. Instead, you speak to a real user of the technology and hear their unbiased experiences with the initial setup and continued use of the product. Even insights into what the vendor is

like to work with can be valuable. If a vendor has the greatest technology in the world but is unresponsive to customer support requests, you may ultimately decide that it is not worth using that technology at all or decide to hold off while you look for a suitable alternative with better support options.

The final way to learn if there is an existing solution to your problem is attending industry conferences. Especially if your problem seems like it might be only somewhat common, AI conferences that target business audiences could be a good place to research solutions. Conferences bring together both AI experts as well as business leaders who are staying abreast of what solutions are available. Given the fact that these people attend conferences, they are likely to also be well connected and able to introduce you to people who have faced similar challenges and created workable solutions.

Employing vs. Contracting Talent

If no existing solution is available, you need to ask yourself at the start of the prototyping phase, “Do I have an engineering team who can successfully build this AI solution?” This will include data scientists, developers, machine learning experts, and so on. Chances are, if you are a small to medium-sized business whose core competency is not technology, the answer to this will be “no.” Even for large companies that have been sticking with traditional technologies, this answer can often be “no.” If this is the case, you need to make a decision: “Do I build the team I need, or do I find one that is already assembled and contract with them for the job or jobs I will need done?” If you anticipate a clear end date to the work you need, or if you required the engineering team to build only your one AI system, it may make sense to outsource the job to a preexisting firm. They will be able to move more quickly with their existing experience and help you avoid any beginner pitfalls.

Finding a Firm

Finding a contracting firm that you can trust to deliver your AI system is similar to finding an employee in some ways. For example, you will want to see examples of their previous work that implements

capabilities similar to the ones you want in your AI system. Most firms should be able to provide case study documents for similar projects. Asking to see these systems with a live demonstration is also a good way to continue the conversation and do some due diligence. Drill down and ask if the system they built is still in use today, and if not, ask why. Additionally, ask if they are still performing work for that same client today. It can be a good sign if their clients want to continue working with them on their future projects.

You will also want to ensure that the firm you are considering is able to quantify the impact of the systems they build. Look for metrics in the case studies the firm provides. Nothing speaks like real numbers. For instance, the claim that “Our AI system led to a 30 percent reduction of human-addressed support tickets” is a good indication that their AI system made a difference and they delivered on their stated value.

With a firm, you are paying for their expertise in a domain, such as AI. Therefore you will also want to ensure the firm is seen as an expert in that space by doing more than just their client work. For instance, ask if they are active in the AI community by presenting at conferences, writing articles, or perhaps doing their own AI research. Their participation is a positive signal, and the materials they have created can be reviewed to give you a more detailed look at their expertise.

Finally, you can get firsthand opinions by asking the firm for references from their previous clients. Be sure to ask the references not only about the firm’s technical ability to deliver but also how they are to work with on a personal level. Most projects will last a few months if not longer. The firm you are considering may generate amazing results, but if they are unresponsive and a pain to work with, the personal toll will outweigh the benefits.

The Hybrid Approach

While the information in this section has been presented in a way that covers outsourcing your entire AI solution, this does not have to be an all-or-nothing proposition. Instead, let’s imagine your team has experience with four of the five technologies you plan to use but lacks, say, an engineer with user experience (UX) skills related to natural language interfaces. Although hiring a dedicated person to fulfill this role is an option, it is typical, especially in smaller

companies, to not have sufficient work to keep such a person utilized 100 percent of the time. Instead, this role might best be fulfilled via a contracted resource.

You will likely pay more per hour than you would with this resource on staff, but you will more than make up the difference by paying only for the time you are using. Additionally, this individual will have more expertise than, say, a developer who fills in as a natural language UX designer when needed. This added experience will pay off since they will use their expertise to be more time efficient and produce a higher-quality end product.

In the scenario where you are contracting out a particular skill to one single person, it may make sense to find the individual yourself and contract with them directly instead of using a firm. Yes, it will be more work to find and vet an individual contractor, but individuals will have lower overhead than a natural language UX designer who is part of a firm. Sites such as Upwork¹ or Freelancer.com² can connect you with individual contractors from around the globe. Although these resources will be cheaper, they can be hit or miss, and you might have to go through a few until you find the right person for the job. These are the trade-offs you must consider when staffing your project.

Scrum Overview

To stay flexible during the prototype's development, we recommend using the Scrum framework from Agile to scope and plan your development activities. To assist you with this, the next two sections will focus on clarifying terms under this framework. To read even more about the Scrum framework, you can refer to the Scrum document, maintained by experts Ken Schwaber and Jeff Sutherland.³

Under Scrum, the team is divided into three major roles:

Product Owner

The product owner is the business end of the Scrum team. They sign off on demos and ensure that development is carrying on according to established timelines. It is the job of the product owner to prioritize the user stories in the backlog and plan the focus of each sprint.

Scrum Master

The scrum master is the person responsible for the development team staying on task and not getting distracted. It is their job to ensure that the development team receives the critical inputs necessary to do their jobs perfectly. It also part of the duties of the scrum master to ensure that Scrum and Agile are being followed and are well understood by the various stakeholders involved.

Development Team

The development team is a cross-disciplinary team, typically with no more than 10 members, whose responsibility is to deliver the releases and demos. The team is cross-functional and developers should be able to do the work of another as much as is possible. The development team clears the product backlog and is responsible for ensuring that the solution is delivered as per their established timelines.

The Scrum process also consists of three distinct parts:

Sprint Planning

In Scrum, development is done in periods of between two weeks and a month, depending on the development team and schedule. Each period is called a *sprint*. At the start of each sprint, the team discusses and sets goals for the sprint. Goals should be set in a way that ensures that there will be a new working “piece” of the system available at the end of the sprint. Usually the goals are selected user stories that identify a piece of functionality. Depending on the size and complexity of the user story and team, multiple user stories can be accomplished during a sprint. As development progresses, bug fixes for previously developed code will also be included in sprint planning.

Daily Stand-ups

Daily stand-ups are daily meetings no longer than 15 minutes that allow each development team member to talk about their progress and their plan for the day. This meeting needs to be strictly time-capped. Typically each developer responds to the

same three questions: “What did I do yesterday?” “What do I plan to do today?” and “Do I have any blockers?” The daily stand-up is run by the scrum master, who is also tasked with helping to resolve any blockers identified by the developers that they themselves are not able to resolve.

Sprint Review

At the end of each sprint, there needs to be a sprint review. This review should discuss what user stories were able to be completed and which ones were deferred. The current working demo of the system should be shown to the product owner and other stakeholders to gather feedback. This feedback will be used to adjust course for the next sprint. The review also needs to consider what went wrong and what went right during the sprint's development. We will discuss feedback further in the next section.

It is pertinent to note here that Agile and Scrum are frameworks and meant to be flexible. You do not have to follow the entire process as is. You can pick and choose what is applicable to your project and team. The objective is to keep development focused on implementing user stories that provide real value, not to create an inflexible bureaucratic process.

User Story Prioritization

As development work on the prototype continues, it is possible that user story prioritizations need to be adjusted. It is the product owner's responsibility to ensure that the project roadmap is clear and prioritized. This is essential to get right; otherwise, the demos at the end of the sprint might only include low-value features or, worse, features that do not solve a current problem. The product owner's key responsibility is to ensure that such a prioritization is logical and can realistically be achieved.

Every demo produced at the end of a sprint should be a working piece of the entire system. The solution should be iteratively grown and implemented. A demo that does not allow for full functionality

cannot be tested and will have to be rejected or accepted purely based on its code, which can lead to erroneous decisions and extend the project deadline. As an example, if the customer password change form is developed before the customer database and procedures for hashing and storing passwords is coded, it will lead to an untestable demo.

To prioritize correctly, adopt a combination approach of value points and story points. The value points will be generated by the product owner based on empathy maps and the product owner's understanding of the users' wants and needs. The story points, on the other hand, should be assigned by the development team, assessing the difficulty and/or labor hours necessary for each user story.

In the first step, value analysis of each user story must be performed, with ranks given based on the value derived from them. "Value" here can be defined as the benefit the user will receive from using the demo generated by the user story, weighed against the labor hours needed to code it. The value points are abstract points and can follow any consistent method of scoring, such as one based on the Fibonacci Sequence (1, 2, 3, 5, 8, 13).

Using a modified Delphi technique can help the product owner to correctly estimate the effort required by involving the Scrum team in the process. The Scrum team can assign story points to each user story, and stories for which disagreement arises can be discussed. Differing opinions will be discussed and points reassigned until there is agreement among the team. The process for assigning story points should involve sizing each user story iteratively before awarding any points. Points can be any arbitrary number system, like the Fibonacci sequence used traditionally in Scrum, or T-shirt sizing like S, M, L, and XL, and so forth.

Using the story points and their value scores, the product manager should sort each story. The stories that maximize user value along with matching complexity in story points should be taken up first. Using this approach to project planning will ensure that you are protected against the pitfall of doing the easy, low-value tasks first and leaving the hard tasks to never get completed.

The Development Feedback Loop

As we mentioned before, one of the primary ways to prevent large issues from occurring in a project is to identify and correct errors as soon as possible. In this way, good code will not be written based on an incorrect foundation. Assuming you are using an Agile methodology while developing the prototype, at the end of each sprint you should have new functionality. This functionality can then be verified to ensure that it fulfills, or is on track to fulfill, the identified user stories. If the functionality does not work or, more likely, works but performs a different function than the use case requires, stakeholders need to speak up. Development can then adjust its course immediately, saving hours or days of work down the line. This feedback can be used to adjust any part of the AI pipeline, as can be seen in Figure 5.1.

One of the best ways to turn feedback loops into a standardized process is through the aforementioned sprint reviews. Reviews happen at the end of each sprint and include the developers along with the stakeholders. During the review, a demo is typically conducted showcasing the newly implemented features for that sprint. This opens a dialogue between stakeholders and developers, which helps build alignment and is crucial to the project’s success as a whole. Without a regular dialogue, development could be left working in isolation and first showing their progress to stakeholders after three months have passed. At that point, a stakeholder might

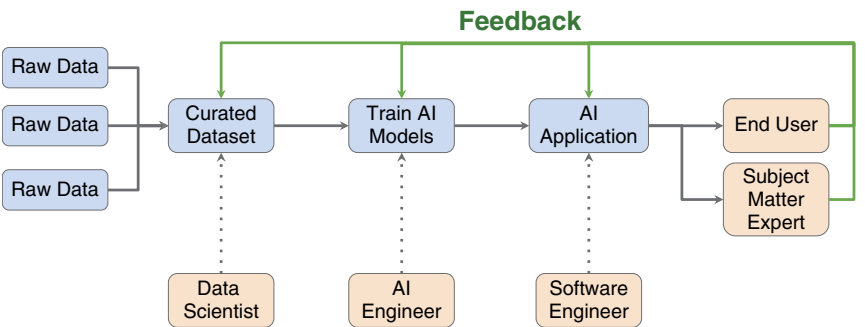


FIGURE 5.1 The Stages and Roles Involved with Feedback

want major changes (causing major delays), either because of an initial misunderstanding or because external requirements have since changed.

Reviews are valuable only if all parties regularly attend. This is typically not a problem for the development team since it is part of their process, but stakeholders can be another story. They tend to be in positions that require their attention to be divided among many competing priorities. Project managers must impress upon each stakeholder the value of not only regularly attending reviews, but also being engaged and vocal with their feedback. They are a stakeholder for a reason, and their success depends in part on the project's success.

During implementation of the prototype, you will also learn and receive feedback that can be codified as lessons learned. For instance, certain capabilities of your selected technologies might not work exactly as advertised. Maybe one of your architectural decisions, when implemented, turned out to be extremely slow and changes had to be made to make it more performant. Documenting this knowledge will be valuable not only for making future decisions on this project, but also when working on future projects.

Designing the Prototype

Assuming there is no existing solution and you are committed to developing your solution in-house, the first step to building a prototype is to define your system's architecture. This begins with the logical components of your system along with the selected technologies that will be providing those capabilities. Note that a single instance of a technology can serve multiple logical roles. For instance, if you need a place to host AI model for Part A of your logical design and another AI host for Part B, you could potentially use a single deployed instance of AI hosting technology such as TensorFlow Serving⁴ to fulfill both.

Logical architectures tend to be defined using a *logical architecture diagram*. The logical architecture diagram shows all the conceptual parts of a system and is helpful to determine which kinds of technology you will need to select. An example of a logical architecture diagram is shown in Figure 5.2.

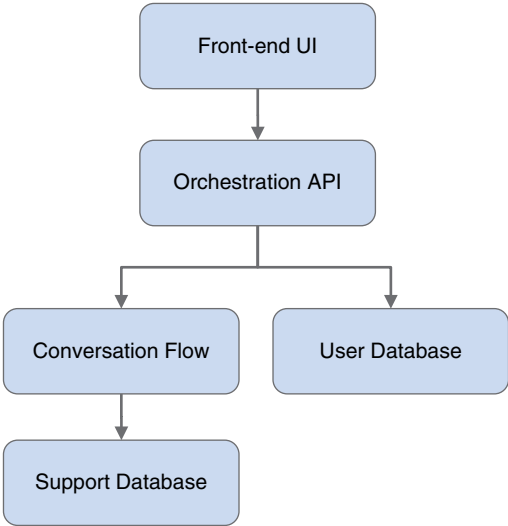


FIGURE 5.2 A Logical Architecture for a Support Chatbot

Technology Selection

With a logical architecture diagram in place, it is now time to start researching which technologies will fulfill the requirements. One approach is to create a spreadsheet of all the capabilities required in the prototype, as noted by your logical diagram. Along the other axis of the spreadsheet, list all the technologies that fulfill one or more of those capabilities. Other factors can be included in the spreadsheet as well, such as price or features, since they may also influence the technology selection. Table 5.1 shows an example of such a table.

After this matrix has been completed and you’ve selected the best technology for the roles, it is time to make a *physical architecture diagram* (see Figure 5.3 for an example). This physical architecture diagram resembles the logical architecture diagram; however, it will include the technologies you are proposing to use. This will continue to help development and stakeholders visualize the solution and pinpoint any issues ahead of time. Fixing a problem now instead of, say, during production will be dramatically cheaper. This concept is called *error cost escalation*, and it is applicable in a number of industries but especially so in software engineering. As we have mentioned before, if development continues to build on top of

TABLE 5.1 Sample tech selection chatbot technologies⁵

Technology name	Role	Price
Dialogflow	Conversation API	Cost per query
Watson Assistant	Conversation API	Cost per query
Microsoft Bot Framework	Conversation API	Cost per query
React	App front-end	Open source
Django	App backend	Open source
ExpressJS	App backend	Open source
MySQL	Relational database	Open source
Oracle	Relational database	Yearly license fee
PostgreSQL	Relational database	Open source

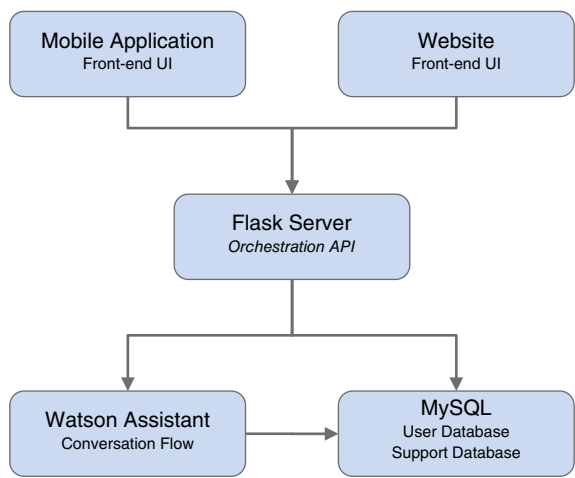


FIGURE 5.3 A Physical Architecture for a Support Chatbot

a mistake, all that work will need to be redone once the mistake is identified. This is also one of the primary benefits of software development and, in a way, the reason we are building this prototype.

One of the other considerations when choosing technologies is “How well do the selected technologies interact with one another?” In some systems, all technologies interact solely with your application code. In this case, evaluating the integration points between components is less important. However, in a scenario such as a database needing to communicate with another system via a message

bus, having a supported integration between those two technologies will make life much easier. In this way, you can avoid having to write and maintain custom integration services.

When making technology selections, it is important to lean on other colleagues in the field. You probably know others in your industry who have experience using the very technologies you have identified in your technology spreadsheet. Some might have had good experiences and recommend a particular technology, but more important are the ones with bad experiences. Although bad technologies might improve with time (especially if the bad experiences were a few years back), you could potentially save yourself from a lot of future pain by getting the details. Lastly, if you select a technology with which your colleagues have experience, you can ask them for help during implementation. If you end up selecting the same technology, you can reciprocate and provide your experiences back to them, which will help both of you.

Another large decision point when building a system is what programming language you will use to build your application. The controversy over which language is best causes many “holy wars” in the industry. From a practical sense, there are two primary determinants for selecting a programming language. The first is the primary language of your development team. Given that they have likely worked on projects together, using the programming language they are already comfortable with will likely have large gains in productivity. Language syntax, built-in library functions, and development tooling will already be familiar to them.

The second determinant when selecting a programming language is whether the language has the software libraries to fulfill your requirements. For instance, most of the current machine learning technologies are primarily available in the Python programming language. Therefore, selecting Python for an AI project may make the most sense. You might find that libraries support multiple programming languages, but keep in mind that the support for alternative programming languages will likely be less. That is in terms of the library’s stability (i.e., more software bugs) and in finding help and documentation in online forums. For instance, TensorFlow, which is a neural network technology open sourced by Google, is primarily used in the Python programming language. Although they have

support for the Java programming language, the TensorFlow website states that “The TensorFlow Java API is not covered by the TensorFlow API stability guarantees.”

Alternatively, it is possible to use multiple programming languages for a single system. This is ideally suited for a microservices architecture, where pieces of the system are implemented as separate units and are integrated through a language-independent method such as a representational state transfer (REST) API. For instance, let’s say your engineering team is strong in JavaScript but the machine learning library that ideally meets your requirements is available only with Python. In this scenario, a machine learning service could be written in Python and called via a REST API from your primary application code, which is written in JavaScript. In this way, the majority of your development team will not need to learn Python just to take advantage of an ideal machine learning library.

When selecting a technology, also consider the requirements for the other user stories you have identified. Although you will not be implementing them during the prototype phase, it is good to avoid using a technology that you know will not support all your other user stories. With this little bit of planning, you can avoid having to replace parts of your technology stack during the production step.

Cloud APIs and Microservices

Traditionally, when developers wanted to add a capability to their application, they would download a code library. Developers would do this for each capability needed. For example, if a programmer needs to manipulate and analyze training data, they could use Python’s pandas library. Libraries like these, however, are programming language specific and typically do not include much (if any) data. This delivery model is fairly limiting, especially for capabilities that are powered by large collections of data.

By contrast, data-powered capabilities are now being delivered through web-based APIs. Instead of downloading a library, a developer makes the appropriate API call and a response is returned. This allows all the requisite data for powering the API to be managed by the API and abstracts the complexity of the implementation completely. Most companies with machine learning technology make

their solutions available through API offerings. Here are a few of the larger companies in this space:

- IBM’s Watson Services⁶ (see Figure 5.4 for their catalog of AI services)
- Google’s Cloud Machine Learning Engine⁷
- Amazon’s Machine Learning on Amazon Web Services (AWS)⁸
- Microsoft’s Machine Learning on Azure⁹

REST APIs, as mentioned earlier, also have the added benefit of being programming language agnostic. If a developer writes a Python application, they can simply include Python’s HTTP library and call a REST API. Similarly, for a NodeJS application, a developer can include the NodeJS HTTP library and make the same REST API call. This enables developers to select the language of their choice without needing to match the language of the library they want to use. For instance, a lot of mid-2000s machine learning programs were implemented in Java, simply because the Java-ML library was the most mature at the time. With the cloud-based API model, libraries are decoupled from programming languages and are easily accessible to all.

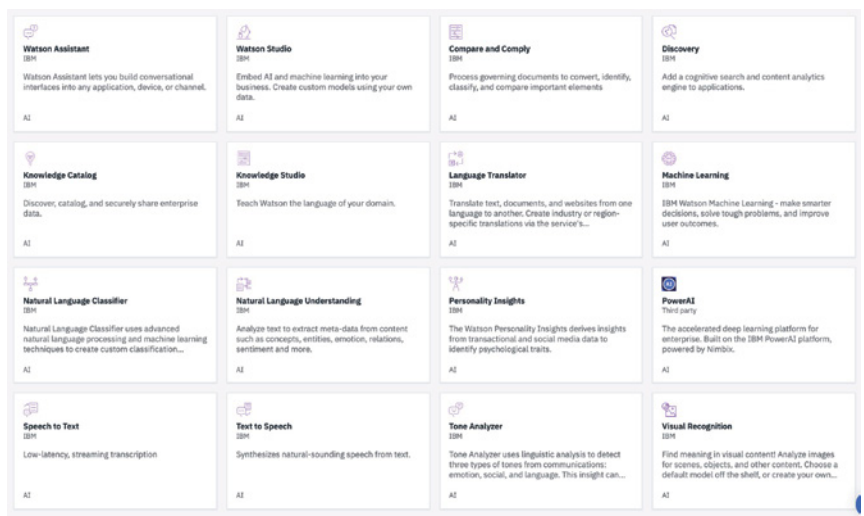


FIGURE 5.4 Sample Catalog of AI Cloud Services from IBM

Internal APIs

Integrating with cloud APIs does not have to be just with third-party vendors. Integrations with your organization's existing back-end systems will also be necessary. For instance, if a user has not received their package in the mail, they might ask your chatbot about its current status. The response to this question is not a simple static answer but rather requires querying a separate shipping system to determine the package's status. The result is then used to craft a natural language response for the user. This is a simple example, but it is possible that some user questions could require responses from multiple systems, including the results of running AI models.

In order for the chatbot to be able to interact with these backend systems, there needs to be an established communication channel. Unlike humans, who typically use a web browser to interact with systems, chatbots require an API to interact with another system. Therefore, it is important to validate that all needed back-end systems already have an API available and that they expose all necessary data. If they do not, you will have to add the creation of the API to your development roadmap for the system that needs to be accessed programmatically. Although this might be straightforward if the back-end system also happens to be created by your team, it can be more difficult if it was created by another team in your organization or a third-party vendor. In this case, you will need to convince them that it is worth the time and resources to add an API to an existing system that has not yet needed to have an API. Explaining that adding an API will enable more integrations with their system, thus making it more valuable, might be enough for them. Additional cost sharing might also make sense, given the funding situations of the different teams.

On a positive note, most systems built today include an API because of the prevalence of automation and open standards. You can typically find out if a system has the required APIs by looking at their documentation. If APIs exist, the documentation will specify the formats and methods for how to call the APIs.

Pitfalls

Here are some pitfalls you may encounter during prototyping.

Pitfall 1: Spending Too Much Time Planning

Although the majority of this chapter dealt with how to break down the prototype requirements and select technologies, it is important not to dwell too much on designing and planning your solution. Given that you will be using an Agile approach and starting a feedback loop as soon as possible, design changes can happen quickly. The start of the project is the point when you necessarily have the least amount of information known. Therefore, it only makes sense to start sooner than later, gaining knowledge by implementing and updating your design as you go. In the end, you will be able to create value more quickly using this approach.

Pitfall 2: Trying to Prototype Too Much

Another frequent pitfall that developers run into during the prototyping phase is setting themselves up for failure by trying to implement too much. A prototype should be limited in scope, provide real value, and be realistically feasible. There will be plenty of time to build large, complex, even moonshot systems once the prototype has been built. However, the prototype is your time to demonstrate value and prove to the stakeholders that AI systems are worth the investment. A prototype that takes too long or that is too ambitious and fails will hurt your organization's chances of ever transforming into an AI-integrated business.

Continuing the chatbot example, it is important to include only a few types of chat interactions during the prototyping phase. For instance, if you are building a chatbot for a movie theater chain, perhaps the prototype version would only handle the ticket purchasing flow. Concepts such as refunds or concessions should be deferred until the production phase. In this way, the prototype can demonstrate the concept and value of purchasing tickets with the understanding that the other interactions can be added later with further investment.

Pitfall 3: The Wrong Tool for the Job

The other common problem is correctly identifying a problem but then assuming it can be solved with the technology *du jour*. During

the technology selection process, you have to ensure that currently popular technologies do not cloud your judgment. Otherwise, at best you will have a needlessly more complex solution. At worst, you will need to replace a core technology midway through development. If your problem requires a hammer, it does not matter how awesome and new that shovel is, it is not the right tool for the job.

With regard to AI, this frequently happens with the misapplication of neural networks. Although it is true that neural networks can solve a large class of problems, it is not the right solution for every problem. For example, naïve Bayes can be a better approach when you do not have a large amount of data. Additionally, if you are in an industry that has to be able to explain its results, neural networks (especially large ones) are notorious for being opaque. They might be accurate given the training data, but because the features it learned are a complex combination of the inputs it is impossible to give a coherent reason why it made the decision it did.

Action Checklist

- ___ Select which of the top user stories are feasible and will be implemented as your prototype.
- ___ Determine if there is an available solution on the market that can be used to save time and resources.
- ___ Decide if you have the necessary talent in your organization or if you need to supplement by contracting resources.
- ___ Design the prototype and use the technology selection process to determine how to build the prototype.
- ___ Use Agile methodologies to iteratively build the prototype with regular stakeholder feedback.

Notes

1. www.upwork.com
2. www.freelancer.com
3. www.scrumguides.org/docs/scrumguide/v2016/2016-Scrum-Guide-US.pdf
4. www.tensorflow.org/tfx/guide/serving

5. Possible technology selections for the chatbot prototype:

- Dialogflow: <https://cloud.google.com/dialogflow/pricing>
- IBM Watson: <https://cloud.ibm.com/catalog/services/watson-assistant>
- Microsoft Azure: <https://azure.microsoft.com/en-gb/pricing/details/bot-service/>

6. <https://cloud.ibm.com/catalog?category=ai>

7. <https://cloud.google.com/ai-platform/>

8. <https://aws.amazon.com/machine-learning/>

9. <https://azure.microsoft.com/en-us/free/machine-learning/>

Production

Now that the prototype is complete and validated, it is time to start building the rest of your AI system. The prototype tackled a few top-priority user stories, but building the production system will be about completing the rest of the user stories you identified during the “Defining the Project” step.

Before your development team starts building the rest of the user stories, however, it is prudent to go through the existing user stories list to ensure that they are all still valid. Over time, your priorities might have shifted or you might have learned more from building the prototype. For instance, perhaps another team already implemented one of the AI models that your system was going to need to use in their system. Instead of implementing the same AI model, you can save development and debugging time by using theirs. Further methods for increasing AI model reuse will be discussed in the next chapter. In this way, user stories may need to be updated or dropped to ensure that users will still receive value for each production user story.

Reusing the Prototype vs. Starting from a Clean Slate

There are differing viewpoints on leveraging prototype code when building a full production system. Some are of the mindset that you should throw away the prototype code completely and start from

scratch, taking only what you have learned. This way, you are not bringing over suboptimal code and initial bad practices. Others say to start with the same code base and continue building—why repeat what you have already done? Both sides have their merits, so we believe the best approach tends to be a hybrid of the two.

The hybrid approach initially looks like starting from scratch. Start with a new design using what you have learned. Perhaps you have discovered there is a more efficient architecture with only a few key changes to make. After the design is in place, create a new code repository for the production code, and then start implementing. This is where the hybridization comes in. Instead of writing all the code from scratch, copy chunks of code from the existing prototype. A lot of code will be directly reusable. The process of copying allows the developers to evaluate the code and look for improvements. For instance, code that connects to APIs and services will typically not need to be changed much, if at all, since documentation usually prescribes this code. This way, you can keep your code organized in the new layout without having to start from square one. After all, this production system will be around for years and good code organization will set the precedent for keeping code orderly and easy to maintain.

For the AI pieces of the project, this process is very similar. The data used in the prototype will almost always be the same data used in the production build. If anything, additional data will be included to improve the representational quality of the ground truth. In some circumstances, the requirements of the model will have changed and constructing a new dataset will be required based on the results of the prototype. If needed, the change should be embraced and resolved at this point so as not to propagate this issue into the production system. If ignored, it will only cause a larger headache with your AI models further in the project, likely close to the time you would like to launch.

Code used to build the AI models will probably not change much between the prototype or production unless the topology of the deep learning technology needs to be modified. What you will likely change in the production phase (and as the project continues) is the hyper-parameters of your model. For instance, tweaking the learning rate during your model's training could improve its overall production accuracy as other components of your data or model change.

Continuous Integration

Once the solution is deployed and users start to depend on its availability, it is important to set up a rigorous process for updating the production instance. This update process should also include automated tests to validate every code change. This is necessary, since a seemingly innocuous change can have detrimental effects, breaking the solution for some users. For instance, imagine a scenario where two developers are calculating the diameter of a circle using its radius. It is possible for one developer to put the line `diameter = 2 * radius` in a function called `getCircleSize()` while another developer concurrently adds the same line of code in a different code location, further up the call chain. Since the code was changed in different files, there is no code conflict between the developers, and the source control (e.g., git) will not complain about any issues. However, the result is a circle size value that is twice as large as it should be. If this code had automated tests written for it, the error would be caught immediately.

The Continuous Integration Pipeline

Another great practice for maintaining stability is to promote code through a series of quality assurance environments. Together, these environments make a *continuous integration pipeline*. A pipeline can consist of any number of environments, but most include the following three:

- The *development environment*, which enables internal developers and product managers to access the latest version of the code
- The *stage environment*, also sometimes called the *test environment*, which is configured similarly to the production environment and runs new code and models that will eventually become production if they pass all quality assurance checks
- The *production environment*, which is the live environment that end users are actively using

The first environment in this continuous integration pipeline is the development environment. It serves as the latest and greatest

version of the system. The development environment can be redeployed after every successful code/model integration, or once a day (typically in the evening). In the latter case, the development environment is said to be running the *nightly build*. Developers and product managers use the development environment to check that closed tickets have been implemented correctly and, separately, that no feature misunderstandings have occurred. This distinction is so important in software engineering that there are defined terms for each. *Verification* is defined as ensuring that the code performs a function without causing errors, and *validation* is defined as ensuring that each function is useful and correctly fulfills the software's intended requirements. Although automated tests can perform verification, it is rare that they are also set up to perform validation since the same developer typically writes the tests for their code. In most cases, other developers and product managers perform validation manually.

Since the development environment is primarily used to ensure feature correctness, it tends to be a smaller version of the fully deployed production environment. This enables the development environment to be updated and redeployed more quickly, but it also means that representative performance tests and other scalability tests cannot be performed until the next stage of the pipeline.

When a sufficient number of features and bug fixes have been pushed to development, it is time to *promote* the development environment to the stage environment (see Figure 6.1). Promoting code to the next level of the pipeline involves deploying the same version of code and models as in the previous environment. Since the stage environment is the same size as the full production environment, it is a good place to test performance and scalability. New code additions, especially if they are the first implementation of a feature, may be fine when run in isolation but cause issues when run hundreds of times concurrently. For instance, new code might call an external system that is not set up to handle scale itself. In this case, reducing calls to the external system by caching results may be necessary to ensure that your users do not experience performance issues from excess load.

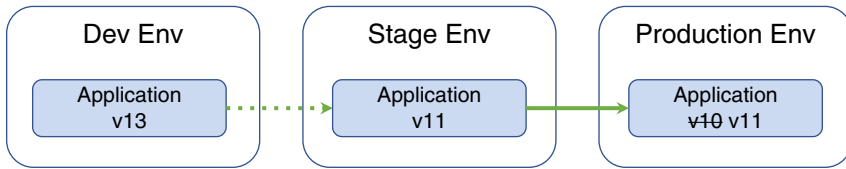


FIGURE 6.1 Promoting Application Code from Stage to Production

Because the stage environment is always the last stage in the continuous integration pipeline before production, quality assurance becomes critical. The stage environment is the last line of defense before bugs are introduced to the production environment, which will affect real users. The stage environment's acceptance process should include both code and models.

Once the stage environment has been thoroughly tested, it is time to promote its code and models to production. In this way, features that were implemented and originally deployed into the development environment have now made their way to providing value to real users. The improved models that are now deployed will provide more accurate results or have increased capabilities. For instance, an object detection model may have been improved through this process to recognize some new objects that have been frequently requested by its users.

For machine learning models in particular, this continuous integration pipeline can promote new models without their even requiring additional programming (see Figure 6.2). Simply including additional data can be sufficient for a model to be retrained and enhanced. This additional data can come from an external source or, more interestingly, from the production users of the system. In the latter case, this feedback loop is perpetually self-improving, an incredibly powerful concept.

True Continuous Integration

Up until now, we have primarily discussed feedback loops in the context of Agile and as a way to ensure that the system is meeting stakeholder requirements. Now, we see that feedback loops can also

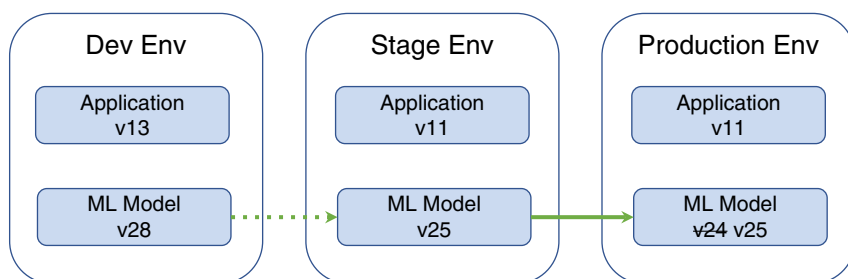


FIGURE 6.2 Promoting a Model from Stage to Production

be applicable, and very useful, in building self-improving machine learning systems. This simple yet revolutionary concept of using the results from a previous activity to partially determine the next activity is a fundamental principle that drives most human learning and behavior. Once you are aware of it, you will start to see this pattern in most intelligent processes. Even a human adjusting how hard to press on a car's accelerator while looking at the speedometer comes from a type of feedback loop, with the desired speed being the goal. We will dive deeper into this concept in the next chapter.

In a world where organizations are global, systems have to stay live 24 hours a day. Any downtime can hurt users' immediate productivity and start to erode their trust in the brand. Therefore, upgrades to production must forgo incurring any downtime whenever possible. In large systems, multiple instances of code may be deployed for intentional redundancy. It is common to replace each redundant instance with the new deployment, one at a time, upon updating. Similarly, models might be deployed multiple times to handle scalability, and they too can be replaced one at a time to ensure that there is no modal downtime.

An alternative upgrade option involves swapping the stage environment completely with the current product environment with a simple networking change. In this scenario, you would have two production environments; call them *A* and *B*. Let's say users are currently using production environment *A*. You would then use production environment *B* as a staging environment. Once environment *B* has passed quality assurance, the network is changed so that `my-system.my-company.com` points to environment *B* instead of environment *A*. There is some complexity with this approach (e.g.,

handling currently-in-flight transactions), but this process will ensure that the system experiences no downtime.

The benefit of continuous integration is that code pushes are immediately tested and validated so that stage could, if stakeholders trust the process enough, be promoted to production as soon as all the tests are complete. Gone are the days where bug fixes have to wait weeks for the next update to be released. This approach works well for a centralized AI system such as one implemented using a software as a service (SaaS)-like architecture. If your system is physically deployed in different locations, this aspect of continuous integration is less practical. There are a number of continuous integration tools, such as Travis CI and Jenkins, that provide detailed usage instructions on their websites.

Continuous integration principles are applicable not only to software, but also to any machine learning models that are being built. For instance, updating an object detection model to recognize one additional object may break its ability to identify a previously recognized object. For scenarios such as this, it is important to have a test suite of images to which precision and recall metrics (discussed in more detail in the next chapter) can be calculated for each object type each time the model is updated or re-created. This way, problems with new models can be identified before they make it through the pipeline and are integrated with the production system.

Ideally, this continuous integration system would have been established during the prototype phase to ensure quality, but as is typical with most prototypes, testing functionality is not the priority. The logic behind this is that if the prototype does not pay off, no further investment has been made in building test frameworks or other supporting infrastructure. Alternatively, if the prototype is successful and quality assurance was postponed until after the prototype was successful, it can be a large undertaking to set up the infrastructure at this point. Unit tests and other automated testing would need to be created for the existing code. Additionally, any incurred technical debt (coding shortcuts, which can also introduce bugs into the system) will need to be resolved at this point in time. If you decide to start your production code from a clean slate or use the aforementioned hybrid approach, most of these issues can be mitigated while either writing fresh code or reviewing it as you copy over pieces of the prototype.

Automated Testing

As mentioned earlier, code must be validated each time it is pushed in order for us to have trust in our continuous integration. This means running a test suite each and every time to ensure code that could negatively affect the end users is never pushed through to production. Ideally, every line of code would be tested in some capacity. For example, the following code has two branching paths:

```
if amount > 100:
    // branch A
else:
    // branch B
```

In this sample code, a good test suite would contain tests that validate both branch A and branch B. This means having tests with the value of `amount` being greater than 100 and also less than or equal to 100. Only in this way can this code be considered to have *100 percent test coverage*.

Test Types

Automated tests are typically broken into three types:

- *Unit tests*, which validate self-contained pieces of code.
- *Integration tests*, which validate code that controls the interactions between systems—for instance, code that talks to a database or a third-party API.
- *Acceptance tests*, which validate the system as a whole, testing functionality in ways that an end user would use the system. These can be direct methods like user interface inputs or programmatic API calls if your solution is an API.

Each of these test types helps validate the code base at different levels and are ordered by increasing scope. Ideally, most bugs will be caught with targeted unit tests where bugs are fairly easy to isolate. As scope increases and multiple systems become involved, problems become more complex and harder to identify. Integration tests are responsible for all bugs that make it past unit testing and will catch them if an API you

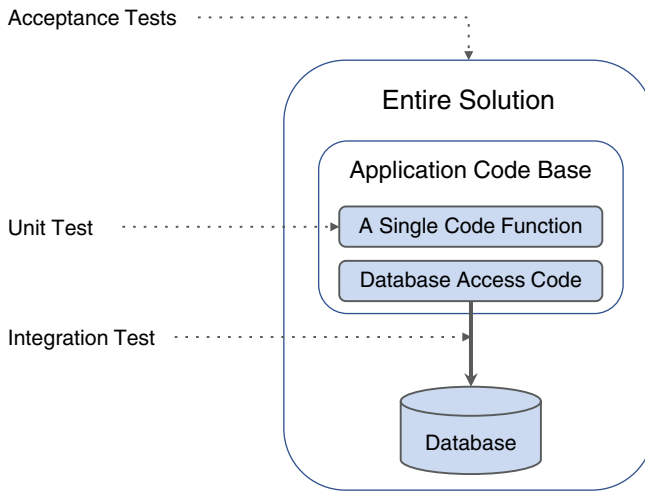


FIGURE 6.3 Acceptance, Integration, and Unit Testing

are using had an interface change (e.g., renamed a parameter). Acceptance tests then become your last line of defense against remaining bugs as they peruse and test large swaths of user features. A good sample acceptance test might be logging in as a user and updating your account information. These sets of tests flow together as depicted in Figure 6.3.

AI Model Testing Example

Given that your AI system will contain AI models that you train, it is important that these models are tested as well. With appropriate tests, you can be certain that new models are as good, if not better, before they replace an older one in production. Let's dive into an example of creating unit tests that evaluate our AI model's ability to determine the correct user intent for a particular user input. Let's use a chatbot in the banking domain for this example. Assume our chatbot contains an AI model that is able to recognize and handle the following three user intents and a "catch-all" intent for if the user's input does not match an intent with enough confidence:

- `HOURS_OPEN`: Provide the user with the hours that the bank is open.
- `OPEN_BANK_ACCOUNT`: Provide the user with instructions for opening a bank account.

- `CLOSE_BANK_ACCOUNT`: Provide the user with instructions for closing a bank account.
- `CATCH_ALL`: A generic response pointing the user to the existing FAQs webpage or that provides the ability to fall back to a human operator.

Since we want to ensure that our system correctly classifies inputs into these intents, we should have unit tests for the AI model that prove it is correctly classifying intents. The tests could be as simple as the following code:

```
# Test HOURS_OPEN
assert classifyUserIntent("What time do you open?") ==
HOURS_OPEN
assert classifyUserIntent("Are you open tomorrow at
9am?") == HOURS_OPEN

# Test OPEN_BANK_ACCOUNT
assert classifyUserIntent("I want to open a new
account with you guys?") == OPEN_BANK_ACCOUNT
assert classifyUserIntent("I am interested in a
checking account?") == OPEN_BANK_ACCOUNT

# Test CLOSE_BANK_ACCOUNT
assert classifyUserIntent("I want to close my
account?") == CLOSE_BANK_ACCOUNT
assert classifyUserIntent("I am moving to a new bank,
how do I do that?") == CLOSE_BANK_ACCOUNT

# Test CATCH_ALL
assert classifyUserIntent("How do I add my spouse to
my account?") == CATCH_ALL
assert classifyUserIntent("Does your bank provide IRA
accounts?") == CATCH_ALL
```

In this example there are only two unit tests per intent, which may not seem like enough but they will provide a sanity check that your system is still generally working as expected. Remember that these unit tests will be run after any model is created and before a new model is promoted into production. This should increase the

confidence that problems will not sneak into production and negatively affect real users.

When working with AI models specifically, there are methods of splitting your model's ground truth into a training set (data used to create the model) and a test set (data used to evaluate the accuracy of the model). This process should be used to generate and select a model, which will then be further checked by the unit tests as defined earlier.

As you increase the number of recognized intents and therefore your chatbot's capabilities, you will also want to make sure that you add their respective unit tests. Classifiers that have to distinguish between a large number of classes (e.g., intents) have to learn more nuanced complexity, causing the overall model to suffer. This shows just how important tests are to feeling confident that your system has not regressed in any way.

What if You Find a Bug?

What happens, however, when a defect is found in the production code? Well, this means that there was no appropriate test case to catch the bug. Therefore, the first step is to write a new test that catches this issue. In doing so, other bugs in the code might also become apparent, and tests for those defects should be written as well. With the test written, we need to validate that the test does indeed catch the offending issue(s). Once we are sure it does, we can then fix the code and validate that the issue is resolved by running the new test cases again. Not only has this particular issue been fixed, but if that same error is ever reintroduced into the code (e.g., code rollbacks), the new test coverage will immediately identify the issue and bring it to your attention before it is rolled out to production with real end users.

Infrastructure Testing

The automated tests we have been discussing are important for ensuring code and model quality. However, a system is only as stable as its deployment configuration and the hardware it is deployed on. If you deploy your solution on a single server and that server goes

down, your users will be affected. If you deploy your solution on three servers in a redundant fashion and one server goes down, your users might not even notice a difference. Therefore, it is important to ensure a robust, multiserver deployment.

But how do you know that your deployment is robust enough to handle issues and recover gracefully? This is the problem Netflix faced in 2011. The solution they came up with is the result of some out-of-the-box thinking. They decided to intentionally cause random errors in their deployment and ensure that users were not affected. Since they were causing the errors, they could immediately stop the issue if the error impacted users. Using this technique, they could identify errors in the deployment configuration, which could then be updated to fill any gaps. That way, the next time any of the errors occurred for real-world reasons, their deployment would stay resilient. Netflix named their successful project Chaos Monkey, after an imagined monkey running around their data center causing havoc. Netflix has since open-sourced the Chaos Monkey code and made it available on GitHub.¹

Because the Chaos Monkey approach operates at the deployment level, there is no need to build a separate Chaos Monkey for your machine learning models. Since the models will be deployed on infrastructure for execution, Chaos Monkeys can easily be pointed to test that part of your deployment as well. In this way, code and models alike will be tested for robustness via the Chaos Monkey approach.

Ensuring a Robust AI System

When you're building a completely autonomous AI system, there will always be times when the system will simply not know how to handle the user's request—whether that's because the user asked their question in an unexpected way or because the user asked an inappropriate question you have no intention of supporting. In these scenarios, the AI system must be able to handle the user's request gracefully.

To demonstrate, let's continue our example of an automated chat support system for a bank. The bank's original design defines a system that can handle any organization-appropriate inquiry with an accurate response. This would include queries such as checking an available

balance, making a deposit, and searching for nearby locations—actions associated with customer banking activities. However, a question such as purchasing movie tickets would be out of left field for the chatbot. Indeed, unless that bank is running a special promotion for movie tickets, an off-topic activity like this is probably not something the developers accounted for when designing their chatbot. One of the best practices that came out of the early efforts in this field is to ensure that chatbots have a mechanism to gracefully handle off-topic requests. It is important to have “guide rails” and instructive responses detailing what a user can achieve with your chatbot and letting them know when they have strayed from the purview of your chatbot’s function. One way you might arrive at this goal is by designing your chatbot to respond to any off-topic queries with a response along the lines of

I am sorry. I am not quite sure what you mean. Would you like to perform one of the following activities?

Conversely, still using our example, when a user is attempting to perform a legitimate activity, such as checking a bank balance, they should never see a message stating that they have gone off topic. Such a message would cause warranted frustration and immediately erode confidence in the system. Your chatbot serves as your organization’s representative, so any lack of performance will directly reflect on the reputation of your organization. With this understanding in mind, we must also recognize that any given project will always have a finite amount of time and budget before the developers must release the first version of their chat support system. The important point is that the system should continue to be improved over time as additional users start to use the system.

Human Intervention in AI Systems

In reality, no matter how robust the chat automation system is designed to be, there will always be issues and failure points. To prepare for these contingencies, your system should be scoped with a “failover” protocol in mind. This can mean using the best practices of guide rails as we just discussed or, alternatively, a hybrid approach

leveraging humans for the cases where the chatbot falls short. The inclusion of humans into our system gives new meaning to acronym of “AI”: augmented intelligence.

However, deciding when to employ a human is not always a simple if-this, then-that type of decision; indeed, the decision tree process is an art unto itself. A simple method of deciding might involve monitoring for users who repeatedly receive your default “I am sorry” response. At this point, the chat system could allow a chat specialist to seamlessly take control of the chat and deliver their own responses. This fallback capability will cover the variety of unforeseen edge cases that are important to users.

Another method to determine when a human should be brought into the chat could leverage a different AI capability: *tone analysis*. Tone analysis technology analyzes not what the user is saying, but rather how the user is saying it. For instance, there is a difference between

I have not received my new account information in the mail yet, would you please be able to take a look at its progress?

and

I have been waiting forever to receive these ridiculous documents for my business and it seems like no one cares about this!

The first and second example both convey the same message but their tone and delivery are drastically different. The second example’s use of hyperbole and vivid language (sometime even explicit language) can be strong indicators that a human’s touch is warranted at this point. Even if the chatbot is able to fulfill the user’s first need by providing an answer to their question, the chatbot might not fulfill the user’s second need, which is recognizing how the user feels about this issue and validating their frustration.

In scenarios such as these, it can be helpful to adopt a hybrid approach of using a chatbot to handle 80 percent of user questions while filling its gaps with a real human. A human resource can be the difference between the failure of a fully automated system versus the

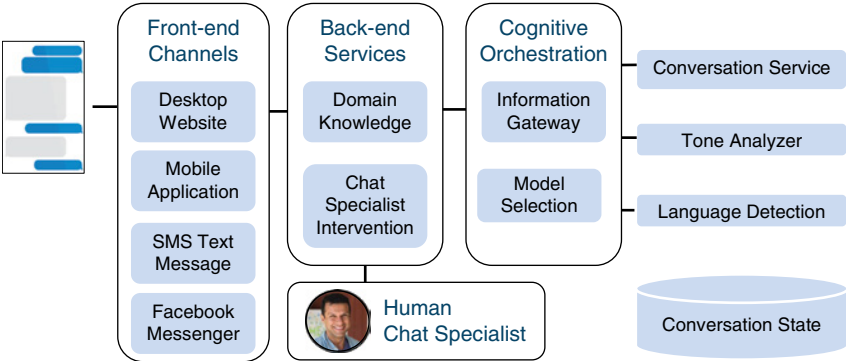


FIGURE 6.4 Sample Chatbot Architecture that Includes a Human in the Loop

success of a human augmented system (see Figure 6.4 for a sample hybrid architecture). Additionally, as time progresses, the chatbot can be improved by looking at the user questions that required human involvement. As the common gaps are filled, the chatbot will begin to rely less and less on human augmentation.

Ensure Prototype Technology Scales

After you start to build the rest of your system, it is important to start as early as possible determining whether the technologies are working—not as much in regard to the functionality they provide, since that is likely not to be any different than what you encountered during the prototype phase, but rather if the technology is scaling suitably. With a small number of users or transactions, this is probably going to be a trivial verification. However, as your number of users or transactions per second grows, technology that has poor scalability becomes a deal-breaker.

Load and performance tests are the primary way of testing whether a system is scalable. A load test, at its core, works by simulating a substantial number of virtual users (for this example, let's say 1,000) using the system concurrently. There is no definitive guide as to how many virtual users to include in a load test, but conservatively testing with 20 percent more users than you expect is a good guideline. That means if you expect 1,000 users to be using your system at any given time, you should be testing with 1,200 users in your load test.

Many open source and proprietary tools are available to perform load testing. Some of the open source options are Apache JMeter and The Grinder. On the proprietary side, you will find offerings such as WebLOAD and LoadRunner. Although some of these load testing frameworks have more features than others, all will be able to simulate concurrent users accessing your system.

While accessing the site, specific load testing for each of your AI models is very important. Even though training a model is an order of magnitude more computationally intense than performing a single evaluation of that finished model, allowing thousands of concurrent model evaluations can bog down any unprepared AI system. With this in mind, you should test the production deployments for each of the AI models you have created. This either means writing a custom load testing driver that calls the model or standing up a thin API that simply passes data to a model for evaluation. In this way, your evaluation can ensure your model's scalability, which is potentially the key value of your entire AI system.

Scalability and the Cloud

Some might argue that modern cloud platforms make load testing irrelevant. They assume that the cloud can automatically scale to any load, as needed. Unfortunately, this is not always the case. At the very least, you will need to perform load tests to validate that the cloud scales as expected. You might also find that the cloud scaling is not instantaneous and there is up to three minutes of inaccessibility as it scales to the correct capacity. This happens especially often with workloads that spike or that have a sudden increase and decrease in usage (see Figure 6.5).

The next validation point is to ensure that your system is implemented in such a way that it takes advantage of the cloud scalability. It may be true that the cloud is scalable, but if user information is stuck in a single database instance, then a scaled application layer, though no longer a bottleneck, will still provide slow responses. In this case, the addition of database mirrors or a server-side cache to maintain session data is required. This is also something that may not become apparent until a load test is performed.

Even for machine learning models, there are dedicated clouds that can spawn multiple model instances, as needed, to limit latency. These clouds are relatively new, such as Google's Cloud Machine

Transactions per Second vs. Time

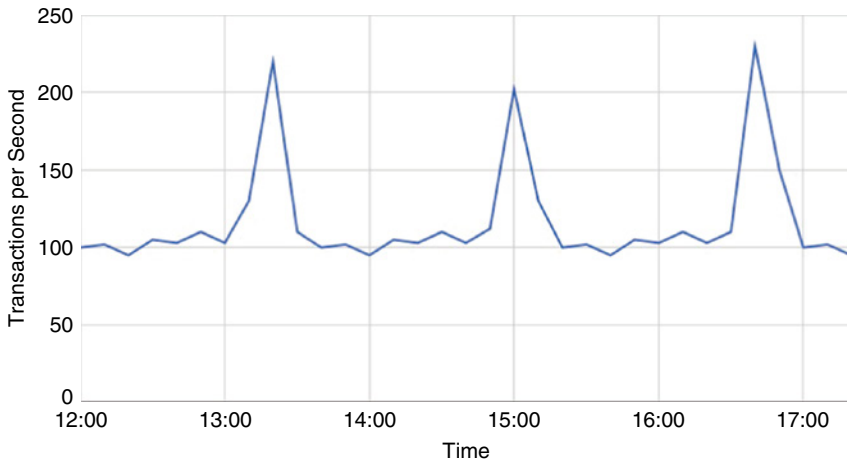


FIGURE 6.5 Example of a Workload that Exhibits Spikes

Learning Engine. For scalability, it is important to determine which parts of your system will likely be a bottleneck using load testing and ensure that those components are designed and deployed accordingly. We will discuss cloud deployments further in the next section.

Cloud Deployment Paradigms

The cloud has morphed over the years, offering a number of deployment patterns. Originally, data centers offered “bare metal,” physical machines that were wholly dedicated to a single user. With the popularization of virtualization during the dot-com boom, physical machines were being dynamically carved into smaller, isolated virtual machines (VMs) as requested by cloud customers. This started the design paradigm of only paying for what you used or “pay as you go.” Customers were now able to request the specific amount of storage, RAM, and disk resources they needed, without needing to buy any of the traditional underlying hardware. They could even turn off their virtual machines and incur costs only for the direct storage of their data. This opened the door to drastically reducing hosted computing costs, and this type of on-demand VM deployment gave rise to what is called infrastructure-as-a-service (IaaS).

As time progressed, cloud providers started to innovate further. Since each virtual machine contains an entire copy of an *operating system* (OS), such as Windows or Linux, a chunk of the VM resources were being consumed just to run the OS. The smaller the VM, the larger the percentage being used for the OS. This was a problem given the increasing trend toward small, dedicated microservices. Instead of using traditional virtualization technology, cloud providers started to turn to another technology called “containers.” Containers are advantageous because they do not have their own OS, but they still have isolated computing resources, which provides security from other containers running on the same computer. Avoiding multiple copies of an OS running on a server may not sound like a huge savings win, but being able to apply the savings across all machines in a data center makes the whole operation more affordable.

One of the more popular container technologies is Docker. It is open source, which has made Docker accessible and somewhat of a de facto standard in the container space. An associated technology named Kubernetes enables the deployment of multiple interconnected containers that can be managed as a single unit. This is powerful because you do not need to manually set up every component in your application. Instead you simply define a pod file which specifies all of the individual containers and their parameters. Then whenever you want to deploy this group of resources you simply provide the pod file to Kubernetes. Pod files can be stored in a version control system and managed similar to code. This is an example of the “infrastructure as code” paradigm where you define your deployment in text files and leverage a framework, such as Kubernetes, to set up the actual computing components. This leads to repeatable infrastructure deployments and the ability to migrate among cloud providers easily if need be.

With all of this great cloud technology, it was only a matter of time before special-purpose AI clouds became available. Traditionally, machine learning models had to be deployed by creating a virtual machine and then installing the model libraries, such as TensorFlow. The situation improved when data centers started providing machines with graphical processing units (GPUs), which greatly improve AI model performance. Today, engineers can simply upload their machine learning model to AI clouds and pay only for the time their

model is executing. Google provides such a service, but it's only one of many. This AI deployment paradigm is powerful because it reduces the knowledge required to host AI models. Instead of worrying about model deployment best practices, engineers can focus their time on building the best possible models.

Cloud API's SLA

Assuming your solution will be deployed on the cloud, you will want to understand the available *service level agreements* (SLAs). An SLA is an agreement between you and the cloud provider for the quality of service you can expect. They will typically specify metrics such as availability, which is how many minutes per month your system can be down. It is important to note that this is not the average expected downtime per month, but rather the worse-case scenario a cloud provider is promising.

Better SLAs will be more expensive, so you will need to determine how much unavailability your application can tolerate. If a cloud provider does not abide by the SLA, consumers typically receive some compensation, such as service credits that can be used for future service. Large outages are uncommon for most modern cloud providers, typically making news since they affect many popular consumer websites.²

Continuing the Feedback Loop

As we discussed in the previous chapter, starting a feedback loop is critical to ensuring that issues are caught early and do not propagate, causing larger issues later. This continues to be true as you build your production system. Regular meetings with stakeholders and end users will continue to be one of the best feedback-generating methods at your disposal. Again, these can be done through the Agile mechanism of sprint reviews.

Pitfalls

Here are some pitfalls you may encounter during production.

Pitfall 1: End Users Resist Adopting the Technology

This pitfall is common with all new technology but especially with AI solutions. Automation technology can be unsettling for end users, since it replaces some of the work they are used to doing themselves. Opinions range from “This technology will just be a hindrance to how I work,” to “It’s only a matter of time until my skills are obsolete, the robots take over, and I’m out of a job.” Change is hard, no matter what form it takes.

Another issue with AI solutions in particular is that most AI systems require input from a subject matter expert (SME) to create the ground truth used to train the underlying machine learning models. These SMEs are also typically the ones directly affected by the integration of a new AI solution. For many reasons, it is important that the AI solution be an augment to the SMEs’ knowledge and capabilities, instead of a direct replacement of their role. Remember that a machine learning model is only as good as the ground truth used to train it.

To avoid this pitfall, early end user engagement is critical. End users need to be part of the planning process to ensure that they fully understand the solution and feel they have contributed to the end product. This might even mean inviting a few influential end users during the Ideation/Use Case phases to build excitement and a voice for your user base. While early input is in no way a guarantee (end users might think they want one thing at first, only to realize they need something else once they start using the solution), it will help mitigate the fear associated with adopting new technology.

Pitfall 2: Micromanaging the Development Team

Under Agile, the development team is given full responsibility for the successful technical implementation of the project. The team works on the combined values of transparency and mutual trust. In such an environment, it would not be prudent to control every aspect of the development team. Nor would it be a good practice to set the targets for each sprint for the developers. This will lead to a lack of motivation, weakening Agile. The development team should be able to focus on the project by themselves with minimal intervention from

the product owner. Some teams pick the Scrum manager from among the dev team as well. This is to ensure that the benefits of Agile are preserved.

Pitfall 3: Not Having the Correct Skills Available

Since building a machine learning system requires a number of specialized skills, it is critical to have these skills available and ready to go before your project starts. Whether this means hiring full-time employees or establishing relationships with contracting firms, it is worth the up-front effort to avoid delays. Skills we have mentioned thus far that will be required include AI, data science, software engineering, and DevOps. The hiring issue is twofold in that you must find the individuals with the proper skillsets for your project and, of course, have the appropriate budget in place to fund them. With these items addressed, there should not be any skill roadblocks in the way of getting your system deployed.

Action Checklist

- ___ Reevaluate user stories to ensure that they are still relevant.
- ___ Establish a continuous integration pipeline with automated tests to ensure system quality.
- ___ Allow the system to involve human intervention as necessary.
- ___ Perform load testing on your system to ensure that it and its components are scalable.
- ___ If your system is deployed in the cloud, review the SLAs and make sure they are sufficient for your user stories.
- ___ Release the live production system to users and begin the feedback lifecycle process.

Notes

1. <https://github.com/Netflix/chaosmonkey>
2. For an example of an SLA, see the Amazon EC2 SLA at <https://aws.amazon.com/compute/sla/>.

Thriving with an AI Lifecycle

Once the system has been developed and deployed, the work is still not complete. As with all other software, an AI project requires regular upkeep and maintenance. Even if you choose never to implement another new feature, there will always be bug fixes, server updates, and other forms of maintenance to keep the system going. Although this commitment is typically not as large as the system's initial implementation, an ongoing investment is still required to “keep the lights on.”

When building an initial project estimate, you must also ensure you have enough resources secured to maintain the system once it goes live. It is typical to see a spike of issues shortly after launch, when end users are first accessing the system. This is so common in fact that there is a name for support when a system first goes live: *hypercare*. It would be a mistake to assume that the system developed will be 100 percent correct and a guaranteed success right out of the gate. Although a good 80 to 90 percent of the project will likely be correct, users will get hung up on the balance. As time progresses, maintenance resource requirements should stabilize. This stable point is what you should use as the estimate to secure the next year's maintenance budget, augmenting it accordingly if large updates or migrations are planned. A few critical activities must be performed once a system is live in order to ensure that the project is healthy and that the benefits from the project will still be accrued

while mitigating any drawbacks. The following steps will help your organization to learn and grow, even in cases where the specific project might turn out to be a total loss. It is naïve to think that all mistakes are unavoidable. The best we can do is learn from them and try not to make the same ones twice.

Incorporate User Feedback

Independent of the decision to continue active development, user feedback remains key to maintaining a usable system. Although a representative sample of users should have been providing feedback throughout the design and implementation phases, it is at this point—when your complete target audience of users start using the system—that a new group of issues will come to light. These issues might not be outright bugs in the code, but rather incongruities in the assumed and anticipated needs of the users with what it turns out they actually need. Having some sort of formal way for users to communicate these incongruities is critical.

The simplest method for receiving user feedback is to include a button in your system labeled Feedback. Clicking this button could take users to a short form where they can submit their feedback. This function alone is not ideal—giving the impression that feedback is being sent into the void—but it is a good thing to have in place before your system launches. If you don't include a Feedback button, when your users have valuable feedback to communicate to you they will have no clear avenue to share it. This leads to frustration, a loss of potentially valuable insights, and a likely decrease in user engagement with your system.

With a little planning, some organizations set up forums as a way of collecting user feedback. The beauty of these forums is that they are open. This enables users to potentially answer other users' questions, which may slightly cut your support costs, but more importantly, forums significantly increase user engagement with your product. They enable users to build on one another's ideas. For instance, if your AI system makes recommendations but does not currently allow users to bookmark those recommendations for later use, users might come to the consensus that this is a key feature that should be implemented. Only after users are given the means to

publicly articulate their thoughts will others be able to share their support for the idea and will you be able to get a sense of how many users feel the same way. Other ideas that are only valuable to a single user will not pick up traction and have the potential of being met by other users who might have found an easy and suitable workaround.

For internal systems, monthly review meetings can be incorporated for at least the first few months of implementation to ensure that shortcomings can be openly discussed and rectified. These meetings will help users come together and share new ways to use the system as well. These kinds of meetings can also be used for public products, assuming your organization has invested in a community manager role to organize such events. Knowledge sharing can become a major success factor for complicated systems with a large and diverse user base such as the ones enabled by AI technologies.

Other common feedback mechanisms include mailing lists and public bug trackers. When selecting the feedback channel(s), take care to choose a path that users will be the most comfortable with. For instance, a project intended to be used by developers can benefit immensely by using a public bug tracker since they are similar to the systems developers use in their day-to-day jobs. Product owners should prepare a tracking process for feedback received and action taken, adding accountability and review of their work. This process will also help the organization to accurately learn about what went wrong and decide how best to avoid the same mistakes in future projects.

Users know what they need and will communicate this with you given the opportunity. Providing a feedback mechanism, either directly through your system or through a forum, will provide you with valuable information about your system—information that can be used to prioritize new features or identify gaps in the system that you were not originally aware of. As you can imagine, the most likely period for any major user issues to surface is immediately following the production deployment, which is why it is critical to have some kind of feedback mechanism in place before any system goes live.

Not all feedback necessarily involves changing the system as per the needs of the users. Some user issues can be alleviated through training and the release of more documentation. There may also be users who are resistant to change or who are simply satisfied or

complacent with using the old system. In a well-managed project, the number of such users will be small, but they will still need to be heard to ensure that the implementation is not derailed.

Ideally, all feedback received should be reviewed by the product owner before any action based on it is taken. As the system's proprietor, the product owner should be the final authority on making changes to the system. They must separate the necessary from the unnecessary and take action only on feedback that is constructive. Even constructive feedback might not be immediately actionable. In this case, such feedback should be analyzed and incorporated into the idea bank for implementation at a later date.

Timely review and action based on user feedback will help alleviate most problems faced during the implementation of a project. We start with assumptions of success, as always, but if things go south, the silver lining comes from what we stand to learn, expand on, and perfect.

AI Systems Learn

One of the best parts about an artificial intelligence system is that it can “learn” and improve over time. This learning process is automated but not automatic; the difference is that someone has to trigger the automation in order to get the ball rolling. The knowledge base that humans use to make decisions is constantly increasing. This makes it imperative to update the knowledge of an AI system as well. When an AI system comes across new kinds of input that are unfamiliar, it can be crippled by them, unable to properly respond. New diseases are routinely being discovered, new products are launched, and customers generate new questions using data from infinitely amalgamated sources. Thus, the contents of an AI system need to be updated periodically to keep up with changing business circumstances and scenarios.

Although most days the changes are small and incremental, updates must be done consistently to ensure that the machine learning system does not become outdated. It is similar to how we as humans need to be updated every day by reading the latest in current events. The AI systems of today cannot access and predict future trends and advancements without additional assistance.

To understand what knowledge your intelligence system is lacking, the product owner should be looking at logs to determine what actions the system has been taking. This process will show how the users are engaging with the system and how the system in turn reacts to those user inputs. This exercise will help the product owner recognize if there are gaps in the system and, if needed, enable the product owner to create a plan to fill those gaps. This is a continuous process that, if not done, will eventually render the AI system functionally outmoded. Just as it is with the human brain, continual learning helps protect your AI system from the natural decay that is experienced by all systems.

Another way to facilitate this periodic updating is by looking at what user questions trigger the equivalent of your default “I am sorry, I cannot help you with that” response, or the questions that lead your chatbot to transfer a conversation to a human. If you can find some common themes among these user questions, you can begin to update your system with new responses to potentially address these topics. This can be accomplished in two ways.

The first way is if a commonly missed theme is one that is already implemented by your AI system. These are easier to fix, as comprehension of a particular language is likely to be all that is standing in your way. It is impossible to predict the myriad questions that can be posed to your AI system, even for intents that are known in advance. For example, if your system is ready to provide specific answers when a user asks “How do I get a mortgage?,” this is a supported use case. While looking at the real usage data, though, if we see that the default “I am sorry” response is being triggered when users ask the same question in the form “How do I buy a house?,” or “How do I get money for a house?,” this indicates a need to expand your system’s recognition of related terms and phrasing. Fixing this problem is simply a matter of tying those unsupported questions to the already-implemented intent of “getting a mortgage.” When you update your ground truth with the new user data, your AI system learns additional ways that users might ask for the answers that your system already knows.

The second way is if the commonly missed theme you identify is a completely new intent that you did not consider would be important to your users. Though not as simple a fix as the former option,

this information is extremely valuable in growing your system and maintaining its relevance to your users. For instance, let's say you notice that 10 percent of the user questions triggering the default response are asking about interest rates on CDs. Armed with this information, you can enable your chatbot to discuss interest rates, satisfying more users and potentially indicating new avenues for growth within your business. This method will expand the capabilities of your chatbot for the use cases your users are actively interested in. The next time someone asks about CD interest rates, your chatbot will have the knowledge to intelligently address this intent.

New Technology

Given that AI is a particularly fast-moving area of innovation, it follows that the available technology will be changing rapidly as well. In order to stay ahead of your competition, you should periodically evaluate the landscape for new technologies. This evaluation process could be done on a quarterly basis. However, if you are looking to stay abreast with the latest developments, it is important to engage with your local AI community. AI-focused meetups are an excellent way to do this. Not only will they have presentations about relevant topics in the field, you will be able to connect with AI professionals who also attend these meetups. They are essentially smaller, localized versions of conferences, which we also recommend attending. Meetups and conferences are a great way to learn about new technologies, in addition to hearing about case studies from others who are implementing AI in their organizations. When new techniques or technologies are identified that could be applicable to your organization, you will want to flag them for further investigation.

Artificial intelligence systems can improve in two ways. The first is improving the quantity and quality of the data that a system runs on. The previous section discussed how you can increase your quantity of data just by capturing the inputs of your users using the current system. Although this might take your system from 92 to 94 percent accuracy, there will likely be diminishing returns. The second way to improve your system is by adopting new technologies. This can be as simple as using a new model topology in the same AI framework or as complex as adopting a whole new technology and using it to build

your models instead. Regardless of the approach you choose, it is smart to devote some of your ongoing development resources to prototyping the more promising technologies. At this stage, you are looking to see if a new technology has promise and, if so, deciding whether to devote more resources and efforts to exploring implementation.

Another way to focus your technology investigation is to keep a “lessons learned” document during implementation. This document will include tech features that were not good enough or required capabilities that had no solution at the time. These challenges likely forced user stories to be cut and pushed out to a later date. In this way, the lessons learned document helps to highlight gaps that need filling. As you stay abreast of AI technology announcements, you can reconcile them with the gaps you encountered during development to see if something new has entered the realm of the possible.

Quantifying Model Performance

When using AI models, it is important to be able to quantify their performance. Empirically bad models should not be shared in a model library other than as a starting point or as an example of what does not work. There are three different metrics used to specify model quality, but first we need to include some definitions. In the simplest case, we have a model that provides a “yes” or “no” answer. For our example, let’s assume we have a classifier that recognizes if a picture contains an animal. If we provide a picture and it is primarily of an animal, it returns a positive answer, “yes.” This is considered to be a *true positive*. If there is no animal in the picture, the model responds negatively, “no”; this would similarly be considered a *true negative*.

In a perfect world, our model would never be wrong and return only correct answers. Unfortunately for us, this is not always the case. Sometimes, we will provide a picture without an animal in it and the model will incorrectly state that it includes an animal. When our model’s response is an incorrectly affirmative one, we call this a *false positive*. Similarly, if our model is given an image with an animal in it but it incorrectly responds that there is no animal, this would be a *false negative*.

These four terms together (true positive, true negative, false positive, false negative) are all the possible results of a simple AI

		Actual	
		Contains Animal	Doesn't Contain Animal
Predicted	Contains Animal	# of True Positives (TP)	# of False Positives (FP)
	Doesn't Contain Animal	# of False Negatives (FN)	# of True Negatives (TN)

FIGURE 7.1 Sample Confusion Matrix for an Animal Classifier

classifier. These numbers are typically shown using what is called a confusion matrix (see Figure 7.1). Using these metrics, we will be able to quantify our models' performance. Specifically, we will introduce three scores built from these metrics.

Precision

Precision is a word used in common English, typically to refer to how specific something is. In the field of machine learning, however, precision has a very specific definition. It refers to all the times a classifier predicted a true positive divided by the number of all its positive predictions. In our example, this means that when our classifier predicted that a picture contained an animal, it was actually correct. This can be quantified by taking all the true positive cases divided by all the positive cases. In other words, precision is

$$\text{Precision} = \text{True Positive} / (\text{True Positive} + \text{False Positive})$$

Recall

The second metric we will introduce is called *recall*. Recall refers to all the times the picture actually contains an animal, we did state that it contained an animal. More generally, how often does the model predict positive when the correct answer is positive? This is subtly different than precision and the difference is important.

$$\text{Recall} = \text{True Positive} / (\text{True Positive} + \text{False Negative})$$

One group of use cases where it is important to consider both precision and recall is when the positive and negative distributions are highly skewed. Take, for instance, oncology, where a machine learning model looks at images and determines if cancer is present. For most of the processed images, there is no cancer present. If we assume 99 percent of the time that there is no cancer, a classifier that always predicts “not cancer” would have a 99 percent precision and by that metric does great. However, when we take a look at that same model’s recall metric, we would find that its recall would be extremely poor. Therefore, it is wise to consider both when evaluating your model’s performance.

F1 Score

Precision and recall are two sides of the same coin. If you want to make sure you have a really high precision score, you will sacrifice recall. Alternatively, if you want to ensure that you have a really high recall score, you will sacrifice precision. Instead, a third score has been created called the *F1 score*. The F1 score is a combination of precision and recall such that it tries to find a middle ground. The F1 score is specifically defined as

$$F1 = 2 * (\text{Precision} * \text{Recall}) / (\text{Precision} + \text{Recall})$$

The F1 score gives us a straightforward way to compare the performance of multiple models so that we can pick the best one. If you have any machine learning models in use and have not yet quantified them using these metrics, it would serve you well to calculate them and set them as your baseline for future improvements.

Updating and Reviewing the Idea Bank

In Chapter 2, “Ideation,” you saw that an idea bank can be immensely important to an organization. Having a process to generate and implement fresh ideas is the value that an idea bank brings to an organization. Employee turnover and attrition makes organizational memory fickle and transient, and the idea bank serves as a permanent knowledge store in these instances.

The idea bank should be controlled by a committee of high-level decision makers, ensuring that no potential is wasted. A chairperson should be appointed from among the committee members every year, whose job will be to oversee the meetings. The idea bank should be open to suggestions from all corners of an organization to ensure the maximum breadth of perspective among the ideas generated. The submissions can come from internal forums, email submissions, or brainstorming meetings, and ideas need not initially be met with the rigorous standards they will ultimately be judged against. They can be fickle and spontaneous when first proposed. In this regard, to maximize the benefits you should encourage idea bank submissions in all forms. A culture of innovation and improvement can be established if the management focuses on the idea bank.

A quarterly meeting for the review of the idea bank should be undertaken. This is to make sure that any projects that can be implemented are started in a timely manner. The review meetings can begin with a discussion about the current goals of the organization and the direction it intends to take in the future. Once the people are in the correct headspace, discussion can start by picking up the last ideas submitted and commencing deliberation of their viability and potential value to the organization. It is not necessary to come out of these reviews with a new project; the discussions are more important.

Knowledge Base

Most large organizations today have an internal knowledge base set up for explaining their own products via user manuals and guides, but a knowledge base can be even more informative and educationally oriented than this. With modern systems, a knowledge base can become a powerful tool for innovation and fostering creativity within the organization. Expanding the scope and attaching more storage to your existing knowledge base can be leveraged to give your employees the power they need to sift through larger amounts of data and gain expertise in more subjects. A cross-disciplinary knowledge base is necessitated by the fact that the world has become more complex, with solutions often emerging at the intersection of multiple disciplines—artificial intelligence and finance, for example.

A knowledge base should not include any documents that are highly confidential, such as business plans and future projections of the market. The knowledge base should also include information from the outside world. All employees maintain their own personal knowledge bases, whether in the form of books, scholarly articles, or a folder of PDFs and large unwieldy documents titled “My Important Links.” A knowledge base shares this individual knowledge across departmental boundaries and enables the exchange of valuable information, which can lead to new ideas and insights.

The most basic features that an online knowledge base should offer are retrieving data (searching), indexing, and enabling users to collaborate. A modern blog like WordPress is easy to set up and comes already installed with most of the features just listed. It is a quick solution to get you up and going. As the information grows and value is created, further investment can be made in more specialized software with better search capabilities. Although books, articles, and links are the obvious choices to include in a knowledge base, it could also have notes and other materials written by the employees themselves. These notes can provide important insights into the minds of the experts running the business. A simple process of searching for and then adding the information will reduce duplication. Either a digital library or a physical library subscription for the employees can also add a considerable wealth of information on both technical and nontechnical topics. The digital library Safari Books Online, for instance, is a good resource for technical materials.

A sample selection of material that can be included in the knowledge base would broadly fall into the following categories:

- A digital or physical library of books used by managers, developers, and analysts to solve problems. This can also be provided through a subscription service.
- Online tutorials, code samples, and other open source projects that have been used internally for implementing company projects, documented in a `README.md`.
- A library of project documents and documentation for all projects whether or not they were fully completed. Sometimes it is the postmortem documents that hold the most insight.

The knowledge base will expand the most during times of a new project implementation, whether the project is intended for internal or external use. At this time, the project planners and executors will encounter many difficult questions and issues that will need to be solved. This should be approached by first searching the knowledge base for already-existing information on the topic and, if the answer cannot be found, finding it and then updating the knowledge base accordingly for future occurrences. This process is likely to grant an increasing ease to future projects and will make organizational memory richer.

A knowledge base, though a great tool, is also subject to the law of decaying information. If the knowledge base is not frequently updated, it will become inaccurate and obsolete. The modern world is changing faster than ever before, and newer technologies are constantly replacing the old. It has become important to remain continuously on the lookout for old and outdated information and marking it as such. This can be done by a database administrator with a policy that all articles older than, say, two years be considered questionable and marked for review. On the other side, users should be allowed to mark and submit articles for archival deletion if they come across such information.

Knowledge bases are generic stores of information, so in the next section, we will look at the storing and retrieval of information-related models, developed as part of an artificial intelligence project.

Building a Model Library

Over time, your organization will start to create more and more machine learning models. These models are independently valuable and can potentially be reused for future projects. This is similar to the way code bases are maintained in a modular fashion for future reuse. Therefore, starting a centralized repository of AI models and their associated components can pay dividends later on. Given your organization's investment in artificial intelligence is intended to be long-lasting, it makes sense to start this centralized repository sooner rather than later.

Models are typically binary files, but they can also be code or human-readable, structured model parameter files. These "instructions" dictate how the computer is to make intelligent decisions. Although regenerating some models types from their initial components does not guarantee an exact replica of the model (for instance, deep learning models randomly initialize neural network weights), the resulting outputs will be similar.

The model library should be stored on an internal microsite, along with documentation on how the model was generated, how to use the model, and possibly even how to tweak the models (with the necessary training data provided) for future use. A social aspect of shared-model reuse via comments or an internal forum can be added to encourage engineers to exchange ideas among themselves. The forums should have links to the model library, documentation, design decisions, and so forth. It is worth monitoring the forum to ensure that decorum and civility are maintained, because this forum is a reflection on your organization.

The model library will be highly dependent on the organization's projects and use cases. If the subject matter is broad, the library can be more generally categorized based on model type, such as deep learning or statistical. Each model should also be tagged with relevant keywords. A standard tagging format can be developed, specifying the minimum tags to be added to a model. For instance, you could have a tagging scheme that follows this format: framework, use case, department. Tagging the models will help increase searching speed and give the added ability of comparing multiple relevant models.

Ensure that your model library has an indexing mechanism that makes it easy to search and allows people to quickly find what they need. The model library metadata should include the underlying framework used (such as TensorFlow¹ or PyTorch²), documentation about the parameters used to generate the model, and the training dataset(s) used to train the model (or a pointer to the dataset). Having such a detailed model library will speed up the adoption of AI in your organization. Documentation should be concise, crisp, and up-to-date. Without documentation, it may become impossible to reuse an already-developed model, squandering the effort spent developing it.

Model Library Components

When submitting a model to the model library, contributors should be detailed. The following is a guide of the fields that should be included with each model entry:

- **Model description:** First and foremost, a model library entry must describe what the model does. What problem is the model solving? What are considered valid inputs? How do I run the model?

- **Model file(s):** These are the actual model file(s) that can be downloaded and executed.
- **Metrics:** As discussed earlier, metrics such as precision, recall, and the F1 score can be used to determine the performance of a model. If the performance of a model in the model library does not meet the standards for a particular use case, the model can still serve as a starting point, which greatly reduces model development time.
- **Model technology:** The technology used to create the model. This could be a library such as TensorFlow for deep learning models or a language that is used to code a more direct, statistical model.
- **Training data:** A zip file of the training data used as well as some metadata, such as how many samples are contained.
- **Validation data:** In an identical format as the training data, differing only in the data examples contained.
- **Licensing:** Sometimes the model or its training/validation data has restrictive rights around it. For instance, the data associated with a model may be the sole property of a particular partner and cannot be directly reused. The licensing, however, might permit the reuse of derivative intellectual property such as a generated model. In such cases, the training and validation data might be absent from the model library entry.
- **Model parameters:** Any parameters used while making the model that could potentially be tweaked, giving different results. Here are a few examples:
 - **Step size (deep learning):** The size of the learning steps the algorithm uses to make adjustments. The higher the step size, the faster the model can train but at the cost of potentially not converging correctly. Step size can also be variable as training progresses and, if so, can be called out here.
 - **Number of epochs (deep learning):** The number of times the training data is processed to update the neural network weights.
- **Previous uses:** This is a list of all the previous times this model has been used. Having the list of uses allows people to see how close their own use cases are to the previous ones. Additionally, including the contact information for the previous use cases

enables increased organizational collaboration. In this way, more customized information can be directly exchanged than what is included in the model library entry.

- **Tags:** To help with discoverability, a list of tags can be added by the contributor to make the model easier to find. Tags should include words that exemplify the goal of the model as well as other unique model characteristics.
- **Comments:** Having the ability for people to comment on your model is a great feedback mechanism. This social component also enables people to address one another's questions, similar to a forum post.
- **Hosted demo (optional):** Nothing communicates what a model does more than seeing it in action. This component is considered optional from the perspective of a model library because setting up a model to run can be involved and requires maintenance to keep the model running as time moves forward. Lowering the contribution bar by making this optional means that more people will contribute to the model library. Since the benefits of a model library increase exponentially with its size, it is more important to have entries than to ensure that each entry also includes a running demo. Therefore, while it is nice to have, the hosted demo is not a requirement.

An Example Model Library Entry

Let's now take a look at what a sample model library entry could look like given our previously discussed animal image classifier.

- **Model description:** This model classifies an image to determine whether or not it contains an animal. It is specifically trained using dogs, cats, birds, squirrels, and foxes.
- **Model file(s):** `animal_image_classifier_model.zip`
- **Hosted demo:** <https://mycompany.com/modelLibrary/animalImageClassifier>
- **Metrics:**
 - Precision: 0.94
 - Recall: 0.87
 - F1 score: 0.904

- Model technology:
 - TensorFlow (version 1.14)
- Training data:
 - Number of examples: 1,600
 - File: `animal_image_classifier_training.zip`
- Validation data:
 - Number of examples: 400
 - File: `animal_image_classifier_validation.zip`
- Licensing: The training and validation dataset came from a public dataset (<https://datasets.com/animals>), which allows for reuse in commercial applications. The generated model is available for internal use cases but not for external sale.
- Model parameters
 - Step size: 0.003
 - Number of epochs: 10,000
- Previous uses:
 - 08/12/2019—This model has previously been used in an engagement with Acne Corporation. They used the model successfully to remove non-wildlife images from their image lake. Please contact the principal, Jake (jake@acne.com), for additional details.
- Tags: animal, dog, cat, bird, squirrel, fox, acne, tensor flow, deep learning
- Comments:
 - “This model was great. I was able to use its training data as a base and add the animals I needed for a special effects studio.”
 - “Is this model performant?”
 - “When deployed on Google’s Machine Learning Cloud’s base tier, it returns in less than 300 milliseconds for a 3 Mb image.”

Model Library Solutions

Although you can create your own model library from scratch, there are existing solutions for most of the components we have mentioned. For instance, the content management system that we suggested for a knowledge base, WordPress, can also maintain most of the data associated with a model library. A new page could be created for each model and the text-based data can serve as the page’s

content. Pages can also have attachments so that the model files and their associated data can be directly uploaded. WordPress also allows pages to have tags to make them more discoverable. Finally, comments can also be enabled for WordPress pages, adding a social capability to your model library.

The main part of the model library that cannot easily live within a content management system are the hosted models. That capability is more dependent on the particular model and the technology used to create it. For example, TensorFlow models can be hosted using TensorFlow Serving on your own infrastructure. You might then write a simple web application that calls the model with some pre-defined input, which then displays the model results. If the model processes an image, the results of the model can then be superimposed onto the image to visually show how the model works.

Regardless of how you implement the model library concept for your organization, it is important to maintain a collective history for future projects. This will give you more confidence when you're building new AI systems while, at the same time, reducing implementation time through reuse and collaboration.

Contributing to Open Source

The open source community has come a long way from the small hobby projects of the past. Although open source has historically been less turnkey and difficult to use, it has proven to be a big boon for businesses worldwide. Most servers today run on open source kernels, web servers, and even backend databases. Larger companies can afford to open-source much larger pieces of their code, but it makes sense even for smaller ones to open-source noncompetitive pieces of software that have been developed in-house. The creation of goodwill and positive public relations are just a few of the intangible benefits from releasing and participating in open source projects.

Identifying parts of a code base that can be open-sourced without hindering competitiveness can be a tricky process. The best part about this process is that the decision is entirely up to you and your assessment of the code base. The decision of allowing code to be open-sourced can rest with a committee consisting of the product manager, developers, and business heads. A varied team can ensure that

business-critical code is not accidentally released. Google, for instance, open-sourced TensorFlow while keeping the models developed using the same technology private. Some companies, such as Red Hat, operate on more liberal open source policies. Whatever you choose, the benefits of such a move are enormous and often far outweigh the downsides of losing some minor competitive edge. It is important to release only code that can work on its own, without any external proprietary dependencies. In essence, release only code that is independently useful.

A lot of people hold on to the belief that open source software will enable hackers to exploit the software more easily. On the contrary, it has been observed that security through obscurity is almost always less effective. The best security practice is to ensure that systems are fully secure, regardless of an attacker having full information of the protocol. Technologies like public/private keypairs have their implementations fully open and vetted. It is the individual private keys, which are kept secret, that provide the actual security.

Open source projects force the issue of security because its code and any possible vulnerabilities are out in the open. People can then more easily call attention to security issues in public forums, which necessitates that the open source code be fixed if people are to continue using it. Additionally, because the code is available, any good Samaritan can submit fixes that the project maintainers can accept into the core code base. Since the fixes are also submitted publicly, other users of the open source project can apply the security fix for themselves, even before the project maintainers have accepted it into the “master” branch. The ability to bring the whole open source community to bear to find and fix security issues quickly is one of the main benefits of leveraging open source.

Leveraging the open source community does not stop at security, however. An open source code base creates a community of interested developers and enthusiasts around it. This community can give you direct feedback and feature improvements to the code base. With an open source repository public hosted on the Internet using a service like `github.com`, you can leverage the power of developers around the globe. Like security patches, these contributions from the community can then be integrated back into the project. By open-sourcing parts of your software, you, too, can benefit from this community of talent in exchange for making available technology that does not erode your competitive advantage.

Data Improvements

Data, data, data . . . “You cannot make bricks without clay,” as explained by Sherlock Holmes. This applies to AI as much as it does to a detective looking for clues. Artificial intelligence depends on training and test data to come to its conclusions about our world. The models are only as good as the data backing them up. Data is not constant; humans are now generating more new data than ever before. New sensors and novel metrics are increasingly becoming available as we start using more Internet of Things (IoT) devices in our homes and elsewhere. Due in part to these newly intelligent devices, previously unmeasured data is starting to become available for inclusion in our systems.

In this regard, every AI project should have a data improvement planning committee that looks at acquiring new data. The committee should have regular meetings to evaluate data being generated from existing sources and ways to augment them. As discussed in Chapter 4, “Data Curation and Governance,” this data can be sourced internally, found for free online, or licensed. In this modern and complex world, there is never such a thing as a “complete” set of data. There is always data that has been ignored, if only for the reasons that acquiring it would be too costly or the means with which to collect the data have yet to be invented. The committee’s meetings should start with discussions about data that is currently being used and then move on to new sources. AI projects could span multiple years, and using stale data at the start of the project can lead to increasingly bad decision making and analyses. It is important to ensure that newer data is incorporated to reflect current trends. Such decisions for improvements must also be made in the meeting.

Let’s take, for example, a fictional company, Super Steel Inc., which has developed an AI to monitor its factory output and waste. The AI uses data gathered from the enterprise resource planning system to estimate costs and variances. With IoT devices being cheaper and so widely available, the data improvement planning committee decides to install them to track the consumption of iron, its temperature, and its density throughout the process. Such an improvement would fill current visibility gaps, driving better insights into the manufacturing processes and increasing the scope for possible improvements.

Alternatively, let's consider another fictional company, Business Analytics Today, which extracts business insights from equity research. A key insight that the data improvement planning committee might suggest is analyzing the language and tone used in research reports, instead of just the numerical data, leading to a new perspective on stock picking. In this way, the suggestion is not to gather more data, but rather a different way to use the existing data.

AI can also have co-relations where none have been spotted before, purely due to the vast amount of statistical number-crunching and comparison going on behind the scenes. If three data streams are being generated, with two thought to be co-related, the addition of another stream might cause the previously unused stream to become useful as well.

It should be an organization's goal, via a data improvement planning committee, to expand their existing data as well as better leverage the data they have to generate a richer set of derived features. These derived features will lead to higher-performing AI systems and therefore improved organizational performance.

With Great Power Comes Responsibility

Artificial intelligence may be capable of solving complex problems that consist of vast numbers of variables, but we must be ever vigilant in not allowing this computing power to be used for destructive purposes. The data that is collected and stored about individuals can only be assuredly used for the good when it is obtained with the subject's consent and the individual is made fully aware of how the data is being used (remember, privacy policies are important for many reasons). Careful attention must always be provided to algorithms that determine the fate of human lives, because data-only approaches can still be prone to bias.

For an example of a company with access to massive amounts of sensitive data, one needs to look no further than Palantir. Palantir was founded in 2003 by Peter Thiel, a PayPal co-founder.³ The company works with the military and police departments to generate analytics from data. The company's software was a big hit in Afghanistan for intelligence gathering and analysis. This led to them winning contracts with the local police departments and the Federal

Bureau of Investigation (FBI). The company has been involved with analyzing and making sense of the vast amounts of data that is processed by intelligence and local police departments to screen and monitor for potential lawbreakers and troublemakers. The company provides its services via Palantir Gotham for military customers, and via Palantir Metropolis for fraud investigation and internal surveillance to banks, hedge funds, and financial service firms. Various police departments and fusion centers have spent over \$50 million with Palantir from 2009 to 2016.

With the sweeping data that is being gathered and used for threat monitoring, there must come a time to pause and reflect on the preservation of the rights and privacy for the users, or subjects, involved. These systems, when entrusted with the decision to make or break lives, need to be treated with the utmost care and concern for humanity, and algorithms should be made public whenever possible. Although these systems are useful tools when used lawfully and conscientiously, their potential for corruption through nefarious or narrow-minded intent should not be underestimated. Without effective oversight, the possibility of misuse grows higher and higher. It takes only one rogue individual to jeopardize an entire system. This is made all the more likely when the algorithms are not maintained and its flaws are left open for exploitation. In one instance, Microsoft's natural language processing Twitter bot, Tay, had to be taken down after some users taught it to express racist remarks.

With great power comes great responsibility. Although this may sound cliché, it has become relevant in this age, where companies have become large data processors, able to make decisions that could dramatically change lives. It becomes imperative that every algorithm implemented that has an impact on human rights, privacy, and equality be thoroughly tested to remove any potential issues.

Pitfalls

Here are some pitfalls you may encounter while iterating on your AI adoption journey.

Pitfall 1: Assuming a Project Ends Once It Is Implemented

Quite a few project managers and businesses assume that once a project is implemented, the project is done. This is incorrect. Once released, projects are subject to entropy, like everything else, and will start decaying quickly unless maintained. This means that the project plan should include some developer time for postimplementation bugs and improvements. The allocation need not be more than 10 percent of the total implementation time, but it can make all the difference between a successful implementation and a failed one. It is also important to limit the scope of these fixes so as to not derail the existing system as a whole. Additionally, there can be other complications with implementation in large projects, like a last-minute hardware failure, and such complications will also need to be managed.

Pitfall 2: Ignoring User Feedback

After implementation, it is vital to collect and analyze user feedback. Ignoring feedback will doom most projects to eventual failure. Users who feel like they are not being heard may fail to utilize the software's full benefits or resist using the software altogether. This reluctance to change can be justified, and in such a case, the only option can be to tweak the released software. A heavy-handed approach that fails to consider users' needs will hamper productivity and efficiency, ultimately wiping away gains made by the implementation of the new software. Forcing users to use broken or incomplete software is not a sustainable option—eventually, the users will abandon the software entirely, tarnishing your organization's reputation. Software is made for the users, so gathering their input and making changes based on their feedback is a critical requirement for success.

Pitfall 3: Providing Inadequate User Training

Software can be made user-friendly only up to a certain point. To ensure that users can utilize the software correctly and receive the maximum benefit, it is vital to train them to use the new software correctly. There is a good chance that the majority of AI projects will be used by business analysts and the like, having little to no

knowledge of computer science. It then becomes important to explain the tool they will be using to do their jobs properly. Training should be more than just quickly explaining the menus and the UI. It should be a full introduction to the software and how it can positively impact a user's day-to-day work. Training should also span a few days, rather than just one sitting, to help users better absorb the information and give them an opportunity to ask questions. Well-trained employees can do their job more efficiently and accurately by making the best use of the software tools provided to them.

Action Checklist

- ___ Establish forms and other ways for users to give feedback.
- ___ Unless infeasible, act on the user feedback received.
- ___ Start a knowledge base to increase collaboration.
- ___ Start a model library to increase reuse of AI models across your organization.
- ___ Discuss which parts of your code base might be a good candidate to open-source.
- ___ Establish data improvement planning committees that meet regularly to identify new data sources and ideate around better utilizing existing data.
- ___ Periodically audit the storage and use of personal data to ensure good stewardship and transparency.

Notes

1. www.tensorflow.org
2. <https://pytorch.org>
3. https://en.wikipedia.org/wiki/Palantir_Technologies

Conclusion

Artificial intelligence has the potential to transform all organizations. The process by which this transformation happens can vary, but the steps will tend to follow the roadmap we have listed in this book. Following all the steps outlined in the previous chapters will enable your organization to implement and excel in the use of AI technology. AI holds the key to unlocking a magnificent future where, driven by data and computers that understand our world, we will all make more informed decisions. These computers of the future will understand not just *how* to turn on the switches but *why* the switches need to be turned on. Even further, they may one day ask us if we need *switches* at all.

Although AI cannot solve all your organization's problems, it has the potential to completely change how business is done. It affects every sector, from manufacturing to finance, bringing about never before seen increases in efficiency. As more industries adopt and start experimenting with this technology, newer applications will be invented. AI will bring a change even more widespread and sweeping than the introduction of computing devices. It will change the way we transact, get diagnosed, perform surgeries, and drive our cars. It is already changing industrial processes, medical imaging, financial modeling, and computer vision. We are well on our way to tapping into this enormous potential, and as a result, the future holds better decision-making potential and faster, better analytics for all.

Technology has a way of making the once impossible become possible. The key is to recognize which technologies can make a difference in your organization and determine whether they are ready to be used in building real systems. This is a skill learned over time and sometimes requires prototyping to know for sure. Prototyping will help bolster innovation while keeping research costs low. Mistakes are good, as long as they are small and can be easily corrected. Implementing new technology can be a daunting task, but making consistent improvements in small increments will go a long way in keeping your organization ahead of the curve. Staying abreast of AI announcements can also pay dividends in terms of organizational optimization.

The Intelligent Business Model

After following the paths laid out in this book, you will be looking at a changed business. This business will now need a new model to understand and follow—a model that relies on making small errors, then recovering quickly, to realize sustainable and handsome returns. Such a model would imply that you might incur occasional losses as you adjust and cope with newer strategies. Regardless, you will still be innovating, growing, and marching into the future at full speed. Applying Agile frameworks to your organization will beckon growth and large improvements in efficiency while keeping errors and costs low.

This new business model is focused on innovation and growth. With artificial intelligence helping you choose the best actions for your company in this forever unpredictable and uncertain world, you will have a mighty ally as you grow. AI will help us all overcome the drawbacks of the human decision makers of yesterday, who were able to hold only a few variables in mind at a time. This is not to say that AI will replace all human decision making, but rather that it will assist its human counterparts in arriving at sounder, safer, and more sustainable conclusions.

The Recap

The process of AI implementation laid out in the book is a malleable one. It is not written in stone or intended to be followed to the letter. The process must be customized to suit the needs of your

organization. Customization can take the form of skipping a step or adding new ones based on your perception of how your organization operates. A blind copy-paste application of the steps listed in this book would probably give your organization a culture shock and hinder growth rather than supporting it. Do your best to avoid that.

All organizations are different, and their methods of operation are different as well. There can never be a one-size-fits-all implementation methodology for bringing changes on a massive scale. Such changes require careful comparison of the present state of the organization to the expected state, identifying the deltas and then figuring out how best to modify the process to better suit your needs. There may be areas where a simple copy-paste could work, like the implementation of an idea bank, but even there you will need to figure out how best to organize ideas, arrange review meetings, and store your idea bank to ensure that this process does not become a drag on employees. It is important that everyone in your organization understands and is able to implement changes correctly.

Keeping in mind its flexible nature, a short summary of the entire process follows.

Step 1: Ideation

Every project starts with an idea. It is the same for projects involving AI. Bringing ideas to the forefront of your organization will involve inculcating a culture of innovation. With innovation-based focus groups and idea banks, your organization will be equipped with the best tools to start generating new ideas on a consistent basis, increasing your chances of being the first mover. At this stage, it is also ideal to study available technology to learn the challenges involved. Keeping an open mind and ensuring that all ideas receive an audience before being adjudicated upon will go a long way in fostering a collaborative, innovative, and support environment where good new ideas regularly emerge.

Step 2: Defining the Project

With the ideas fleshed out and chosen for implementation, it is time to break an idea into actionable steps. Three ways to draw out a project plan from an idea are design thinking, system analysis, and

the Delphi method. Design thinking is about abstract ideas, finding a wholly new way of looking at existing processes. Systems thinking will help you improve the current systems, whereas the Delphi method is most suited to areas where expert opinion would be required. Using the incorrect technique can give you a bad project plan or doom the project entirely, though the techniques are not always mutually exclusive and it may suit your project to use them in tandem. Measurement criteria for the project should be decided at this stage. Without criteria for success, it is impossible to know what went right or wrong, because no concept of right and wrong will have been established.

Step 3: Data Curation and Governance

An AI system's prime component is data. Without data training and testing, the AI system will be useless. Data can be gathered from internal as well external sources. Internally sourced data will have the lowest hassle and should be able to be used readily. Internal data might need to be digitized if not already stored within computers. Only the data with perceived value should be converted to digital. As you start gathering larger and larger datasets, it becomes necessary to establish data governance procedures if they are not already implemented. Especially since the advent of GDPR regulations in the European Union, harnessing user data now requires establishing governance procedures before the data is collected. Care should be taken to stick to positive data collection techniques to avoid problems of legality or goodwill.

Step 4: Prototyping

Prototyping is the development of small iterations of the project plan, which can be used to demonstrate the project's early value. A prototype must necessarily be functional. Broken software with "pass" statements inside functions are not real prototypes. Before building a prototype, you should look at existing solutions available in the market, since reusing code is generally cheaper and therefore less risky. Defining a logical architecture diagram would be the first step to developing a prototype. At this juncture, some big decisions need to

be made that will have a huge impact on the final outcome: technology selection, which programming language to use, cloud APIs, and microservices. Prototypes should be developed with stakeholder involvement using Agile methodologies.

Step 5: Production

After the prototype is showing value, it is time to scale it up and complete the system for release to end users. A hybrid approach should be adopted for reusing the code from the prototype in the production release, but that should only be done after organizing the code to make it easier to maintain. Automated testing and continuous integration should be implemented before the project is released to the users. Continuous integration techniques help avoid code conflicts, and automated tests alert you to problems before they are pushed out to the users. Implementing a continuous integration pipeline will provide the project with higher-quality code and better software overall. In production environments, hybrid AI systems that use humans to fill in gaps where the AI algorithm fails will give you augmented intelligence. Production environments need to be scalable to handle user loads without failing the system completely or making it unusable. Deploying in the cloud with computing services like IBM, AWS, Google Cloud Platform, Microsoft Azure, and other providers offers solutions to the scale problem with limited effort.

Repeat: Thriving with the AI Lifecycle

The project does not end upon its release. Bugs will need to be swatted. User feedback will need to be incorporated. Gathering user feedback via forms built into the software as well as surveys will help you to identify flaws that need to be resolved. Increasing the intelligence of the system by providing it with newer data and examples is another task that is required to keep the project in a usable state.

A review of your idea bank on a regular basis will help you to spot new opportunities. Based on the resources collected in the project (and resources collected in general), a knowledge base should be implemented in your organization and updated regularly, since it will aid all future projects. A model library implemented as part of

your knowledge base will aid any long-term AI projects and their adoption. Open-sourcing the solution (or pieces of it) will invite the community to help you. Finally, ensuring that data is fresh and regularly updated will keep your models chugging along nicely. Outdated models will give bad predictions and might be detrimental to your business.

So What Are You Waiting For?

This book has covered the journey of adopting AI technologies. From the initial ideation to assembling user stories and available data, to implementing a prototype and then a final production system, your journey has been thoroughly mapped out.

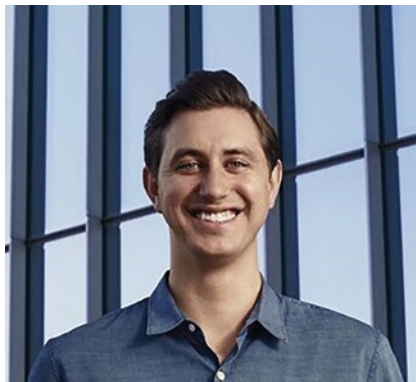
Some of you may be just starting out on your journey whereas others may already be in the final lifecycle stage of your AI system. Even if you have already built your first AI system, there is value in following these adoption methods for future projects or for major system revamps. The roadmap provides a structure—a checklist of what must be accomplished along the way. We wish you and your organization success in making the future a reality today.

AI Experts

AI Experts

In the course of writing this book, we engaged with a number of experts in the field. We spoke with not only technical experts, such as machine learning engineers and data scientists, but also business leaders who have already taken the AI adoption journey. We asked each of our experts five similar questions, resulting in some interesting similarities and disparities among their perspectives. You are invited to glean whatever insights you may from the comparison of their responses.

Chris Ackerson



Chris Ackerson leads Product for Search and Artificial Intelligence at AlphaSense, where his team applies the latest innovations in machine learning and natural language processing (NLP) to the information discovery challenges of investment professionals and other knowledge workers. Before AlphaSense, Chris held roles in both product and engineering at IBM Watson where he led successful early commercialization efforts. He is based in New York City.

1. Given you have expertise with AI and how people interact with it, are there any insights or tips that you'd like to share?

AI development is a new and different form of software development requiring new tools, processes, and job roles. Building an effective AI development organization requires investment in each of these areas. My experience is that teams often focus too much on algorithm design and architecture at the expense of building an effective AI development organization. The result is they back themselves into a corner as technology rapidly improves and they are unable to keep up. Just check out the change in the leaderboards for state-of-the-art performance in any AI task to see what I mean. It goes without saying that algorithm design is central to AI, but teams should expect to rip and replace algorithms on a regular basis. While every project is different, a good general rule is to prioritize open source, be leery of black-box APIs promising superior performance, and invest most of your resources in a system that allows you to train and deploy completely new model architectures in regular intervals with low overhead.

2. What has been your biggest challenge while adopting AI?

Data collection continues to be the biggest barrier to broad adoption of AI. State-of-the-art deep learning models require massive quantities of clean data in order to make accurate predictions. For example: Google's influential BERT model for natural language processing, which was trained on a corpus containing billions of words; self-driving AI trains on millions of real and simulated hours of driving; and the sentiment analysis algorithms we developed at AlphaSense, which learn from hundreds of thousands of corporate earnings calls.

Whether or not these types of datasets are even accessible in your AI project is the first question you should answer, but assuming you can acquire the data, the bigger challenge is developing the tools, processes, and labor to clean and label it. The research community has made enormous progress in the last decade, improving the raw predictive power of AI, but commercial applications have lagged behind because of the practical challenges of building robust datasets to take advantage of algorithmic horsepower.

3. What advances in AI do you envision over the next five years?

In the next five years, I believe we will solve the problem of generalizing AI models within the scope of a narrow domain. That sounds like an oxymoron, but the vast majority of successful AI projects to date have been developing models for just a single prediction objective. This is the narrowest definition of “narrow AI” possible. Let’s say you want to build an AI to monitor social media for stock trading signals; today you would train independent models for sentiment analysis, entity recognition, topic extraction, and any other classification task relevant to your project. When IBM developed its oncology technology, the company spent enormous resources developing independent models for each individual cancer, with few economies of scale between each new target. The practical implication is that the cost of solving real-world problems that generally involve many separate objectives is prohibitive. Intuitively we understand that all of those social media-monitoring tasks are highly related, as a human interpreting the sentiment of a tweet is critical to extracting the key topics, and vice versa. Transfer learning—that is, sharing the learning from one task to a related task—is an area of intense investment in the AI community. Google’s BERT is an important step in that the model is pre-trained in an unsupervised way by consuming huge amounts of unlabeled text, and is then fine-tuned with supervision for specific tasks like sentiment analysis. Having a single framework for solving multi-objective problems like social media monitoring or identification of anomalies in medical images will lead to a huge increase in the number of real-world AI applications.

4. What job functions do you see as a prime target for AI assistance over the next three years?

The next three years will be dominated by advances in natural language processing. In the same way that structured data analytics and visualization have become ubiquitous across the enterprise, all knowledge workers will be assisted by AI that draws insights from the vast number of unstructured text documents related to their job function. Consider analysts in corporate strategy; every day there are tens of thousands of new research reports, news articles, corporate filings, and regulatory actions that contain information absolutely critical to making the right investment decisions for their company. In the same vein, a sales executive going into an important meeting, an HR rep deciding on a corporate policy, a product manager prioritizing features in an application will all benefit from AI that surfaces timely insights from this sea of text. As a knowledge worker, imagine the productivity increase if you were given a team of analysts to prepare you each day. Or if you have a team, imagine everyone on your team was given a team. The result is a boost to the collective IQ that will lead to better decisions across all industries.

5. Any other thoughts about AI adoption you'd like to share?

One of the most common reasons AI projects fail is that teams underestimate the importance of great product design. The mystique of AI leads to a belief that the algorithm is all that matters. This is especially common in internal enterprise applications. The reality is that great product design is just as important in AI-powered products as in traditional software applications. Consider Google Search, where the placement and construction of every link, feature, and answer box is meticulously crafted to increase the speed of access to information. Over many years Google slowly and thoughtfully evolved how AI could enhance the search results pages for billions of queries without getting in the way of user experience. If AI is to reach its potential and become ubiquitous across software applications, it behooves companies to hire and develop product managers, engineers, and designers who understand the capabilities and limitations of AI and invest in ways that prioritize great user experience over technology choices.

Jeff Bradford



Jeff Bradford is the founder and CEO of Bradford Technologies, a developer of real estate appraisal software. He has over 31 years of experience providing appraisers with innovative software solutions and is a nationally recognized expert in computer technology and analytics. Jeff has been recognized as a Valuation Visionary by the Collateral Risk Network and as a Tech All Star by the Mortgage Bankers Association. Prior to founding Bradford Technologies, Jeff notably worked at Apple Computer, Structural Dynamics Research, and FMC Central Engineering Labs. He holds three master degrees in engineering mechanics, computer science, and business administration.

1. What has been your experience pulling together data for AI purposes and what are your thoughts on the need for building an initial prototype?

If you are a first-time, wanna-be user of AI/deep learning technology, you definitely want to start with a prototype case. Something simple. This is also not the time for trial and error and learning as you go. You want to know if this is going to work as quickly as possible for the least amount of money, so select a consulting firm that has considerable experience in AI. Have them evaluate your training data.

Is it good enough? Does it have to be massaged? What AI model are you going to use? Which is best for your application? Have them run some tests. What are the results? If the results are promising, then you can make some strategic decisions about incorporating AI into your products and services.

2. What has been your biggest challenge while adopting AI?

The biggest challenge has been the training data—selecting and assembling the data into structures that can be used to train a model. How well the model works is directly related to how good the data is, so the quantity and quality of the data is critical to the success of the AI model.

3. What advances in AI do you envision over the next five years?

I think that in the next five years, AI will have advanced to the point where models can be pieced together to form very comprehensive systems that can augment many facets of work environments as well as everyone's lifestyle. Everyone may have their own virtual companion or work assistant.

4. What job functions do you see as a prime target for AI assistance over the next three years?

Any function that involves communication, data, or the providing of recommendations can be automated or augmented by a virtual assistant. These are the areas that will be targeted by AI systems.

5. Any other thoughts about AI adoption you'd like to share?

Everyone should learn about AI: its potential and its shortcomings. It is definitely going to be part of everyone's life. It is not going away. It could become Big Brother, as it is already becoming in China, or it could be used to enhance one's life and work experiences. Learn how to use it to enhance humanity.

Nathan S. Robinson



Nathan S. Robinson is an Ohio-born, China-raised, Oklahoma-schooled, Austin, Texas, transplant. Nathan began his career in product management and artificial intelligence at IBM Watson. He currently works as a product manager at Babylon Health, an organization leading the charge in making a health service accessible and affordable to all using technologies like AI. Prior to joining Babylon Health, he worked in product management at Keller Williams, where he owned Kelle, a virtual assistant and mobile application hybrid for realtors.

1. What are the key benefits of AI based on your experiences using it within your organizations?

There are many benefits of utilizing artificial intelligence in an organization. AI can help automate simple and repetitive tasks, freeing up time for team members to focus on higher value, human-centric tasks.

Artificial intelligence can be leveraged to process and understand large values of data, and unlock insights that would not have normally been found.

2. What has been your biggest challenge while adopting AI?

Aligning both the organization's and the end user's expectations to reality. Artificial intelligence is powerful and can provide immense value, but setting expectations for the end users, as well as the internal organization, is key in having a great experience.

Everyone has data, but not everyone has good data that is ready to use. Finding, cleaning, normalizing, and preparing data is often the bulk of many AI proofs of concept in an organization.

3. What advances in AI do you envision over the next five years?

There will undoubtedly be incremental advancements in existing technologies as a result of better datasets, tweaking of models, improvements to training methods, and more.

There will also be advances that come as a result of increased processing speed from technologies like quantum computing.

It is also likely to see new innovations within the field to "leap" us forward. Things such as combining technologies to simultaneously use multiple forms of input as the training data in order to gain a "1 + 1 = 3" type of value from the output. Think: using both audio and video inputs to train a model. Or finding alternative ways (not just improvements to existing methods) to train models in the first place.

4. What job functions do you see as a prime target for AI assistance over the next three years?

I don't have any new insights here. Things that are repetitive or require little creativity and unique problem solving are at risk of automation. However, more than replacement, AI will more commonly be used to augment humans doing a job. Rather than AI replacing people directly, it's more likely that it will augment fewer people to be more productive.

5. Any other thoughts you think are relevant to AI adoption?

Culture and human experience is crucial in a company's successful adoption and implementation of AI. The organization's culture can

both dictate whether or not they successfully utilize AI, as well as how quickly they can achieve value from it. Setting the expectations of the end users of the AI, as well on the organization's side, building/investing in it, is crucial in an organization's long-term AI strategy.

Evelyn Duesterwald



Dr. Evelyn Duesterwald is a principal research staff member and manager of the AI Lifecycle Acceleration team at IBM Research AI. She holds a PhD in computer science and is passionate about all aspects of AI engineering and the AI lifecycle, with a current focus on securing AI models and AI operations. Her interdisciplinary research focuses on the intersections of artificial intelligence, security, and software engineering.

1. Given your expertise with AI security, are there any insights or tips that you've found to be effective?

AI security is usually not about hacking and breaking into systems. AI security threats can result from seemingly harmless interactions with exposed AI interfaces such as APIs. An adversary may craft inputs to force a particular model response (these are called adversarial inputs) or to poison the data that the underlying models are trying to learn from. For example, in speech recognition systems you just have to add

a little bit of imperceptible adversarial noise to a speech recording (e.g., “Hello, how are you?”) to cause a dramatic and targeted misclassification by the speech model (e.g., the model hears “Order a pizza.”)

We don’t want to be surprised by what might happen as a result of malicious users of AI, so we have to build security into our models from the ground up so as to train robust models that are resilient to these kinds of adversarial exploitations. Training robust models is a very active research area, and new approaches are being developed all the time by the larger AI research community, as well as by our IBM Research AI teams.

2. What has been your biggest challenge while adopting AI?

The significant lack of automation and process standardization that you typically see across the AI lifecycle. Everybody is training their models in their own way and without an accountable or repeatable process. There is an urgent need to bring the rigor of modern software engineering to AI. Even very basic engineering concepts like systematic version management are often lacking.

3. What advances in AI do you envision over the next five years?

We need to move AI from an art into an engineering discipline. Some of the largest recent gains in AI capabilities have been possible through deep learning, especially in speech and image recognition. But deep learning is also one of the areas where AI most resembles an art, which severely hinders adoption. We cannot trust an AI that we cannot fundamentally understand. So I expect the most impactful future advances in AI to be in the areas of interpretability and “explainability” of AI.

4. What job functions do you see as a prime target for AI assistance over the next three years?

We are starting to see some traction in making AI itself as a target for AI assistance—in other words, AI-assisted AI. For example, teams are working on new AI-powered tools that aid the data scientists in building AI by assisting in processing their data, in extracting relevant features, and in training and testing their models. Just imagine talking to

your AI assistant: “Hey, I have data streams of customer comments; build me a model that can tell me what my customers are talking about.”

5. Any other thoughts about AI adoption you’d like to share?

We are seeing great advances of AI in assistance roles—personal assistants in your home as well as professional assistants in the office. Adoption of AI in more autonomous decision-making roles is further behind. Autonomous vehicles are a great example, but here we are slower to move past experimentation. To make autonomous AI a practical reality, we first need to address the various adoption obstacles that I mentioned earlier. We need more productivity in our AI engineering practice, and we need robustness assurances and safety guarantees through more explainable and trustworthy AI. AI is an incredibly fast-moving field. We will undoubtedly get there, and it’s a very exciting journey be on!

Jill Nephew



Jill Nephew is CEO and founder of Inqwire, PBC, an interactive platform whose goal is to help people make sense of their lives. From scientific modeling to software language and tools construction, to domain-specific AI and IA systems, Jill has always sought out novel ways to use technology to help people think better.

1. What has been your experience pulling together data for AI purposes, and what are your thoughts on the need for building an initial prototype?

Back in the 1990s, I worked on supply chain planning and scheduling solvers. We tried out a method that has similarities to today's artificial neural nets in that it iterated, did hill climbing, started with a random landscape, and was a black-box algorithm designed to converge on a global optimum. We had widespread adoption because we could show that we produced an objectively better solution. However, what we didn't anticipate was the need for "explainability" of the results.

The schedulers and planners using the software most often weren't the ones who made the purchasing decisions and also weren't given the opportunity to weigh in on the black-box nature of the algorithm. So when they attempted to use it, they expected that if it gave an answer that they thought didn't make sense, they should report that as a bug and we should fix it. Since this was a black-box algorithm, most of the time there was no way to fix it, and this was unacceptable. The actual workers whose decisions we were supplying decision support to were held accountable by their decisions, and this meant we were dealing with a kind of unanticipated revolt from the ground up against the adoption of our solver.

If the system had perfect answers or answers that were visually, obviously correct in the absence of explanation, this pressure might have been relieved by getting global organizational buy-in that the decisions made by the algorithm could be accepted without question. However, for our system we couldn't converge on this. I am skeptical that any system can, because when a human inspects the thinking of a machine, the strong unstated assumption is that if you are saying the system is intelligent, then it has common sense and its decisions will make sense. We didn't find any organizational support for pushing back against this expectation.

Things got worse and better at the same time when we were tasked with fixing the algorithm to include common sense and explainable results. We did this by building a constraint-based solver. The core solver would not yield to this shift in approaches, so a layer was built on top that could. The global optimization solution would serve as starting conditions for the constraint-based solver that would be a kind of clean-up phase. This went a long way to "straighten out"

where the core algorithm went wrong, but we still found it incredibly challenging to undo far-upstream decisions by the system. What happened next was the really big insight.

When we deployed this new clean-up phase, the users loved it and then came back with a request that we give them a way to skip the global optimization as the starting solution and just have the constraint-based system generate one. So basically, they asked for a steepest descent, explainable solver. We gave them that option and it was widely preferred over the other one, despite the fact that we could show the final solution was less optimal.

The takeaway was that, when you are thinking in objective functions and global optimizations, it seems absurd to build anything that would settle for a less optimal solution. But when humans have to take responsibility for the decisions of the system, a more explainable, suboptimal solution might just win out.

I think we are living in interesting times for algorithms. The market is just getting educated enough to start to ask hard questions about accountability and “explainability,” and anyone implementing black-box solutions who can’t do either should probably think through a backup plan. Further, making sure that the actual users of the software (not necessarily the ones who are making the purchasing agreement) understand the implications of having a black-box algorithm making decisions is essential to avoid being caught in a support nightmare.

2. What has been your biggest challenge while adopting AI?

Black-box algorithms. No matter how much we explained the nature of the algorithm and that we couldn’t just “reprogram” it to fix it, the users still expected it to work like all the other software systems they have used. There is an obvious commonsense problem with the solution, and therefore the system needs to be fixed. We never found any way to push back on this expectation, and it drove our team to ultimately decide to abandon a black-box approach.

3. What advances in AI do you envision over the next five years?

What I envision is very shaped by past experiences and may not at all reflect where the world actually goes. But if I had to do the

thought experiment, I would imagine that as the market is educated on how global optimization currently comes at the price of “explainability,” we will abandon any AI that uses black-box algorithms and switch to systems that don’t.

I think that we will also face a real crisis around what a pure data solution can actually deliver. There is a lot of faith right now that we are accumulating enough data to overcome first-principle limitations such as most of the world generating sparse, dynamic, evolving, and non-normative data. There is also a complete lack of understanding of the value of incorporating process models in AI.

I think there will still be an opportunity to keep the dream alive of using cognitive-based technologies in new ways to attack wicked problems, provided the field can pivot toward systems that can start to incorporate process models and simulation instead of optimization. If we can pivot, I would predict that AI experts who know how to work intimately with domain experts and do modeling will have budgets to build a lot of new and imaginative systems and solutions as they re-channel this market interest in cognitive technologies.

4. What job functions do you see as a prime target for AI assistance over the next three years?

By “prime target” I am speaking to where I think there is a need, not another business model. The main need I can see AI addressing is looking for patterns in domains where patterns aren’t expected—particularly in the human life support system. Monitoring the natural environment isn’t cut and dried. Scientists need to be alerted when something has changed. We don’t currently have a way to do this. We find things, like the ozone hole, by accident.

The software in place to monitor the ozone didn’t expect a hole. The scientists who programmed the algorithms didn’t expect a hole was possible based on their understanding of the stratosphere, and they programmed monitoring algorithms to discard outlier data as a systematic error instead of reporting it thoughtfully. This oversight took years to track down, and we are fortunate that it eventually was. The mechanisms behind the ozone hole formation were so complex that it took multiple scientists years to figure out that it was actually

coming from propellants in spray cans. The takeaway is that monitoring, along with the thoughtful reporting of anomalies, is critical for the ongoing protection of our human life support system. We need many more of these systems, and I hope someday that information technologists and scientists together can find ways to convince decision makers of this point.

5. Any other thoughts about AI adoption you'd like to share?

When I talk with newer AI enthusiasts, I hear a common misconception that if we had infinite data, we would have perfect knowledge. It has a commonsense ring to it and often goes as an unstated assumption. Anyone who has studied the nature of the physical world, feedback mechanisms, chaos, quantum mechanics, numerical analysis, the nature of counter-factual information, and how mechanisms can't be restored from pure data, or simply thought through the pragmatics of what it would take to collect perfect knowledge, knows this is not true. Confronting this head-on as a starting assumption with any discussion of adopting an AI project may cost some sales, but ultimately it may mean long-term success in terms of setting realistic expectations of project outcomes.

Rahul Akolkar



Rahul Akolkar leads Worldwide Technical Sales for Data Science and Artificial Intelligence at IBM. He has worked on some of the largest implementations of AI in past few years and established AI delivery teams on four continents. Rahul previously held a number of roles in IBM Research and Corporate Technology. Rahul has contributed to open source projects and W3C standards, and he is an author of numerous research papers and inventor of dozens of granted patents. He holds graduate degrees in computer science and mechanical engineering from the University of Minnesota.

1. Given you have expertise with AI and how people interact with it, are there any insights or tips that you've found?

There are a number of detailed recommendations that exist on every aspect of creation and evolution of AI, but the singular insight I can offer is one must focus maniacally on the elimination of friction between the interaction of AI and other actors, whether people or processes. Some of the ubiquitous embodiments of AI that we find today in terms of assistants on our phones or devices in our homes can also be the source of frustration. AI that either creates an aura that it can answer anything or is not engaging in the right user experience is bound to become shelfware over time. We need AI that people want to interact with, that is available in the right context, that is using the right modality, that provides information and value consistently, and that fades in and out of the experience as needed.

An important part of eliminating the friction in the experience revolves around the general notions of trust, transparency, auditability, and ethical behavior of AI. People who interact with AI increasingly, and rightfully, demand that AI can no longer be magic, not to them, or to regulators, auditors, overseers, when AI is providing input to influence increasingly important decisions in fields such as medicine, insurance, crime prevention, investment, and security, both IT and physical.

2. What has been your biggest challenge while adopting AI?

It's very important to lay out an organization's journey to AI while providing the right organizational and technical governance models and to begin delivering incremental value in short order. There are

certainly elements such as customer engagement and the broad category of recommender systems that can go to production in short order, whereas other areas such as deep domain expertise and esoteric vocabularies of interaction often require a phased implementation. The two common fallacies I find in terms of AI adoption are (a) moonshot or nothing, where one tries to win it all by going for a large prize, without incremental gains in process, and (b) one and done, with the plethora of data science and AI platforms and tools out there, it is fairly easy to deliver quick value and celebrate success prematurely, but it's important to keep track of the principles that will deliver lasting value with AI; otherwise, a successful launch can quickly devolve into a less than optimal experience.

Those who get the best business value with AI are those who have taken the due measures to foster a fitting culture for AI adoption within their organization. Where AI differs from a lot of traditional software and systems is that it requires near constant care and feeding, as it is trying to provide value in a changing environment and adapting as the real world or its environment changes (think of retraining models or adding a new component to an ensemble recipe). One might then wonder, why bother with AI at all? Essential as tabulating and procedural computing devices are, the world has moved on in terms of expectations from technology. A vast number of afore-unthinkable experiences can now be powered by AI, and those who harness those effectively will conquer the hearts and the minds of their audiences and customers.

3. What advances in AI do you envision over the next five years?

We're already seeing advances in the transparency of AI, not just around classification and structured predictions, but also around neural nets and deep learning techniques. I see this work around "explainability" of AI, and detecting bias in AI, as one that is ripe for advances, both in operational runtimes as well as tools. We will get much better at interacting with data through virtual data pipelines and distributed querying of disparate sources to power AI. Another advancement we're already seeing the tip of the iceberg on, and I see accelerating, is the use of AI techniques in the design of AI—for example, trainless predictions in hyperparameter search spaces and

neural network architecture selection, a second-order function illustrating a field at the cusp of new breakthroughs. We will also see advances in creating compelling AI interfaces in terms of the sophistication of conversational agents, visual and speech services such that they can be more effective and scale (a welcome addition to anyone who has had to repeat an utterance numerous times to their phone or home device). Finally, of immediate significance to the creators of AI, we are going to see a growing emphasis on open platforms and end-to-end governance via AI fabrics and a maturing field of AIOps, with practices similar to the analogous ones in DevOps and DevSecOps.

4. What job functions do you see as a prime target for AI assistance over the next three years?

Job functions that are based on collection and retrieval of data are prime targets for AI assistance. The second area is around providing expert assistance in context and covering vast areas of information. Predictive models that help make recommendations to human subject matter experts for any key decision is another area. There is a longer list of broad categories, but in essence any job function that requires customer engagement, or entrenched learning and incremental expertise development, is a good target. This has led to some level of uneasiness when we talk about AI and jobs, but I don't see it largely differing from, say the Industrial Revolution, where a number of job functions were lost to machines replacing human workers in industrial settings on the shop floor or in the farmlands. To summarize, I think AI will enable humans to be more productive, to make better informed decisions and focus on the less mundane tasks that AI can easily master.

5. Any other thoughts about AI adoption you'd like to share?

AI is here to stay. It is creating and will continue to create disruptive innovation at a faster pace. Organizations owe it to their future selves to explore how they can adopt AI, and there is a wealth of knowledge available for anyone who seeks to learn more.

Steven Flores



Steven Flores is an AI engineer at Comp Three Inc. in San Jose, California. He leverages state-of-the-art methods in AI and machine learning to deliver novel business solutions to clients. In 2012, Steven earned his PhD in applied mathematics from the University of Michigan, and in 2017, he completed a postdoc in mathematical physics at the University of Helsinki and Aalto University.

1. Given your expertise with AI, are there any insights or tips that you've found to be effective?

Much of the academic and popular literature paints AI in an idealistic light, which can lead to misconceptions about its practice. Spectacular and even miraculous results hide underlying limitations and a long list of failed ideas that preceded them. Indeed, when one actually uses AI, breakthroughs take a long time to achieve. Models are constantly redesigned, tuned, babysat during training, tested, and discarded, and the decisions that go into this process may have vague, unsatisfying motivations. Good results are hard-fought for over months and even years of work. They rarely work well out of the box.

Because of this reality, AI practitioners must participate in the larger AI community to be successful. They must read the research literature, attend talks, discuss their problem with experts, and listen to experts talk about theirs. They must apply insights from expert

knowledge and experience to their problems, because the solutions usually do not live in textbooks. Doing so can speed up progress on work in the nebulous frontier of AI.

2. What has been your biggest challenge while adopting AI?

One of my biggest challenges is proper formulation of the problem to solve with AI and how to measure success. This is difficult because the problems that we encounter in real life are vague and qualitative, but the problems that we can solve with AI are precise and quantitative. Translating between these worlds is challenging.

Another challenging aspect of AI is that it seems to be more a bag of mathematical techniques and models than a unified theory. Practitioners of AI always want to find and use the best methods, but they may not understand why one is best. I think that making informed strategy and design decisions is difficult in this environment.

3. What advances in AI do you envision over the next five years?

Reinforcement learning is one of many areas in AI research that has seen recent breakthroughs. One of the most famous examples is attaining super-human performance on the classic Chinese game of Go. However, as impressive as these gains are, the solutions behind them often do not generalize beyond the original problem. Progress on this front is expected to grow in the coming years, with huge implications for industry.

4. What job functions do you see as a prime target for AI assistance over the next three years?

AI will have a growing impact on many sectors of our workforce. To anticipate the future, let's look at some examples of what is happening today.

In media, AI assists reporters with research, data crunching, and in some cases, content creation. This frees up reporters to work on less mundane and more nuanced parts of their work that AI cannot handle.

In the service sector, chatbots handle end-to-end routine questions made to call centers, and they route calls regarding more complicated matters to the proper human staff member.

In medicine, AI is used for diagnostics, like tumor detection, data mining in medical records, and more.

In genetics research, AI is used to significantly enhance our understanding of the human genome. Future results may lead to the eradication of genetic disease and insights into healthier life choices for certain genetic fingerprints.

Throughout industry, computer vision has seen widespread adoption. Today, computers read handwriting, detect nanoscale defects in silicon wafers, and provide safety assistance to car drivers.

In all of these cases, AI is used to enhance human ability by finding hidden patterns in complex data and making better decisions with it, and to free up human talent for more creative or subjective work by handling more routine and objective tasks.

5. Any other thoughts about AI adoption you'd like to share?

Right now, we are living in the first generation of AI integration into our lives. AI in its present state has potential applications well beyond its current use, so there is a lot of room for innovation that builds on its current status. As this happens, future breakthroughs will push the horizon of possibility further, from simple decision making based on complex models made from big data, to perhaps something smarter and more human: complex decision making based on simpler models made from less data.

Roadmap Action Checklists

Step 1: Ideation

- ___ Start by building a culture of innovation in your organization. Ideas will come from the most unexpected places once this culture is set in place.
- ___ Form an innovation focus group consisting of top-level managers who have the authority to make sweeping changes.
- ___ Start maintaining an idea bank.
- ___ Gather ideas via scrutinizing standard operating procedures, process value analyses, and interviews.
- ___ Sort and filter the idea bank using well-defined criteria.
- ___ Do timely reviews to trim, refine, and implement ideas.
- ___ Learn about existing AI technologies to gain a realistic feel for their capabilities.
- ___ Apply the idea bank to the AI models learned in the previous step to find those ideas that are suitable for the implementation of AI.

Step 2: Defining the Project

- ___ Identify the idea to be implemented.
- ___ Identify all possible stakeholders for the system you want to build.

- ___ Select the appropriate method for making the project plan (design thinking, systems thinking, scenario planning, etc.).
- ___ Use design thinking (if applicable) to come up with persona(s) who will be using your system.
- ___ Define and prioritize measurable user stories that, when implemented, will provide user value.
- ___ Establish success criteria for the entire project.
- ___ Finalize the project plan and begin prototyping.

Step 3: Data Curation and Governance

- ___ Determine the possible internal and external datasets available to train your system.
- ___ Have a data scientist perform a data consolidation exercise if data is not currently easily accessed.
- ___ Understand the data protection laws applicable to your organization and implement them.
- ___ Appoint a data governance board to oversee activities relating to data governance in order to ensure your organization stays on the right track.
- ___ Put together a data governance plan for your organization's data activities.
- ___ Create and then release a data privacy policy for how your organization uses the data it accesses.
- ___ Establish some data security protections such as using data encryption, providing employee security training, and building relationships with white-hat security firms.

Step 4: Prototyping

- ___ Select which of the top user stories are feasible and will be implemented as your prototype.
- ___ Determine if there is an available solution on the market that can be used to save time and resources.
- ___ Decide if you have the necessary talent in your organization or if you need to supplement by contracting resources.

- ___ Design the prototype and use the technology selection process to determine how to build the prototype.
- ___ Use Agile methodologies to iteratively build the prototype with regular stakeholder feedback.

Step 5: Production

- ___ Reevaluate user stories to ensure that they are still relevant.
- ___ Establish a continuous integration pipeline with automated tests to ensure system quality.
- ___ Allow the system to involve human intervention as necessary.
- ___ Perform load testing on your system to ensure that it and its components are scalable.
- ___ If your system is deployed in the cloud, review the SLAs and make sure they are sufficient for your user stories.
- ___ Release the live production system to users and begin the feedback lifecycle process.

Thriving with an AI Lifecycle

- ___ Establish forms and other ways for users to give feedback.
- ___ Unless infeasible, act on the user feedback received.
- ___ Start a knowledge base to increase collaboration.
- ___ Start a model library to increase reuse of AI models across your organization.
- ___ Discuss which parts of your code base might be a good candidate to open-source.
- ___ Establish data improvement planning committees that meet regularly to identify new data sources and ideate around better utilizing existing data.
- ___ Periodically audit the storage and use of personal data to ensure good stewardship and transparency.

Pitfalls to Avoid

Step 1: Ideation

Pitfall 1: A Narrow Focus

Artificial intelligence is an emerging field with wide applications. Although trying to solve every problem with artificial intelligence is not the right approach, care should be taken to explore new potential avenues and ensure that your focus is not too narrow. During the ideation stage, it is essential to be as broad-minded as possible. For instance, consider how AI might be able to improve not only your core business but also auxiliary functions such as accounting. Doing so while acknowledging the limits of real-world applications will facilitate idea generation. Some applications for AI can also be relatively abstract, benefiting from lots of creative input. All ideas that are considered plausible, even those in the indeterminate future, should be included in the idea bank.

Pitfall 2: Going Overboard with the Process

It is easy to get carried away with rituals, thus sidelining the ultimate goal of generating new ideas. Rituals such as having regular meetings and discussions where people are free to air their opinions are extremely important. Apart from these bare necessities, however, the focus should be placed on generating ideas and exploring creativity,

rather than getting bogged down by the whole process. The process should never detract from the primary goal of creating new ideas.

Pitfall 3: Focusing On the Projects Rather than the Culture

For an organization, the focus should be on creating a culture of innovation and creativity rather than generating ideas for current projects. A culture of innovation will outlast any singular project and take your organization to new heights as fresh ideas are implemented. Creating such a culture might involve a change in the mindset around adhering to old processes, striving to become a modern organization that questions and challenges all its existing practices, regardless of how long things have been done that way. Such a culture will help your organization much more in the long run than just being concerned with implementing the ideas of the hour.

Pitfall 4: Overestimating AI's Capabilities

Given machine learning's popularity in the current tech scene, there are more startups and enterprises putting out AI-based systems and software than ever before. This generates a tremendous pressure to stay ahead of the competition, sometimes using just marketing alone. Although incidents of outright fraud are rare, many companies will spin their performance results to show their products in the best possible light. It can therefore be a challenge to determine whether today's AI can fulfill your lofty goals or if the technology is still a few years out. This fact should not prevent you from starting your AI journey since even simple AI adoption can transform your organization. Rather, let it serve as a warning to be aware that AI marketing may not always be as it seems.

Step 2: Defining the Project

Pitfall 5: Not Having Stakeholder Buy-In

AI solutions tend to affect all parts of an organization. Their transformative nature requires data from multiple groups and stakeholders. As a whole, organizations are often resistant to change, so it is important that each stakeholder's input is incorporated at the earliest

stages of the project. It is in every project's best interest to ensure that its projected benefits have been clearly explained to everyone involved. The only possible path to shaking the fear of change is to explain why that change is in the best interest of the stakeholders. The current users of the system might be facing a problem that the proposed system is not addressing. New research may have revealed an alternative revenue prospect that your organization was previously unaware of. Whatever the cause for change, in the event that a stakeholder is not consulted beforehand, they may become the biggest roadblock for your project's adoption. The effects of this pitfall will likely not be felt until you've deployed your solution in production during step 5, but it is important to address this pitfall during the project definition step, before it has time to snowball.

One way to avoid this is to have an initial kickoff meeting that includes all possible stakeholders. It is important to be liberal with the attendee list because this is the meeting where everyone will formerly discuss the project for the first time. Although people may have heard about the project in hallway conversations, they must at some point be officially informed about the project and invited to participate in its development and success. Keeping people in the loop, with a clear channel to propose comments and suggestions, will enable them to have a sense of ownership with the project down the line during its implementation.

Pitfall 6: Inventing or Misrepresenting Actual Problems

One hazard when exploring a new technology is searching for a problem you can solve with it, rather than the other way around. Just because you have this shiny new hammer does not mean the world is suddenly full of nails. The focus, as we have discussed, should be on the pain points of your organization. Decide what problems exist or what opportunities are out of reach, and then develop a solution to resolve the problem. If you avoid this advice, you will end up trying to fix things that are not broken or addressing problems that never even existed in the first place. Do not try to fix what is not broken; that will only lead to extra costs and delays. Having a firm grasp of what the process flows of the company look like will help you target only the deficient areas.

Pitfall 7: Prematurely Building the Solution

As you work through developing your project plan, you may be peripherally aware of services and commercial technologies available today that provide AI capabilities. It would be a mistake, at this point, to select a particular vendor with whom to partner. This is a common mistake made by organizations that, by selecting a vendor too early, unintentionally limit themselves to the capabilities offered by that vendor. Instead, continue focusing exclusively on developing your user stories. Think about the users of your system and how you can make their lives easier. If you know your users use social media, for example, consider making social media integration part of your scope. Establishing these requirements during the project definition phase will greatly simplify selecting a vendor once you reach that stage.

Another risk you run in prematurely selecting a vendor is being tempted to include features available through your vendor that have no relevance to your users. For example, if that chat technology you found provides a text-to-speech capability, you might be tempted to include that in your project, not because it is helpful to your users, but because you want to fully take advantage of your investment. This runs the risk of complicating your efforts, obscuring your project's aim, and unintentionally spending your limited resources on capabilities with minimal value. Again, your focus at this stage must be your users and defining your project plan, not on picking a technology or partner.

Pitfall 8: Neglecting to Define Formal Change Request Procedures

Change is a natural part of the Agile process. Although this change can be a great thing, it is also possible for projects to become bogged down or impeded by a lack of consistency. If the requirements of a project change too often, or without sufficient cause, that can lead to developer confusion and a failed project. For this reason, it is vital to establish a lightweight but formal change request procedure. Every change should be approved by the product manager, as they have the ultimate responsibility for the project. The product owner should scrutinize and evaluate each change request carefully to assess the

impact on the project. Requests that have no clearly illustrated necessity or benefit should be ignored and not passed on to the developers.

Pitfall 9: Not Having Measurable Success Criteria

For large and small projects, it is imperative to assess their impact on the organization. Such an assessment will identify shortcomings and lessons for future projects. In order for this kind of assessment to take place, the project's scope and success, as discussed earlier in the section "The Components of a Project Plan," need to be defined clearly. Projects can fail. It is what you learn from each one as an organization that will separate you from the rest. People who are afraid to fail might neglect to set empirical goals by which to measure their success or failure. However, it is vitally important for you to know if you are off track sooner than later so that you can make course corrections. Agile is built around the philosophy of fail quickly, fail regularly, and fail tiny. Small failures are easier to recover from and allow learning in a nondestructive manner.

Step 3: Data Curation and Governance

Pitfall 10: Insufficient Data Licensing

When it comes to data, having sufficient licensing is critical. Using unlicensed data for your use case is the quickest way to derail a system just as it is about to launch. Sometimes, developers will take liberties with data in the name of exploration, stating "I am just seeing if this approach will even work first." As time goes on, the solution is built using this "temporary" data while the sales and marketing teams run with it, without knowing that the underlying data licensing has not been resolved. At this point, hopefully, the data licensing problem surfaces before users are onboarded to the system. In the worst case, you find the issue when the data owners bring legal action against your organization. To prevent this, it is imperative to have a final audit (or even better, periodic audits) to review all the data being used to build the system. This audit should also include validation of third-party code packages, because this is another area where licensing tends to be ignored for the sake of exploration.

Pitfall 11: Not Having Representative Ground Truth

This pitfall relates primarily to the role data plays in training a machine learning system. Specifically selected data will serve as the system's ground truth, which means the knowledge it will use to provide its answers. It is important that your ground truth contains the necessary knowledge to answer these questions. For instance, if you are building the aforementioned daytime and nighttime classifier but your ground truth does not include any nighttime images, it will be impossible for your model to know what a nighttime image is. In this case, the ground truth is not representative given the target use case, and it should have included training data for every class you wished to identify.

Pitfall 12: Insufficient Data Security

For information to be useful, it has to satisfy three major conditions: confidentiality, integrity, and accessibility. In practice, however, accessibility and integrity overpower confidentiality. In order to ensure legal, ethical, and cost-effective compliance, security should not be an afterthought, especially for your data storage systems. Data stores should be carefully designed from the start of the project. Data leakage can lead to major trust issues among your customers and can prove to be very costly. Companies have gone bankrupt over inefficient security. Customer data should be stored only in an encrypted format. This will ensure that even if the entire database is leaked, the data will be meaningless to the hackers. It should be confirmed that the encryption method that is selected has sufficient key strength and is used as an industry standard, like RSA (Rivest–Shamir–Adleman) or Advanced Encryption Standard (AES). The key size should be sufficiently long to avoid brute-force attempts; as of this writing, anything above 2,048 bits should be sufficient. The keys should not be stored in the same location as the data store. Otherwise, you could have the most advanced encryption in the world and it would still be useless.

Employees also need to be trained in security best practices. Humans are almost always the weakest link in the chain. Spear phishing is the technique of using targeted phishing scams on key persons in the organization. Such techniques can be thwarted only through

adequate training of personnel. It is important to include not only employees but also any contract resources that you are using to ensure that they are trained in the best security practices. Training and hardening your organization's managers, engineers, and other resources, just like your software, is the best way to avoid security compromises.

Computer security is a race between hackers and security researchers. In such a scenario, one other critical component to winning is to patch everything as soon as possible. Auditing your infrastructure and servers by professional penetration testers will go a long way in achieving your organization's security goals. These specialists think like hackers and use the same tools that are used by hackers to try to break into your system and give you precise recommendations to improve your security. Although getting security right on the first attempt might not be possible, it is nonetheless necessary to take the first steps and consider security from the beginning of the design phase.

Pitfall 13: Ignoring User Privacy

Dark designs are design choices that trick the user into giving away their privacy. These designs work in such a way that a user might have given consent for their data to be analyzed/stored without the user understanding what they have consented to. Dark design should be avoided on an ethical and, depending on your jurisdiction, legal basis. As the world progresses into an AI era, more data than ever is being collected and stored, and it is in the interest of everyone involved that the users understand the purposes for which their consent is being recorded. A quick way to judge whether your design choices are ethical is to check whether answering “no” to data collection and analysis imposes a penalty on the user beyond the results of analysis.

If third-party vendors are used for data analysis, it becomes imperative to ensure that anonymization of the data has taken place. This is to lessen the likelihood that the third party will misuse the data. With third-party vendors, it becomes necessary to take further measures like row-level security, tokenization, and similar strategies. Conducting software checks to ensure that the terms of a contract are upheld is very important if third parties are going to be allowed to

collect data on your behalf. Cambridge Analytica abused its terms of service as Facebook merely relied on the good nature and assumed integrity of Cambridge Analytica's practices. Having software checks that ensured third parties could access data only as defined in their contracts would have shortened Cambridge Analytica's reach by a huge amount, as it would not have been able to then collect data on the friends of people taking their quizzes.

Respecting users' rights and privacy in spirit is a process. Although it might be costly, it is necessary given the amount of data that is now possible to collect and analyze. When fed into automated decision-making AIs, these large amounts of data have the potential to cause indefinite and undue misery. It is in this interest that it becomes necessary to implement policies that make the user aware of how their data is being collected, how it will be analyzed, and most importantly, with whom it will be shared.

Pitfall 14: Backups

Although most people today understand the importance of backups, what they often fail to do is implement correct backup procedures. At a minimum, a good backup plan should involve the following steps: backing up the data (raw data, analyzed data, etc.), storing the backup safely, and routinely testing backup restorations. This last step is frequently missed and leads to problems when the system actually breaks. Untested backups fail to recover lost data or produce errors and require a lot of time to restore, thus costing the organization time and money to fix the problems. To resolve this, you should routinely restore full backups and ensure that everything still works while operating on the backup systems. A full data-restore operation should be undertaken on preselected days of every year, and all live systems should be loaded with data from the backups. Such a mock drill will identify potential engineering issues and help locate other problems as well, enabling you to develop a coherent and reliable restoration plan should the actual need for one ever arise.

With cloud storage becoming so commonplace, it is essential to remember that the cloud is "just another person's computer" and it can go down, too. Although cloud solutions are typically more stable than a homegrown solution because they are able to rely on the economies of scale and the intelligence of industry experts, they

can still have issues. Relying only on cloud backups may make your life easier in the short term, but it is a bad long-term strategy. Cloud providers could turn off their systems. They could have downtime when you need to do that critical data recovery procedure. It is therefore necessary to implement off-site and on-site physical storage media backups. These physical backups should also be regularly tested and the hardware regularly upgraded to ensure that everything will work smoothly in the case of a disaster.

All data backups should be encrypted as well. This is especially important to prevent a rogue employee from directly copying the physical media or grabbing it to take home. With encrypted backups, you will have peace of mind and your customers will sleep soundly, knowing their data is safe.

Step 4: Prototyping

Pitfall 15: Spending Too Much Time Planning

Although the majority of this chapter dealt with how to break down the prototype requirements and select technologies, it is important not to dwell too much on designing and planning your solution. Given that you will be using an Agile approach and starting a feedback loop as soon as possible, design changes can happen quickly. The start of the project is the point when you necessarily have the least amount of information known. Therefore, it only makes sense to start sooner than later, gaining knowledge by implementing and updating your design as you go. In the end, you will be able to create value more quickly using this approach.

Pitfall 16: Trying to Prototype Too Much

Another frequent pitfall that developers run into during the prototyping phase is setting themselves up for failure by trying to implement too much. A prototype should be limited in scope, provide real value, and be realistically feasible. There will be plenty of time to build large, complex, even moonshot systems once the prototype has been built. However, the prototype is your time to demonstrate value and prove to the stakeholders that AI systems are worth the investment. A prototype that takes too long or that is too ambitious and fails will

hurt your organization's chances of ever transforming into an AI-integrated business.

Continuing the chatbot example, it is important to include only a few types of chat interactions during the prototyping phase. For instance, if you are building a chatbot for a movie theater chain, perhaps the prototype version would only handle the ticket purchasing flow. Concepts such as refunds or concessions should be deferred until the production phase. In this way, the prototype can demonstrate the concept and value of purchasing tickets with the understanding that the other interactions can be added later with further investment.

Pitfall 17: The Wrong Tool for the Job

The other common problem is correctly identifying a problem but then assuming it can be solved with the technology du jour. During the technology selection process, you have to ensure that currently popular technologies do not cloud your judgment. Otherwise, at best you will have a needlessly more complex solution. At worst, you will need to replace a core technology midway through development. If your problem requires a hammer, it does not matter how awesome and new that shovel is, it is not the right tool for the job.

With regard to AI, this frequently happens with the misapplication of neural networks. Although it is true that neural networks can solve a large class of problems, it is not the right solution for every problem. For example, naïve Bayes can be a better approach when you do not have a large amount of data. Additionally, if you are in an industry that has to be able to explain its results, neural networks (especially large ones) are notorious for being opaque. They might be accurate given the training data, but because the features it learned are a complex combination of the inputs it is impossible to give a coherent reason why it made the decision it did.

Step 5: Production

Pitfall 18: End Users Resist Adopting the Technology

This pitfall is common with all new technology but especially with AI solutions. Automation technology can be unsettling for end users, since it replaces some of the work they are used to doing themselves. Opinions

range from “This technology will just be a hindrance to how I work,” to “It’s only a matter of time until my skills are obsolete, the robots take over, and I’m out of a job.” Change is hard, no matter what form it takes.

Another issue with AI solutions in particular is that most AI systems require input from a subject matter expert (SME) to create the ground truth used to train the underlying machine learning models. These SMEs are also typically the ones directly affected by the integration of a new AI solution. For many reasons, it is important that the AI solution be an augment to the SMEs’ knowledge and capabilities, instead of a direct replacement of their role. Remember that a machine learning model is only as good as the ground truth used to train it.

To avoid this pitfall, early end user engagement is critical. End users need to be part of the planning process to ensure that they fully understand the solution and feel they have contributed to the end product. This might even mean inviting a few influential end users during the Ideation/Use Case phases to build excitement and a voice for your user base. While early input is in no way a guarantee (end users might think they want one thing at first, only to realize they need something else once they start using the solution), it will help mitigate the fear associated with adopting new technology.

Pitfall 19: Micromanaging the Development Team

Under Agile, the development team is given full responsibility for the successful technical implementation of the project. The team works on the combined values of transparency and mutual trust. In such an environment, it would not be prudent to control every aspect of the development team. Nor would it be a good practice to set the targets for each sprint for the developers. This will lead to a lack of motivation, weakening Agile. The development team should be able to focus on the project by themselves with minimal intervention from the product owner. Some teams pick the Scrum manager from among the dev team as well. This is to ensure that the benefits of Agile are preserved.

Pitfall 20: Not Having the Correct Skills Available

Since building a machine learning system requires a number of specialized skills, it is critical to have these skills available and ready to go before your project starts. Whether this means hiring full-time

employees or establishing relationships with contracting firms, it is worth the up-front effort to avoid delays. Skills we have mentioned thus far that will be required include AI, data science, software engineering, and DevOps. The hiring issue is twofold in that you must find the individuals with the proper skillsets for your project and, of course, have the appropriate budget in place to fund them. With these items addressed, there should not be any skill roadblocks in the way of getting your system deployed.

Thriving with an AI Lifecycle

Pitfall 21: Assuming a Project Ends Once It Is Implemented

Quite a few project managers and businesses assume that once a project is implemented, the project is done. This is incorrect. Once released, projects are subject to entropy, like everything else, and will start decaying quickly unless maintained. This means that the project plan should include some developer time for postimplementation bugs and improvements. The allocation need not be more than 10 percent of the total implementation time, but it can make all the difference between a successful implementation and a failed one. It is also important to limit the scope of these fixes so as to not derail the existing system as a whole. Additionally, there can be other complications with implementation in large projects, like a last-minute hardware failure, and such complications will also need to be managed.

Pitfall 22: Ignoring User Feedback

After implementation, it is vital to collect and analyze user feedback. Ignoring feedback will doom most projects to eventual failure. Users who feel like they are not being heard may fail to utilize the software's full benefits or resist using the software altogether. This reluctance to change can be justified, and in such a case, the only option can be to tweak the released software. A heavy-handed approach that fails to consider users' needs will hamper productivity and efficiency, ultimately wiping away gains made by the implementation of the new software. Forcing users to use broken or incomplete software is not a sustainable option—eventually, the users will

abandon the software entirely, tarnishing your organization's reputation. Software is made for the users, so gathering their input and making changes based on their feedback is a critical requirement for success.

Pitfall 23: Providing Inadequate User Training

Software can be made user-friendly only up to a certain point. To ensure that users can utilize the software correctly and receive the maximum benefit, it is vital to train them to use the new software correctly. There is a good chance that the majority of AI projects will be used by business analysts and the like, having little to no knowledge of computer science. It then becomes important to explain the tool they will be using to do their jobs properly. Training should be more than just quickly explaining the menus and the UI. It should be a full introduction to the software and how it can positively impact a user's day-to-day work. Training should also span a few days, rather than just one sitting, to help users better absorb the information and give them an opportunity to ask questions. Well-trained employees can do their job more efficiently and accurately by making the best use of the software tools provided to them.

- 20 percent rule (Google), 24
- 100 percent test coverage, 124
- A
- Acceptance, 125f
 - tests, 124
- Access
 - control systems, 87
 - methods, 75–76
- Accountability, 181
- Ackerson, Chris, 169–172
- Advanced Encryption Standard (AES), 91
- Agile, 101
 - framework, 103
 - application, 164
 - process, 68
 - term, usage, 48
 - user stories, generation, 59
 - weakening, 136–137
- Akolkar, Rahul, 183–186
- Algorithms, public
 - appearance, 159
- AlphaSense, 170
- Amazon Web Services (AWS), 111, 167
- Analytics software, 82
- Apache Hadoop, usage, 8, 85
- Apache JMeter, 132
- Application code, promotion, 121f
- Application programming interface (API), 75
 - internal APIs, 112
 - programmatic calls, 124
 - usage, 110–111
- Artificial Intelligence (AI)
 - adoption, 6–10, 98, 172
 - challenge, 170, 174, 176, 178, 181, 184, 188
 - roadmap, 7f
 - advances, 171, 174, 176, 178, 181, 185, 188
 - AI-focused meetups, 144
 - algorithm, usage, 6
 - assistance, job function
 - target, 172, 174, 176, 178, 182, 186, 188
 - benefits, 175
 - Bin-Picking product, model
 - reliance, 3

- capabilities, 64
 - overestimation, problem, 45, 196
- chatbots, avoidance, 43
- Cloud Services catalog, sample (IBM), 111f
- data consumption, 58–59
- deep learning (relationship), Venn diagram (usage), 21f
- development, 157
- expertise, 184, 187
- experts, 169
- focus, narrowness, 44
- implementation, process, 164–165
- lifecycle, 10, 51, 139, 167–168
 - pitfalls, 159–161, 206–207
 - roadmap action checklist, 193
- limitations, 41–44
- pitfalls, 44–45
- potential, 163
- power/responsibility, 158–159
- project plan, building, 64–66
- security, 177–178
- solution, outsourcing, 100–101
- strong/weak artificial intelligence, 42
- technology, data scientist application, 82
 - training AI models, data availability, 73f
- Artificial Intelligence (AI) models
 - comments, 154
 - data availability, 73f
 - description, 151, 153
 - files, 152, 153
 - hosted demo, 153
 - library
 - building, 150–155
 - components, 151–153
 - entry, example, 153–154
 - indexing mechanism, 151
 - solutions, 154–155
 - licensing, 152, 154
 - metrics, 152, 153
 - parameters, 152, 154
 - performance, quantification, 145–147
 - tagging, 151
 - tags, 153, 154
 - technology, 152, 154
 - testing, example, 125–127
 - training data, 152, 154
 - validation data, 152
- Artificial Intelligence (AI) system
 - creation, 76
 - human intervention, 129–131
 - improvement, 144–145
 - information flow, 30f
 - knowledge, 71
 - learning process, 142–144
 - output generation, 82

- robust AI system,
 - ensuring, 128–129
- Asset, Liability, Debt and Derivative Investment Network (Aladdin)
 - software,
 - deployment, 5–6
- Automated chat support
 - system, 128–129
- Automated testing, 124–128
- Awesome Public Datasets, 78
- Azure (Microsoft), 167

B

- Babbage, Charles, 13–14
- Back propagation (back-propagation), 18, 42
 - neural network, 22
- Backups, usage, 93–94, 202–203
- Bad data (AI limitation), 43
- Balanced ground truth, 72–73
- Balanced scorecard,
 - perspectives, 63
- BERT model, 170, 171
- Binary files, 150
- Black box, 42
- BlackRock, Inc., case study, 5–6
- Bootstrapping, 79
- Boundaries, 58–69
- Bradford, Jeff, 173–174
- Brainstorming, usage, 38–41
- Business

- business-critical code,
 - release, 156
- process mapping, 27–28
- stakeholders,
 - categories, 32

C

- Cambridge Analytica,
 - practices, 92–93, 202
- Capital allocation, group (filtering example), 36
- Categorizing, 34–37, 79
- Cause and effect (AI limitation), 42
- Chance encounters,
 - value, 38–41
- Change
 - Agile process, 68
 - proactive policy, 25
 - request procedures,
 - neglect, 198–199
 - request process, 52
- Chaos Monkey code (Netflix), 128
- Charges/budgets, 52
- Chatbots, 23, 130
 - architecture, sample, 131f
 - login interface, 50
 - product comparisons/
 - reviews, 98
 - support chatbot, logical architecture, 107f
 - technologies, sample, 108t
 - usage, hybrid approach, 130–131

- Chief Technology Officer (CTO), process map creation, 33
- Chunking, 17
- Classifier
 - confusion matrix, ample, 146f
 - image classifier, building, 79–80
 - machine model, 72
- Classifying, 35–37
- Client-dependent
 - variables, requirement, 4
- Cloud
 - APIs, 110–111
 - APIs, SLA, 135
 - cloud-based API model, 111
 - deployment paradigms, 133–135
 - provider, 135
 - scalability, relationship, 132–133
 - solutions, 202–203
- Cloud Engine Learning Engine (Google), 132–133
- Cloud Machine Learning Engine (Google), 111
- Cloud Platform (Google), 167
- Code base, parts (identification), 155–156
- Code defect, appearance, 127
- Code repository, creation, 118
- Comma-separated value (CSV)
 - files, 78
 - format, 75
- Computer security, 92, 201
- Conclusion, forming, 38
- Confusion matrix, sample, 146f
- Constructive criticism, destructive criticism (contrast), 39
- Constructive feedback, 83–84
- Containers, 134
- Continuous integration,
 - 119–123
 - benefit, 123
 - pipeline, 119–121
 - principles, 123
 - true continuous integration, 121–123
- Convolutional neural network (CNN), 23
- Core competency, 99
- Co-relations, AI usage, 158
- Core technology, replacement, 114
- Cost-benefit analysis, usage, 38
- Creativity, fostering, 24
- Cross-departmental exchanges, 40–41
- Cross-disciplinary knowledge base, 148
- Crowdsourcing, platform/usage, 79–80
- Culture, focus, 44–45, 196

- Customer
 - balanced scorecard
 - perspective, 63
 - interaction, reduction, 65
 - satisfaction
 - increase, 65
 - KPI example, 62
 - user stories, 65
- Customer relationship
 - management (CRM)
 - system, 84
- D**
- Daily stand-ups (team role), 102–103
- Dark designs, 92
- Data, 173–174
 - access, 84–85
 - analysis, third-party
 - vendors usage, 92–93
 - anonymization, 87
 - availability, 27
 - bad data (AI limitation), 43
 - collation, 29
 - consumption, 58–59
 - curation (technology adoption phase), 8, 71, 157, 166
 - pitfalls, 90–94, 199–203
 - roadmap action
 - checklist, 192
 - tasks, 81
 - data-powered
 - capabilities, 110–111
 - data-restore operation, 202
 - exploration, 74–75
 - governance, 166
 - roadmap action
 - checklist, 192
 - improvement, 157–158
 - licensing
 - disadvantage, 79
 - insufficiency, 90, 199
 - non-free data, licensing
 - (disadvantages), 78–79
 - readiness, 89–90
 - recovery, untested backup
 - failure, 93
 - responsibility, 89
 - science flow, 82f
 - scientist
 - AI technology
 - application, 82
 - role, 81–82
 - security, insufficiency, 91–92, 200–201
 - storage technology, 82
 - temporary data, usage, 90
 - text-based data, 154–155
 - training data, 152, 154
 - transformation, tasks, 81
 - tweaking, 9
 - validation data, 152
- Database structure, reverse engineering, 75–76
- Data collection, 73–80, 180–181
 - crowdsourcing, usage, 79–80
 - licensing, usage, 77–79
 - opt-in/opt-out, 86
 - policies, 86

- Dataflows, tracking, 29–30
- Data.gov, 78
- Data governance (technology adoption phase), 8, 71, 85–89, 157
 - board, creation, 87–88
 - completion, goal, 86
 - initiation, 88
 - pitfalls, 90–94
- Datasets, labeling, 80
- Decision tree process, 130
- Deduction complexity, compounding, 4
- Deep learning, 21, 82, 152
 - AI relationship, Venn diagram (usage), 21f
 - models, usage, 3
- Defect tracking, digitization, 57
- Deliverables
 - defining, 50
 - list, 51
- Delphi method, 7, 60, 166
- Delphi technique, usage, 104
- Demo, production, 103–104
- Departmental cross-talks, implementation, 40–41
- Design thinking, 7, 53–58, 61, 166
 - process, 54f, 64–65
 - session, sample, 53–54
- Developers, code conflict (absence), 119
- Development
 - environment, 119
 - promotion, 120
- feedback loop, 105–106
- team
 - micromanagement, 136–137, 205
 - role, 102
- Digital internal data
 - collection, 74–76
- Digital neuron, firing, 20–21
- Direct database connection, 75–76
- Docker (container technology), 134
- Documentation, quality, 151
- “Do Not Track,” browser requests, 85
- Duesterwald, Evelyn, 177–179
- E**
- Electromagnetic interference (EMI), external stressor, 59
- ELIZA, usage, 16
- Embedded code, ubiquity, 1
- EMNIST dataset, 72
- Empathy, AI limitation, 43–44
- Empathy maps
 - creation, 55–56
 - generation, 65
 - sample, 56
- Employee
 - impact, group (filtering example), 36
 - performance (KPI example), 62

- time, saving (KPI example), 62
- turnover/attrition, 147
- Encryption, 86
- Epochs, number (deep learning), 152
- Error cost escalation, 107–108
- Explainability, 42, 181
- External stressors, mitigation, 59
- F**
 - F1 score, 147
 - Failover protocol, 129–130
 - Fallback capability, 130
 - False positive/negative, 145
 - FANUC Corporation
 - case study, 2–3
 - robot, example, 4f
 - FANUC Intelligent Edge
 - Link & Drive (FIELD), usage, 3
 - Feedback
 - constructive feedback, 83–84
 - gathering, 103
 - ignoring, 160
 - loops, 82–84, 105–106
 - application, 121–122
 - continuation, 135
 - mechanisms, 141
 - qualitative feedback, 65
 - receiving, 142
 - stages/roles, 105f
 - user feedback, incorporation, 140–142
 - Fibonacci Sequence, 104
 - File export, 75
 - Filtering, 34–35
 - Filters, usage, 26
 - Financial balanced
 - scorecard perspective, 63
 - Financial success, measurability criteria, 51
 - Firm
 - discovery, 99–100
 - references, request, 100
 - First-mover advantage, achievement, 23–24
 - Flores, Steven, 187–189
 - Flowcharts
 - sample, 34f
 - usage, 28–29
 - Formal change request
 - procedures, defining (neglect), 68
 - Freelancer.com, 101
 - Friedl, Jeffrey, 16
 - Fully connected neural network, multiple layers, 21f- G**
 - General Data Protection Regulation (GDPR), 88–89, 166
 - Generalizations (AI limitation), 41–42

- GitHub, 128
- Goals
 - defining, 56–57
 - setting, 102
 - transformation, 57
- Goldstein, Rob, 6
- Google search results,
 - usage, 81
- Governance
 - data governance,
 - 8, 71, 85–89
 - term, application, 85
- Graphical processing units (GPUs), usage, 134–135
- Grinder, The, 132
- Ground truth, 71
 - absence, 90–91, 200
 - balanced ground
 - truth, 72–73
 - distribution, building
 - methods, 72
 - proportional ground
 - truths, 73
- Guide rails, presence, 129
- H**
- Hashing, 86–87
- Health Information
 - Technology for
 - Economic and Clinical Health Act (HITECH Act), 88
- Health Insurance Portability and Accountability Act (HIPAA), 88
- Hidden Markov models,
 - 19–20
- H&R Block, case
 - study, 4–5
- Hybridization, 118
- Hype (AI limitation), 43
- Hypercare, 139
- Hypothesis/predictions,
 - forming/testing, 38
- I**
- IBM
 - AI Cloud Services catalog,
 - sample, 111f
 - Watson, capabilities, 4
 - Watson Services, 111
- Idea
 - creation/discovery, 31
 - expected returns,
 - group (filtering example), 36
 - grouping, example, 37f
 - independent requests
 - (number), group (filtering example), 36
- Idea bank
 - control, 148
 - maintenance, 25–27
 - organization, 35–37
 - review, 37–38, 147–148
 - sample, 26t–27t
 - updating, 147–148
- Ideation, 165
 - focus, narrowness, 195
 - pitfalls, 44–45, 195–196

- process, problems,
 - 44, 195–196
 - roadmap action
 - checklist, 191
 - technology adoption
 - phase, 6–7, 13
 - If-this, then-that decision,
 - 130
 - Image classifier,
 - building, 79–80
 - Image recognition/
 - classification, 23
 - Imitation game, 14
 - Indexing mechanism, 151
 - Information
 - flows, 29–31, 30f
 - personal information, 77
 - tracking, 29–30
 - Infrastructure-as-a-service (IaaS), 133
 - “Infrastructure as code”
 - paradigm, 134
 - Infrastructure testing,
 - 127–128
 - Innovation
 - culture, impact, 44–45
 - innovation-focused
 - organization, 23–25
 - method, 38–39
 - priority, 25
 - Integration, 125f
 - tests, 124
 - Intelligent business
 - model, 164
 - Interactive Voice Response (IVR) system, 43
 - usage, 64
 - Internal APIs, 112
 - Internal data collection
 - digital component,
 - 74–76
 - physical component,
 - 76–77
 - Internal process, balanced
 - scorecard
 - perspective, 63
 - Internal systems, monthly
 - review meetings (incorporation), 141
 - Internet of Things (IoT), 1
 - devices, 74
 - usage, 157
 - Inverse document frequency, 18
- ## J
- Java-ML library, 111
 - JavaScript, usage, 110
 - Jenkins (CI tool), 123
 - Just-in-time (JIT) ordering, 2
- ## K
- Kaggle, 78
 - Key drivers, selection/
 - modeling, 60
 - examples, 62
 - Key performance indicators (KPIs), 48
 - measurability criteria, 51
 - Keys, 17

Knowledge base, 148–150
 cross-disciplinary
 knowledge base, 148
 expansion, 150
 materials, categories, 149
 online knowledge base,
 features, 149
Knowledge sharing, 141
Kubernetes, 134

L

Language-independent
 method, usage, 110
Learning chatbot, project
 creation, 65
Learning/growth,
 balanced scorecard
 perspective, 63
“Lessons learned” document,
 usage, 145
Licensing, usage, 77–79
LoadRunner, 132
Load testing, usage, 132
Load tests, 131
Logical architecture diagram,
 usage, 106
Logic, explainability, 42
Long short-term memory
 (LSTM) neural net-
 works, usage, 22–23
Lovelace, Ada, 13–14

M

Machine learning, 18–19, 82

 models, 121, 132–133
 precision, definition, 146
 system, building, 205–206
Machine Learning on Azure
 (Microsoft), 111
Machine model, 72
Mailing lists, feedback
 mechanisms, 141
Market research, usage,
 54–55
Markov chains, 19
Markov models, 19
 hidden Markov models,
 19–20
Markov property, 19
*Mastering Regular Expres-
 sions* (Friedl), 16
Measurability criteria, 51
Metadata, curation, 5
Microservices, 110–111
Models. *See* Artificial
 Intelligence models
Monthly report, replacement,
 57
Multiserve deployment, 128
Mutual trust, value, 136–137

N

Naive Bayes approach, 204
Narrow AI, definition, 171
Natural language processing
 (NLP), 15, 82
 application, 5
 BERT model, 170, 171
 machine learning, 17

- programmatic NLP, 15–17
- statistical NLP, 17–18
- Need-based approach, 40
- Nephew, Jill, 42, 179–183
- Networked devices, 74
- Neural networks, 20–23
 - convolutional neural network (CNN), 23
- impact, 114
- single neuron example, 20f
- Nightly build, running, 120
- NodeJS application, 111
- Non-free data, licensing
 - (disadvantages), 78–79
- Numerical analysis, 82

O

- Observation, making, 38
- OneSignal, profitability, 77
- Online knowledge base,
 - features, 149
- Open source, 168
 - community, leveraging, 156
 - contribution, 155–156
 - projects, impact, 156
 - technologies, usage, 98
- Operating system
 - (OS), copy, 134
- Organizational chart,
 - enhancement, 28f
- Organizational flowcharts,
 - usage, 28–29
- Out-of-the-box IoT
 - equipment, usage, 2–3
- Overfitting, 84

P

- Palantir, founding, 158
- Parameter tweaking, 9
- Performance
 - AI model performance,
 - quantification, 145–147
 - benchmark level,
 - defining, 51–52
 - testing, 120
 - tests, 131
- Periodic updating,
 - facilitation, 143
- Permission structures, 10
- Personal information, 77
- Personas
 - determination, 54–55
 - development, 64–65
- Physical architecture, 108f
 - diagram, 107
- Physical data, value, 77
- Physical internal data
 - collection, 76–77
- Precision, term (usage), 146
- Present circumstances,
 - assessment, 60
- Probability, 17
- Problems, invention/
 - misrepresentation, 67
 - pitfalls, 197
- Process
 - flowchart, 34f
 - map, CTO creation, 33

- Production, 117, 167
 - code defect, appearance, 127
 - code repository,
 - creation, 118
 - environment, 119
 - model, promotion, 122f
 - pitfalls, 135–137, 204–206
 - roadmap action checklist, 193
 - skills, absence, 137
 - team/schedule, 30
 - technology adoption
 - phase, 9–10
- Product owner (team role), 101
- Products, tagging/
 - categorization, 34
- Programmatic NLP, 15–17
- Programming language,
 - selection, 109–110
- Programming techniques,
 - usage, 16
- Project
 - AI pieces, process, 118
 - assumptions, 51
 - breakdown,
 - approaches, 53–61
 - completion/assumption, 160, 206
 - completion criteria, 51–52
 - defining (technology adoption
 - phase), 7–8, 47, 165–166
 - pitfalls, 196–199
 - roadmap action
 - checklist, 191–192
 - definition, pitfalls, 66–68
 - deliverables, list, 51
 - focus, problem, 44–45, 196
 - governance, 49–50
 - kickoff, 50
 - measurability, 62–63
 - oversight activities, 49–50
 - plan
 - building, 64–66
 - components, 48–52
 - metrics, 48
 - roadmap, clarity, 103
 - scope, 49
 - stakeholders, feedback, 97
 - success criteria, 65
 - work schedules/
 - locations, 49
- Proportional ground truths, 73
- Prototype
 - code, leveraging, 117–118
 - design, 106–107
 - reuse, 117–118
 - technology scales,
 - ensuring, 131–133
- Prototyping (technology adoption phase), 8–9, 97, 166–167
 - excess, problem, 113, 203–204
 - pitfalls, 112–114, 203–204
 - planning, problem, 113

- roadmap action
 - checklist, 192–193
 - solutions, 97–99
 - tool, problem, 113–114
 - Python
 - HTTP library, 111
 - selection, 109–110
 - PyTorch, 151
- Q**
- Qualitative feedback, 65
 - Quality assurance (QA)
 - specialists, 56, 57
 - team, monitoring activity, 55
- R**
- Ranking, 17, 35–37
 - Recall
 - equation, 146
 - metric, 146–147
 - Recurrent neural network (RNN), 22–23
 - Red Hat, 156
 - Reports accuracy (KPI example), 62
 - Representational state transfer (REST) API, 110–111
 - Research, performing, 38
 - Results, iteration/sharing, 38
 - Risk, group (filtering example), 36
 - Rituals, importance, 44
 - Rivest-Shamir-Adleman (RSA), 91
 - Robinson, Nathan S., 175–177
 - Robust AI system,
 - ensuring, 128–129
 - Row-level security, 92, 201
- S**
- Safari Books Online, 149
 - Salesforce (CRM system), 84
 - Scalability
 - cloud, relationship, 132–133
 - handling, 122
 - testing, 120
 - Scenario planning/
 - analysis, 60, 61
 - Schedule, 30, 49
 - estimation, 52
 - Schwaber, Ken, 101
 - Scientific method,
 - usage/steps, 38
 - Scrum
 - framework, 103
 - master (team role), 102
 - overview, 101–103
 - Service level agreement (SLA), 135
 - Shortcuts, coding, 123
 - Single sign-on (SSO), 10
 - Smoothing, inclusion, 19
 - Social media, usage, 67
 - Sociological, technological,
 - economic,
 - environmental,
 - and political inputs
 - (STEEP) model, 60

- Software as a service (SaaS)
 - architecture,
 - usage, 123
 - Software checks, conducting, 92–93, 201–202
 - Solutions, 97–99
 - premature construction, 67–68, 198
 - Sorting, 34–35
 - Source control, code conflict (absence), 119
 - Spear phishing, 91
 - Specification document, design (problem), 49
 - Sprint
 - planning (team role), 102
 - review, 103
 - Stage environment (test environment), 119
 - testing, 121
 - Staging environment, 122–123
 - Stakeholders
 - buy-in, absence, 66, 196–197
 - categories, 32
 - feedback, 97
 - gathering, 103
 - Standard operating procedures (SOPs), 28–29
 - Statistical NLP, 17–18
 - Step size (deep learning), 152
 - Story
 - points, usage, 104
 - user stories, 57–58, 65
 - Strong artificial intelligence, 42
 - Subject matter experts (SMEs), 51–52, 72, 136, 205
 - feedback, 83
 - usage, 76
 - Subsystems, usage, 59–60
 - Success, criteria, 50, 65
 - absence, 68, 199
 - Support chatbot
 - logical architecture, 107f
 - physical architecture, 108f
 - Support query, resolution (KPI example), 62
 - Sutherland, Jeff, 101
 - Switches, 163
 - System
 - bootstrapping, 79
 - building, decision point, 109
 - confidence, erosion, 129
 - power, leveraging, 81
 - Systems planning, 7, 61
 - Systems thinking, 58–60
- ## T
- Tagging, 34
 - format, 151
 - Tags, 153
 - selection, 37
 - Talent, employing/contracting (contrast), 99–101
 - Technological evaluation, 9
 - Technology, 144–145
 - adoption
 - end user resistance, 136

- fear, mitigation, 205
 - phases, 6–10
 - selection, 107–110
 - chatbot technologies, sample, 108t
 - Temporary data, usage, 90
 - TensorFlow, usage, 109–110, 151, 155
 - Term frequency, 18
 - Term frequency-inverse document frequency (tf-idf), 17
 - reliance, 19
 - Test-driven development (TDD), 10
 - Test environment, 119
 - Testing frameworks, 10
 - Test types, 124–125
 - Text-based data, 154–155
 - Text generators, 23
 - Text-to-speech capability, provision, 67
 - Thiel, Peter, 158
 - Think Fridays (IBM concept), 24
 - Third-party vendors, usage, 92–93
 - Time, group (filtering example), 35–36
 - Tokenization, 17, 92, 201
 - Tone analysis technology, 130
 - Training
 - AI models, data availability, 73f
 - data, 152, 154
 - Transparency, value, 136–137
 - Travis CI, 123
 - True continuous integration, 121–123
 - True positive/negative, 145
 - Turing, Alan (Turing test), 13–15, 42
 - standard interpretation, 14f
- U
- Unit testing, 125f
 - Unit tests, 124
 - creation, 125
 - Upwork, 101
 - User
 - password hashing, 86–87
 - privacy, ignoring, 92–93, 201–202
 - rights/privacy, respect, 93
 - training, inadequacy (provision), 160–161, 207
 - type, defining, 55
 - User experience (UX)
 - skills, 100–101
 - User feedback
 - ignoring, 160, 206–207
 - incorporation, 140–142
 - User/security model, 9–10
 - User stories
 - accomplishment, 102
 - defining/creating, 57–58

- development team
 - construction, 117
- establishment, 65
- prioritization, 97, 103–104

V

- Validation, defining, 120
- Value
 - analysis, 31–34, 104
 - example, 33–34
 - concept, 31–32
 - defining, 104
 - identification
 - process, 32–33
 - monetary value,
 - relationship, 32
- Vendor selection, risk, 67–68
- Venn diagram, usage, 21f
- Verification, defining, 120
- Virtual machines (VMs), 133
- Visibility gaps, 157

- Visual defect identification
 - system, building, 56
- Visualization,
 - popularization, 133

W

- Walls, separation
 - ability, 40–41
- Weak artificial intelligence, 42
- Web-based APIs, usage,
 - 110–111
- WebLOAD, 132
- Weizenbaum, Joseph, 16
- Word2vec, development, 20
- Word embedding, 22
- WordPress (blog), 149, 154
- Work
 - schedules/locations, 49
 - stream activities, 50–52
- Workload, example (spike exhibit), 133f

WILEY END USER LICENSE AGREEMENT

Go to www.wiley.com/go/eula to access Wiley's ebook EULA.